Reproducible Polyglot Data Science

Bruno Rodrigues

2026-08-23

Table of contents

Welcome! **1**

A modern, unified, and language-agnostic workflow for data science using Nix and T. 1

Preface **5**

1 Introduction **11**

1.1 Who is this book for? 14
1.2 What is the aim of this book? 14
1.3 Prerequisites 17
1.4 What actually is reproducibility? 18
1.4.1 Using open-source tools is a hard requirement 19
1.4.2 Hidden dependencies can hinder reproducibility 21
1.4.3 The requirements of a RAP 22
1.4.4 Are there different types of reproducibility? 23

2 The Nix Package Manager **29**

2.1 Introduction 29
2.2 Why Reproducibility? Why Nix? 30
2.2.1 Motivation: Reproducibility in Scientific and Data Workflows 30
2.2.2 Problems with Ad-Hoc Tools 30
2.2.3 Nix: A Declarative Solution 31

2.3 Important Concepts 32
2.3.1 Derivations 33
2.3.2 Dependencies of derivations 34
2.3.3 The Nix store and hermetic builds 35
2.3.4 Other key Nix concepts 36
2.4 Caveats 38
2.5 In summary 40

3 Setting Up Your Environment 41
3.1 Installing Nix 41
3.1.1 For Windows Users: WSL2 Prerequisites . 41
3.1.2 The Determinate Systems installer 43
3.2 Already Have Nix? 44
3.2.1 Step 1: Add yourself as a trusted user . . 44
3.2.2 Step 2: Add the binary cache 45
3.3 Verifying installation with Temporary Shells . . . 47
3.4 Configuring your IDE 48
3.4.1 Why a text editor (and not a notebook)? . 48
3.4.2 Prerequisites 49
3.4.3 direnv 50
3.4.4 In summary and next steps 51

4 Reproducible Development Environments and Pipelines with T 53
4.1 Introduction 53
4.1.1 The Reproducibility Puzzle 58
4.1.2 Reproducibility Is Not Just About the Environment 59
4.1.3 The Mindset Shift: From Tasks to Rules . 60
4.1.4 Thinking in Systems in an AI-First World 62
4.1.5 The Polyglot Challenge 65
4.1.6 Enter T 65
4.1.7 A candid note: who this is for (and who it isn't) 68

4.2 A Quick Tour of T Syntax 70
4.2.1 Variables and assignment 71
4.2.2 Conditionals 71
4.2.3 The two pipes 71
4.2.4 Imports 72
4.2.5 Sequence generation 72
4.2.6 Operators at a glance 73
4.2.7 What can a T value be? 75
4.2.8 How T evaluates 76
4.2.9 Types, briefly 77
4.3 Transitioning to Nix-Managed R 77
4.3.1 Bootstrapping T without a local installation 78
4.4 The T Project 80
4.4.1 Declaring Dependencies with `tproject.toml` 80
4.4.2 Working with R, Python, and Julia 84
4.4.3 Alternative installation methods 84
4.4.4 System Dependencies and LaTeX 89
4.5 Summary 90

5 Your First T Pipeline 91
5.1 Introduction 91
5.2 How a T Pipeline Executes 91
5.3 The Simplest Possible Pipeline 92
5.4 The Core Development Loop 94
5.5 Node Types: Calling R, Python, and Julia 95
5.6 Serializers 97
5.7 A Complete Worked Example 98
5.8 Inspecting the Pipeline 100
5.9 Comparing Builds with `t diff` 102
5.10 What `t check` Catches 102
5.11 Interactive Development with the T REPL in Positron 105
5.11.1 Inspecting Live Nodes: `read_node()` . . . 105
5.11.2 Inspecting Historical Builds: `read_past_node()` 106

5.11.3 High-Fidelity Representation of Foreign Objects . . . 106
5.11.4 Stepping into Guest Runtimes: `debug_node()` (A Debugging Appetizer) . . . 107
5.12 Pointing Nodes at External Scripts . . . 109
5.13 Pairing with an AI Agent . . . 109
5.13.1 A worked example: building a pipeline with an agent . . . 110
5.13.2 Streaming build events with `t run --json` 117
5.14 Summary . . . 120

6 Pipelines as Function Composition 123
6.1 Introduction . . . 123
6.2 The Problem with State . . . 124
6.3 Pure Functions: The Mathematical Ideal . . . 125
6.4 T Nodes *Are* Pure Functions . . . 126
6.5 A Pipeline Is Function Composition . . . 128
6.6 Polyglot Composition: Passing Values Across Languages . . . 129
6.7 Errors as Values, Not Exceptions . . . 131
6.8 What This Means for Your R and Python Code . 133
6.9 Nodes Can Point to External Scripts . . . 133
6.10 Summary . . . 136

7 Professional Workflows: Testing and Git 139
7.1 Introduction . . . 139
7.2 Part 1: Unit Testing . . . 140
7.2.1 The Philosophy of a Good Unit Test . . . 140
7.2.2 Unit Testing in Practice . . . 141
7.2.3 Testing as a Design Tool . . . 147
7.2.4 The Modern Data Scientist's Role: Reviewer and AI Collaborator . . . 148

7.3 Part 2: Advanced Version Control 150
7.3.1 A Better Way to Collaborate: Trunk-Based Development 151
7.3.2 Collaborative Hygiene: Rebase vs. Merge . 152
7.3.3 Working with LLMs and Git: Managing AI-Generated Changes 153
7.4 Part 3: Testing T Pipelines with Testcraft 155
7.4.1 The `t test` Command 155
7.4.2 Agent- and CI-friendly output 156
7.4.3 The `expect_*` Primitives 158
7.4.4 Testing Node Outputs 159
7.4.5 Testing the Pipeline Structure 161
7.4.6 The Fixture-Chain Pattern 162
7.4.7 Summary of Testing Levels 163
7.5 Summary 164

8 Building T Pipelines 165
8.1 Introduction 165
8.2 A Realistic Worked Example 166
8.3 All Node Runtimes at a Glance 170
8.3.1 Shell Nodes: `shn()` 170
8.3.2 Julia Nodes: `jln()` 172
8.3.3 Quarto Nodes: `qn()` 173
8.4 Fetching Remote Data 175
8.4.1 Step 1: Prefetch the hash 176
8.4.2 Step 2: Use `fetchurl` in the pipeline . . . 176
8.5 Sharing Code Across Nodes 178
8.5.1 The `functions` Parameter 178
8.5.2 The `include` Parameter 180
8.5.3 Global Pipeline Options 181
8.6 Environment Variables in Nodes 182
8.7 Build Logs and Artifacts 183
8.7.1 Reading the Build Log 184
8.7.2 Copying Artifacts Out of the Store 185

8.7.3 Garbage Collection 185
8.8 Incremental Builds and Caching 186
8.8.1 Previewing Cache Hits 187
8.9 CI/CD with `pipeline_to_ga()` 188
8.10 Summary 191

9 Advanced Pipeline Patterns 193
9.1 Pipelines as Data 194
9.2 Querying the DAG 194
9.2.1 `pipeline_to_frame` 194
9.2.2 Structural queries 196
9.2.3 Column projection with `select_node` . . 197
9.3 Modifying Nodes Surgically 198
9.3.1 `filter_node` 198
9.3.2 `mutate_node` 199
9.3.3 `rename_node` 200
9.3.4 `swap` 201
9.3.5 `rewire` 201
9.3.6 A realistic "what-if" workflow 201
9.4 Set Operations 202
9.4.1 `union` 202
9.4.2 `difference` 203
9.4.3 `intersect` 203
9.4.4 `patch` 204
9.4.5 `prune` 204
9.4.6 A realistic use-case: shared ETL, swapped model 205
9.5 DAG-Aware Transformations 206
9.5.1 `upstream_of` 206
9.5.2 `downstream_of` 206
9.5.3 `subgraph` 207
9.6 Pipeline Composition 207
9.6.1 `chain` 208
9.6.2 `parallel` 208

9.6.3 `pipeline_of` meta-pipelines 209
9.6.4 Pipeline templates via lambdas 210
9.7 Dynamic Branching 211
9.7.1 `map_pattern` 211
9.7.2 `cross_pattern` 213
9.7.3 Selector patterns 214
9.7.4 Expanding and inspecting 216
9.8 Cross-Node Artifact Retrieval 217
9.9 Static Conditionals 218
9.9.1 `node_when` 219
9.9.2 `node_fork` 219
9.10 Custom Flakes Per Node 221
9.11 Validation 222
9.11.1 `pipeline_validate` 223
9.11.2 `pipeline_assert` 224
9.11.3 Pipeline state as DataFrames 224
9.12 Handling Ambiguous Dependencies 225
9.13 Summary 228

10 Debugging and Error Handling 231
10.1 Errors Are Values 232
10.2 Resilient Builds and Fail-Fast 233
10.3 Pipes: Propagate or Recover 234
10.4 Pattern Matching 236
10.4.1 Matching errors 237
10.4.2 Recovery patterns 238
10.5 When a Pipeline Node Fails 239
10.6 Inspecting a Failed Build 240
10.7 Reading Artifacts from R and Python 242
10.8 Interactive Debugging 244
10.8.1 The `t debug` Command 244
10.8.2 `debug_node()` from the REPL 245
10.8.3 Keeping the Debug Environment Reproducible 246

10.9 Polyglot Error Handling 247
10.10 Common Errors 248
10.11 Best Practices 249
10.12 Summary 250

11 Containerisation With Docker 251
11.1 Introduction 251
11.1.1 Spatial vs Temporal Reproducibility . . . 252
11.1.2 The Best of Both Worlds 253
11.1.3 Interactive Development vs Distribution . 254
11.2 Docker Essentials 254
11.2.1 Installing Docker 254
11.2.2 The Rocker Project and Image Registries . 256
11.2.3 Basic Docker Workflow 258
11.3 Making Our Own Images 260
11.3.1 A Minimal Dockerfile with Nix 260
11.3.2 Splitting Into a Reusable Base Image . . . 262
11.3.3 Making T Available in the Image 264
11.4 Publishing Images on Docker Hub 265
11.4.1 Sharing Without Docker Hub 266
11.5 What If You Don't Use Nix? 267
11.6 Building Docker Images Directly with Nix 268
11.6.1 Why Skip the Dockerfile? 268
11.6.2 A Basic Example 269
11.6.3 Using T with dockerTools 270
11.6.4 Benefits Over Dockerfiles 272
11.6.5 Layered Images for Efficiency 272
11.6.6 When to Use This Approach 273
11.7 Dockerising a T Pipeline 273
11.7.1 Step 1: The Dockerfile 274
11.7.2 Step 2: The Pipeline 275
11.7.3 Step 3: Build and Run 275
11.7.4 Alternative: Using dockerTools for T . . . 276
11.8 Summary 278

11.9 Further Reading 279

12 A Short Intro to Packaging Your Code in R and Python 281
12.1 Introduction 281
12.2 Part 1: Creating an R Package with `{usethis}` and `{devtools}` 282
12.2.1 Step 1: Project Setup 283
12.2.2 Step 2: Write and Document a Function . 283
12.2.3 Step 3: Add Unit Tests 285
12.2.4 Step 4: Check and Install 286
12.2.5 Step 5: Install from GitHub 287
12.2.6 Step 5bis: Install it locally 288
12.3 Part 2: Creating a Minimal Python Package with uv 289
12.3.1 Step 1: Project Setup with uv 289
12.3.2 Step 2: Write a Function and Declare Dependencies 290
12.3.3 Step 3: Add Unit Tests 292
12.3.4 Step 4: Build and Install 293
12.3.5 Step 5: Install from GitHub 295
12.4 Part 3: Integrating Your Packages into a T Pipeline 295
12.4.1 Adding an R Package 296
12.4.2 Adding a Python Package 296
12.5 Conclusion: The Packaging Mindset in the Age of AI 298

13 Continuous Integration with GitHub Actions 301
13.1 Introduction 301
13.2 Getting your repo ready for GitHub Actions . . . 302
13.3 Nix and GitHub Actions 304
13.4 Running a T Pipeline on GitHub Actions 306
13.4.1 Caching T pipeline outputs between runs 308
13.5 Running a dockerized workflow 310

13.6 Building a Docker image and pushing it to a registry 313
13.7 A Complete T Pipeline Workflow on GitHub Actions 314
13.7.1 Caching Pipeline Outputs Between Runs . 315
13.7.2 The Easy Way: `pipeline_to_ga()` 317
13.8 GitHub Actions without Nix 317
13.9 Advanced patterns 319
13.9.1 Caching with Cachix 319
13.9.2 Storing outputs in orphan branches 321
13.9.3 Creating workflow summaries 322
13.10 Conclusion 324

14 T for Data Manipulation 327
14.1 The core verbs 328
14.1.1 select(): choose columns 328
14.1.2 filter(): choose rows 328
14.1.3 mutate(): add or transform columns . . . 329
14.1.4 arrange(): sort rows 329
14.1.5 group_by() and summarize() 329
14.1.6 Composing verbs 330
14.2 Joins, duplicates, and conditional values 330
14.2.1 Joining tables 331
14.2.2 Duplicates and counts 331
14.2.3 Conditional values 332
14.2.4 Operators in data work 332
14.3 Reshaping and missing values 333
14.3.1 pivot_longer() and pivot_wider() 333
14.3.2 separate() and unite() 334
14.3.3 nest() and unnest() 334
14.3.4 Missing values 335
14.4 Working with strings 335
14.5 Dates and times 336
14.6 Factors and categorical data 337
14.7 Arrays 338
14.8 Formulas and models 339

14.9 Quick stats in T 339
14.10 Lenses: reaching the nested 341
14.11 Where to go from here 342

15 Beyond the Basics: T in Depth 343
15.1 The Interactive T: Magic Commands 343
15.1.1 Navigating your environment 344
15.1.2 Inspecting what you have 344
15.1.3 Timing and capturing work 345
15.2 Functions in Depth 346
15.2.1 Lambda syntax 346
15.2.2 Closures 347
15.2.3 Higher-order functions 347
15.2.4 Auto-quotation (`$param`) 347
15.3 The Type System 348
15.3.1 Lambda signatures 348
15.3.2 Two modes: repl and strict 349
15.3.3 How far it goes today 349
15.3.4 Semantic type names 350
15.4 Metaprogramming: Quotation and Quasiquotation 350
15.4.1 Capturing code: `to_expr` and `quo` 351
15.4.2 Unquoting: `!!` and `!!!` 352
15.4.3 Dynamic names and symbols 356
15.4.4 Non-standard evaluation 358
15.5 Data Access and Lenses 360
15.5.1 The `get()` built-in 361
15.5.2 Lenses 362
15.6 Introspection: `explain()` and `intent` 363
15.6.1 `explain()`: a value's self-description . . . 363
15.6.2 `intent`: code says what, intent says why . 365
15.7 Escaping to the Shell: `?<{ }>` 366
15.7.1 Running a command 366
15.7.2 Capturing the output 366
15.7.3 Changing directory 367

15.7.4 Multiline commands 367
15.7.5 Errors 368
15.8 Serializers in Depth 368
15.8.1 The built-in serializers 368
15.8.2 Symbols, variables, and the string trap . . 369
15.8.3 Custom serializers 371
15.9 Plotting in Pipelines 373
15.9.1 Returning a plot from a node 373
15.9.2 Reading a plot back 376
15.9.3 Plots in literate documents 377
15.10 Nix Build Options and Orchestration 377
15.10.1 The `nix_options` dictionary 378
15.10.2 Dry runs and plans 379
15.10.3 Where environment comes from 379
15.10.4 Building on other machines 379
15.11 Building T Packages 380
15.11.1 Initialising a package 380
15.11.2 Public and private API 381
15.11.3 Testing a package 381
15.11.4 T-Doc documentation 382
15.11.5 Publishing 382
15.11.6 Importing 382
15.12 Property-Based Testing with Propcraft 383
15.12.1 The idea 383
15.12.2 Generators 384
15.12.3 The property contract 384
15.12.4 A concrete example 385
15.12.5 Seeding and sample size 385
15.12.6 Running with `t test` 385
15.13 Where to go from here 386

16 Conclusion: Adopting Reproducibility in the Real World 387
16.1 Introduction 387

16.2 Start with yourself 388
16.3 The two-minute demo 388
16.4 Make it easy for others 389
16.5 Pick your battles 390
16.6 The gradual adoption path 390
16.7 Common objections and how to address them . . 391
16.8 LLMs as adoption accelerators 392
16.9 A note on literate programming 394
16.10 Final thoughts 395

References **397**

Welcome!

A modern, unified, and language-agnostic workflow for data science using Nix and T.

This is still WIP! Expected first finalized release at the end of Q4 2026. This book is a complete reimagining of my previous work, "Building Reproducible Analytical Pipelines with R." If you're looking for that book, you can find it here[1]. But if you're ready for the next step, you're in the right place.

Data scientists, statisticians, analysts, researchers, and many other professionals write a lot of code.

Not only do they write a lot of code, but they must also read and review a lot of code as well. They either work in teams and need to review each other's code, or need to be able to reproduce results from past projects, be it for peer review or auditing purposes. And yet, they never, or very rarely, get taught the tools and techniques that would make the process of writing, collaborating, reviewing and reproducing projects possible.

Which is truly unfortunate because software engineers face the same challenges and solved them decades ago.

[1] https://www.raps-with-r.dev

The aim of this book is to teach you how to use some of the best practices from software engineering and DevOps to make your projects robust, reliable and reproducible. It doesn't matter if you work alone, in a small or in a big team. It doesn't matter if your work gets (peer-)reviewed or audited: the techniques presented in this book will make your projects more reliable and save you a lot of frustration!

As someone whose primary job is analysing data, you might think that you are not a developer. It seems as if developers are these genius types that write extremely high-quality code and create these super useful packages. The truth is that you are a developer as well. It's just that your focus is on writing code for your purposes to get your analyses going instead of writing code for others. Or at least, that's what you think. Because in others, your team-mates are included. Reviewers and auditors are included. Any people that will read your code are included, and there will be people that will read your code. At the very least future you will read your code. By learning how to set up projects and write code in a way that future you will understand and not want to murder you, you will actually work towards improving the quality of your work, naturally.

The book can be read for free on https://b-rodrigues.github.io/reproducible-data-science/ and you'll be able to buy a DRM-free Epub or PDF on Leanpub[2] once there's more content.

This book is the culmination of my previous works. I started by writing a book focused on R, and then began working on a Python edition. During that process, I had a realization: tackling reproducibility one language at a time was solving the symptoms, not the root cause. The real solution needed to be universal,

[2]https://leanpub.com/

powerful, and capable of handling any language or tool we might need.

That universal solution is Nix.

This book moves beyond language-specific tooling. It presents a holistic workflow where R, Python, and Julia are not competitors, but collaborators in a single, cohesive, and perfectly reproducible environment. We will cover:

- The Nix Philosophy: Why Nix is the ultimate tool for solving the "it works on my machine" problem, once and for all: derivations, component closures, and hermetic builds.
- Declarative Environments with T: How to define exact, bit-for-bit reproducible software environments that include specific versions of R, Python, Julia, their packages, and any system-level dependencies, all in one simple file.
- Polyglot Pipelines with T: How to orchestrate complex analytical pipelines that seamlessly pass data between different languages, all managed by the Nix build system.
- Functional Programming and Pipelines as Function Composition: Core principles for writing robust, testable, and maintainable code, no matter the language.
- Professional Workflows: Unit testing, Git, and debugging techniques that keep your analyses reliable as they grow.
- Advanced Pipeline Patterns: Treating pipelines as data: querying, transforming, and composing them.
- Distribution and Automation: How to package your entire reproducible pipeline into a Docker container for easy sharing and automate your workflow with GitHub Actions.

While this is not a book for beginners (you should be familiar with at least one data-centric programming language before reading this), I will not assume that you have any knowledge of the tools

discussed. But be warned, this book will require you to take the time to read it, and then type on your computer. Type *a lot*.

I hope that you will enjoy reading this book and applying the ideas in your day-to-day, ideas which hopefully should improve the reliability, traceability and reproducibility of your code.

If you find this book useful, don't hesitate to let me know! You can submit issues, suggest improvements, and ask questions on the book's GitHub repository[3].

If you want to get to know me better, read my bio[4].

[3]https://github.com/b-rodrigues/reproducible-data-science
[4]https://brodrigues.co/about.html

Preface

In 2023, I wrote a book with a straightforward premise: by borrowing a few key ideas from software engineering, people who analyse data could save themselves a great deal of frustration. The response exceeded anything I had hoped for and confirmed a suspicion I'd had for some time: as a community, we are hungry for better ways to work.

That first book focused exclusively on the R ecosystem. One question came up again and again: "This is great, but what about Python?" It was a fair question. Data science is not a monologue; it's a conversation between languages. Writing a Python edition seemed like the obvious next step.

I mapped out the chapters and identified the equivalent tools: `pipenv` for dependency management, `ploomber` for pipelines, and started writing. But the deeper I went, the stronger a nagging feeling became. I was solving the same problems all over again, just with a different set of tools. The rapid churn within the Python ecosystem only reinforced that feeling. How many package managers have been created to manage virtual environments? As I write this, `uv` is all the rage. It may well be here to stay, but history suggests that a new contender is always waiting just around the corner.

This pointed to a larger issue. I became convinced that the future of data science is polyglot. An R user and a Python user could both follow the advice from my original book and

end up with reproducible projects, yet their workflows would remain fundamentally incompatible. They couldn't easily share an environment or build a single pipeline that leveraged the strengths of both languages. Companies such as Posit have made excellent progress in making it easier to call Python from R, but building a truly integrated development environment remains challenging. And what if you also want to bring Julia into the mix? It may not be as popular as Python or R, but it has its own strengths and deserves a place in the conversation.

I realised I was treating the symptoms, not the disease.

The root problem wasn't "How do I make R reproducible?" or "How do I make Python reproducible?". The real challenge was the lack of a universal foundation that could handle *any* language, *any* tool, and *any* system dependency with absolute, bit-for-bit precision.

That's when I stopped writing the Python book.

The solution wasn't to create another language-specific guide but to find a tool that operated at a more fundamental level. That tool, I am now convinced, is **Nix**.

Nix is not just another package manager; it is a powerful, declarative system for building and managing software environments. It allows us to define the *entire* computational environment (from the operating system libraries up to the specific versions of our R and Python packages) in a single, simple text file. When you use Nix, the phrase "it works on my machine" becomes obsolete. It is replaced by the guarantee: "it builds identically, everywhere, every time," with a few caveats that we will explore, of course.

Since discovering Nix, I've built several packages around it: `{rix}`, `{rixpress}`, and its Python counterpart, `ryxpress`. `{rix}` has been particularly successful and has received

contributions from many people. To me, that validated my original intuition: people want better tools, but they often need help getting started.

Nix has a steep learning curve for many reasons, so a package like `{rix}`, which provides a friendlier interface for R users, lowers the barrier considerably.

`{rixpress}` has had a more modest reception. I suspect that's because it asks users to fundamentally change the way they structure their work. `{rix}` simply generates a Nix file describing an environment. `{rixpress}`, by contrast, asks users to adopt reproducible pipelines. That is a much bigger behavioural shift. It draws heavily on `{targets}` (Landau 2021), and while `{targets}` is an outstanding package, pipeline-based workflows remain something of a niche. Most people still write long, sequential "spaghetti" scripts, and changing that habit takes more than good tooling.

But there is now a new opportunity which I think will truly change this: large language models (LLMs). I believe that LLMs will be a game changer and help democratize the adoption of tools like `{rix}`, `{rixpress}` and more broadly Nix. You, as the data scientist, can now 100% focus on the actual data science and leave setting up the reproducible environment and writing of the pipeline code plumbing to LLMs. These tasks are purely mechanical, low-value added (from a scientific or business perspective) and can also be easily verified: if the pipeline doesn't run and it's not because you made a mistake in the scientific or business logic code, then the issue comes from the pipeline code!

The idea, then, is that you shouldn't have to write that code anymore. LLMs can write most of it for you. That said, as capable as today's models are, they are far from perfect. They

make mistakes, and I also believe there is value in giving them tools that reduce the surface area for errors in the first place.

This is why I decided to build T.

T is a new programming language that I designed in OCaml and implemented entirely with the help of LLMs. I was the architect; they wrote the code.

T is a domain-specific language, just as R is a language specialised for statistics. T is specialised for pipelines. Its only purpose is to make writing, inspecting, debugging, and running pipelines as simple as possible.

More importantly, you're not really expected to learn T. In fact, you're not even expected to write it. T is designed to be used both by humans who want full control and by LLMs acting on their behalf.

Everything you can do with T, like running and inspecting pipelines visually in its command prompt, can also be executed by an LLM which will get the output in structured JSON, so it's easily actionable by the LLM. When starting a new project with T, it comes with an AGENTS.md and a language reference in markdown to help get LLMs started quickly.

T also makes it easy to coordinate R, Python and Julia, so you can use the best language for any task. You don't have to think much about how R will communicate with Python or Julia (or vice-versa), as T handles that for you.

Finally, the last thing that makes T quite unique, is that it is built on top of Nix: this means that any T project is actually a Nix project, and that Nix is used for both setting up the reproducible environment but also for running the pipeline. This will be explained in detail in the next chapters.

This book is the result of several years of work. I first started as a user with a simple need: I should be able to re-install the packages I used for an analysis easily. So I started using `{renv}`, then continued through Docker and Nix, led to packages for both R and Python, and ultimately culminated in an entirely new programming language.

In the chapters that follow, you'll learn how to build pipelines in which R, Python, and Julia are not merely neighbours but collaborators, working together in a single reproducible environment.

The core message from three years ago remains unchanged. You, as someone who writes code to analyse data, are a developer. Your work is important, and it deserves to be reliable. Using LLMs, you can now spend more of your energy on what actually matters. This book aims to give you the tools and the mindset to achieve that. The journey is more ambitious this time, but the payoff is far greater.

I hope you'll join me.

You can read this book for free online at https://b-rodrigues.github.io/reproducible-data-science/.

You can submit issues, suggest improvements, and ask questions on the book's GitHub repository[5].

[5]https://github.com/b-rodrigues/reproducible-data-science

1 Introduction

Just like the previous, R-focused edition of this book, this one will not teach you about machine learning, statistics, or visualisation.

The goal is to teach you a set of tools, practices, and project management techniques that should make your projects easier to reproduce, replicate, and retrace, with a focus on polyglot (multilingual) projects. I believe that with LLMs, polyglot projects will become increasingly common.

Before LLMs, if you were a Python user, you would avoid R like the plague, and vice versa. This is understandable: even though both are high-level scripting languages, and an R user could likely read and understand Python code (and vice versa), it is still a pain in the ass to set up another language just to run a few lines of code. Now, with LLMs, you can have a model generate the code, and depending on what you're doing, it's very likely to be correct. However, setting up another language is still quite annoying. This is where Nix comes in. Nix makes adding another language to a project extremely simple.

You can now use several languages for a project: if you're a Python user, you can now use R's `{ggplot2}` quite easily by simply adding R and `{ggplot2}` to your project using Nix. However, in a way, this is just moving the goalposts: as annoying as it was to set up R and `{ggplot2}`, and no matter how effortlessly Nix

solves this, now you need to set up Nix and get familiar with it!

But thankfully we live in the AI age, and LLMs can be quite helpful here. To make this even easier, we can combine Nix with T, a domain-specific language (DSL) designed specifically for building reproducible pipelines. My goal is for you not to have to worry (too much) about project setup, letting LLMs handle the heavy lifting. The way T is designed, is for LLMs to write T code, and you, together with an LLM if you wish to, are responsible for the R, Python or Julia code of your analysis.

Let me also show you alread what a reproducible analytical pipeline written in T looks like:

```
p = pipeline {
  -- 1. Load the data
  data = read_csv("datasets/country_level_data.csv")

  -- 2. Add a derived column in Python
  derived = pyn(
    command = <{
      derived = data.assign(
pct_change =
 ↪  data["average_price_m2_nominal_euros"].pct_change()
 ↪  * 100
      )
    }>,
    deserializer = ^ipc,
    serializer = ^ipc
  )

  -- 3. Plot the result in R
  plot = rn(
```

```
    command = <{
      library(ggplot2)
      plot <- ggplot(derived,
        aes(x = year, y =
↪   average_price_m2_nominal_euros)) +
        geom_line() +
        geom_point()
    }>,
    deserializer = ^ipc
  )
}

build_pipeline(p)
```

You don't need to understand every line yet, just three things. First, `p = pipeline { ... }` declares a set of named steps (`data`, `derived`, `plot`), and nothing runs yet. Second, each step can run in a different language: here Python adds a column and R draws the plot, and T wires each step's output into the next step's input automatically, so you never write glue code. Third, `build_pipeline(p)` runs every step in dependency order and caches each step's output, so rerunning the pipeline only rebuilds what changed. T comes with many features that make it easy for LLMs that have never seen T code to generate it flawlessly. It is also possible to load Python or R scripts instead of embedding the code directly in the T pipeline, which makes collaboration easier.

1.1 Who is this book for?

This book is for anyone who uses raw data to build any type of output. This could be a simple quarterly report, in which data is used for tables and graphs, a scientific article for a peer-reviewed journal, or even an interactive web application. The specific output doesn't matter, because the process is, at its core, always very similar:

- Get the data;
- Clean the data;
- Write code to analyse the data;
- Put the results into the final product.

This book assumes some familiarity with programming, particularly with the R and Python languages. I will not discuss Julia in great detail, as I am not familiar enough with it to do it justice. That being said, I will show you how to add Julia to a project and use it effectively if you need to.

This book is also aimed at people that are not afraid to experiment with brand new tools and brand new ways of doing data science. I will talk about T, a new domain-specific language I started building from 2025 onwards that is built to be used by LLMs and humans alike. T is purpose-built for writing reproducible analytical pipelines (RAPs) in the age of LLMs.

1.2 What is the aim of this book?

The aim of this book is to make the process of analysing data as reliable, retraceable, and reproducible as possible, and to do this by design. This means that once you're done with the analysis, you're done. You don't want to spend time, which you often

don't have anyway, rewriting or refactoring an analysis to make it reproducible after the fact. We both know this is not going to happen. Once an analysis is finished, it's on to the next one. If you need to rerun an older analysis (for example, because the data has been updated), you'll simply figure it out at that point, right? That's a problem for Future You. Hopefully, Future You will remember every quirk of your code, know which script to run at which point, which comments are outdated, and what features of the data need to be checked… You had better hope Future You is a more diligent worker than you are!

Going forward, I'm going to refer to a project that is reproducible as a "Reproducible Analytical Pipeline", or RAP for short. There are only two ways to create a RAP: either you are lucky enough to have someone on your team whose job is to turn your messy code into a RAP, or you do it yourself. The second option is by far the most common. The issue, as stated above, is that most of us simply don't do it. We are always in a rush to get to the results and don't think about making the process reproducible, because we assume it takes extra time that is better spent on the analysis itself. This is a misconception, for three reasons.

The first is that employing the techniques we will discuss in this book won't actually take much time. As you will see, they are not things you "add on top of" the analysis, but are part of the analysis itself, and they will also help with managing the project. Some of these techniques, especially testing, will even save you time and headaches.

The second reason is that an analysis is never a one-shot. Only the simplest tasks, like pulling a single number from a database, might be. Even then, chances are that once you provide that number, you'll be asked for a variation of it (for example, disaggregated by one or several variables). Or perhaps you'll be asked for an update in six months. You will quickly learn to

keep that SQL query in a script somewhere to ensure consistency. But what about more complex analyses? Is keeping the script enough? It is a good start, of course, but very often, there is no single script, or a script for each step of the analysis is missing.

The third reason is that we can actually offload so much of this to LLMs that there truly is no reason not to do it.

Of course, building a RAP is not just a technical challenge; it is also an organisational one. I've seen this play out many times in many different organisations. It's that time of the year again, and a report needs to be written. Ten people are involved, and just gathering the data is already complicated. Some get their data from Word documents attached to emails, some from a website, some from a PDF report from another department. I remember a story a senior manager at my previous job used to tell: once, a client put out a call for a project that involved setting up a PDF scraper. They periodically needed data from another department that only came in PDFs. The manager asked what was, at least from our perspective, an obvious question: "Why can't they send you the underlying data in a machine-readable format?" They had never thought to ask. So, my manager went to that department and talked to the people putting the PDF together. Their answer? "Well, we could send them the data in any format they want, but they've asked for the tables in a PDF."

So the first, and probably most important, lesson here is: when starting to build a RAP, make sure you talk to all the people involved.

1.3 Prerequisites

You should be comfortable with the command line. This book will not assume any particular Integrated Development Environment (IDE), so most of what I'll show you will be done via the command line. That said, I will spend some time helping you set up a data science-focused IDE, Positron, to work seamlessly with this workflow. The command line may be over 50 years old, but it is not going anywhere. In fact, thanks to the rise of LLMs, it seems to be enjoying a resurgence. Since these models generate text, it is far simpler to ask one for a shell command to solve a problem than to have it produce detailed instructions on where to click in a graphical user interface (GUI). Knowing your way around the command line is also essential for working with modern data science infrastructure: continuous integration platforms, Docker, remote servers… they all live in the terminal. So, if you're not at all familiar with the command line, you might need to brush up on the basics. Don't worry, though; this isn't a book about the intricacies of the Unix command line. The commands I'll show you will be straightforward and directly applicable to the task at hand.

This means, however, that if you are using Windows (first of all, why? and second of all), you will have to set up Windows Subsystem for Linux (WSL). This is because there is no native Nix implementation for Windows, and so we need to run the Linux version through WSL. Don't worry though, it's not that hard, and you can then use an IDE from Windows to work with the environments managed by Nix in a seamless way. (But seriously, if you can, consider switching to Linux. How can you tolerate ads (ADS!) in the Start menu?).

Ideally, you should be comfortable with either R or Python. This book will assume that you have been using at least one of

these languages for some projects and want to improve how you manage complex projects. You should know about packages and how to install them, have written some functions, understand loops, and have a basic knowledge of data structures like lists. While this is not a book on visualisation, we will be making some graphs as well.

Our aim is to write code that can be executed non-interactively. This is because a necessary condition for a workflow to be reproducible and to qualify as a RAP is for it to be executed by a machine, automatically, without any human intervention. This is the second lesson of building RAPs: there should be no human intervention needed to get the outputs once the RAP has started. If you achieve this, then your workflow is likely reproducible, or can at least be made so much more easily than if it required special manipulation by a human somewhere in the loop.

This is also, in part, why we will not be working in Jupyter notebooks. A notebook is a record of an interactive conversation with the computer; a reproducible pipeline is a plain-text description of a system. We return to this in Chapter 3.

1.4 What actually is reproducibility?

A reproducible project means that it can be rerun by anyone at zero (or very minimal) cost. But there are different levels of reproducibility, which I will discuss in the next section. Let's first outline the requirements that a project must meet to be considered a RAP.

1.4.1 Using open-source tools is a hard requirement

Open source is a hard requirement for reproducibility. No ifs, ands, or buts. I'm not just talking about the code you wrote for your research paper or report; I'm talking about the entire ecosystem you used to write your code and build the workflow.

Is your code open? Good. Or is it at least available to others in your organisation, in a way that they could re-execute it if needed? Also good.

But is your code written in a proprietary program, like STATA, SAS, or MATLAB? Then your project is not reproducible. It doesn't matter if the code is well-documented, well-written, and available on a version control system. The project is simply not reproducible. Why?

Because on a long enough time horizon, there is no way to re-execute your code with the exact same version of the proprietary language and operating system that were used when the project was developed. As I'm writing these lines, MATLAB, for example, is at version R2025a. Buying an older version may not be simple. I'm sure if you contact their sales department, they might be able to sell you an older version. Maybe you can even re-download older versions you've already purchased from their website. But maybe it's not that straightforward. Or maybe they won't offer this option in the future. In any case, if you search for "purchase old version of Matlab," you will see that many researchers and engineers have this need. ::: {.content-hidden when-format="pdf"}

Wanting to run older versions of analytics software is a recurrent need.

:::



Figure 1.1: Wanting to run older versions of analytics software is a recurrent need.

And if you're running old code written for, say, version R2008a, there's no guarantee that it will produce the exact same results on version R2025a. That's without even mentioning the toolboxes (if you're not familiar with them, they're MATLAB's equivalent of packages or libraries). These evolve as well, and there's no guarantee that you can purchase older versions. It's also likely that newer toolboxes cannot even run on older versions of MATLAB.

Let me be clear: what I'm describing here for MATLAB could also be said for any other proprietary program still commonly (and unfortunately) used in research and statistics, like STATA,

SAS, or SPSS. Even if some of these vendors provide ways to run older versions of their software, the fact that you have to rely on them for this is a barrier to reproducibility. There is no guarantee they will provide this option forever. Who can guarantee that these companies will even be around forever? More likely, they might shift from a program you install on your machine to a subscription-based model.

For just 199€ a month, you can execute your SAS (or whatever) scripts on the cloud! Worried about data confidentiality? No problem, data is encrypted and stored safely on our secure servers! Run your analysis from anywhere and don't worry about your cat knocking coffee over your laptop! And if you purchase the pro licence, for an additional 100€ a month, you can even execute your code in parallel!

Think this is science fiction? There is a growing and concerning trend for vendors to move to a Software-as-a-Service model with monthly subscriptions. It happened to Adobe's design software, and the primary reason it hasn't yet happened for data analytics tools is data privacy concerns. Once those are deemed "solved," I would not be surprised to see a similar shift.

1.4.2 Hidden dependencies can hinder reproducibility

Then there's another problem. Let me suppose you've written a thoroughly tested and documented workflow and made it available on GitHub (and let's even assume the data is freely available and the paper is open access). Or, if you're in the private sector, you've done all of the above, but the workflow is only available internally.

Let's further assume that you've used R, Python, or another open-source language. Can this analysis be said to be reproducible? Well, if it ran on a proprietary operating system, then the conclusion is: your project is not fully reproducible.

This is because the operating system the code runs on can also influence the outputs. There are particularities in operating systems that may cause certain things to work differently. Admittedly, this is rarely a problem in practice, but it does happen[1], especially if you're working with high-precision floating-point arithmetic, as you might in the financial sector, for instance.

Thankfully, as you will see in this book, there is no need to change your operating system to deal with this issue.

1.4.3 The requirements of a RAP

So where does that leave us? For a project to be truly reproducible, it has to satisfy the following requirements:

- Source code must be available and thoroughly tested and documented (which is why we will be using Git and GitHub).
- All dependencies must be easy to find and install (we will deal with this using dependency management tools).
- It must be written in an open-source programming language (no-code tools like Excel are non-reproducible by default because they can't be used non-interactively, which is why we will be using languages like R, Python, and Julia).
- The project needs to run on an open-source operating system (we can deal with this without having to install and learn a new OS, thanks to tools like Docker).

[1] https://github.com/numpy/numpy/issues/9187

- The data and the final report must be accessible, if not publicly, then at least within your company. This means the concept of "scripts and/or data available upon request" belongs in the bin.



Figure 1.2: A real sentence from a real paper published in *THE LANCET Regional Health*. How about *make the data available and I won't scratch your car*, how's that for a reasonable request?

1.4.4 Are there different types of reproducibility?

Let's take one step back. We live in the real world, where constraints outside of our control can make it impossible to build a true RAP. Sometimes we need to settle for something that might not be perfect, but is the next best thing.

In what follows, let me assume the code is tested and documented, so we will only discuss the pipeline's execution.

The *least* reproducible pipeline would be something that works, but only on your machine. This could be due to hardcoded paths that only exist on your laptop. Anyone wanting to rerun the pipeline would need to change them. This should be documented

in a README, which we've assumed is the case. But perhaps the pipeline only runs on your laptop because the computational environment is hard to reproduce. Maybe you use software, even open-source software, that is not easy to install (anyone who has tried to install R packages on Linux that depend on `{rJava}` knows what I'm talking about).

A better, though still imperfect, pipeline would be one that could be run more easily on any similar machine. This could be achieved by avoiding hardcoded absolute paths and by providing instructions to set up the environment. For example, in Python, this could be as simple as providing a `requirements.txt` file that lists the project's dependencies, which could be installed using `pip`:

```
pip install -r requirements.txt
```

Doing this helps others (or Future You) install the required packages. However, this is not enough, as other software on your system, outside of what `pip` manages, can still impact the results.

You should also ensure that people run the same analysis on the same versions of R or Python that were used to create it. Just installing the right packages is not enough. The same code can produce different results on different versions of a language, or not run at all. If you've been using Python for some time, you certainly remember the switch from Python 2 to Python 3. Who knows, the switch to Python 4 might be just as painful!

The takeaway message is that relying on the language itself being stable over time is not a sufficient condition for reproducibility. We have to set up our code in a way that is *explicitly* reproducible by dealing with the versions of the language itself.

So what does this all mean? It means that reproducibility exists on a continuum. Depending on the constraints you face, your project can be "not very reproducible" or "totally reproducible". Let's consider the following list of factors that can influence how reproducible your project truly is:

- Version of the programming language used.
- Versions of the packages/libraries of said programming language.
- The operating system and its version.
- Versions of the underlying system libraries (which often go hand-in-hand with the OS version, but not always).
- And even the hardware architecture that you run the software stack on.

By "reproducibility is on a continuum," I mean that you can set up your project to take none, one, two, three, four, or all of the preceding items into consideration.

This is not a novel or new idea. Peng (2011) already discussed this concept but named it the *reproducibility spectrum*:

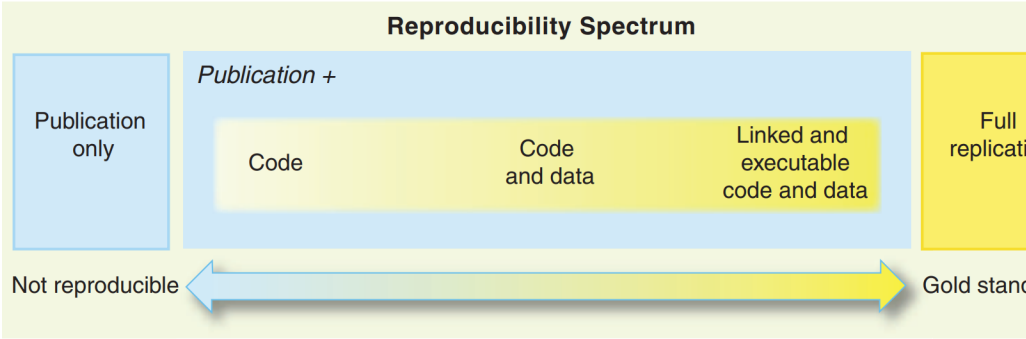


Figure 1.3: The reproducibility spectrum from Peng's 2011 paper.

Notice, however, what the spectrum does *not* address: it is concerned entirely with the *environment*, whether the same packages and language version are available. It says nothing about whether the computation can actually be *re-executed*. A project can sit at the far right of Peng's spectrum, with every dependency pinned and documented, and still be practically irreproducible because nobody can figure out which script to run first, in what order, and with what inputs. Environment reproducibility is necessary but not sufficient; you also need what we might call *computational reproducibility*: an explicit, runnable description of the pipeline itself, not just the software it runs on. We will return to this distinction in Chapter 4, where it becomes the central motivation for T's mandatory pipeline architecture.

Let me finish this introduction by discussing the last item on the list: hardware architecture. In 2020, Apple changed the hardware architecture of their computers. Their new machines no longer use Intel CPUs, but instead Apple's own proprietary architecture (Apple Silicon) based on the ARM specification. Concretely, this means that binary packages built for Intel-based Apple computers cannot run on their new machines, at least

not without a compatibility layer. If you have a recent Apple Silicon Mac and need to install old packages to rerun a project (and we will learn how to do this), they need to be compiled to work on Apple Silicon first. While a compatibility layer called Rosetta 2 exists, my point is that you never know what might come in the future. The ability to compile from source is important because it requires the fewest dependencies outside of your control. Relying on pre-compiled binaries is not future-proof, which is another reason why open-source tools are a hard requirement for reproducibility.

For Windows users, don't think that the preceding paragraph does not concern you. It is very likely that Microsoft will push for OEM manufacturers to build more ARM-based computers in the future. There is already an ARM version of Windows, and I believe Microsoft will continue to support it. This is because ARM is much more energy-efficient than other architectures, and any manufacturer can build its own ARM CPUs by purchasing a license, a very interesting proposition from a business perspective.

It is also possible that we will move towards more cloud-based computing, though I think this is less likely than the hardware shift. In that case, it is quite likely that the actual code will be running on Linux servers that are ARM-based, due to energy and licensing costs. Here again, if you want to run your historical code, you'll have to compile old packages and programming language versions from source.

Okay, this might all seem incredibly complicated. How on earth are we supposed to manage all these risks and balance the immediate need for results with the future need to rerun an old project? And what if rerunning it is never needed?

As you shall see, this is not as difficult as it sounds, thanks to

Nix, the tireless efforts of the `nixpkgs` maintainers who work to build truly reproducible packages, and T (which brings it all together into a clean, declarative pipeline).

Let's dive in!

```
"Hello from T!"
```

```
Hello from T!
```

Over the next two chapters, we'll unpack all of this and write our first pipeline from scratch.

2 The Nix Package Manager

2.1 Introduction

Nix is a package manager that can be installed on your computer, regardless of the operating system (but requires WSL2 on Windows). If you are familiar with the Ubuntu Linux distribution, you have likely used `apt-get` to install software. On macOS, you may have used `homebrew` for similar purposes. Nix functions in a comparable way but has many advantages over classic package managers, as it focuses on reproducible builds and downloads packages from `nixpkgs`, currently the largest software repository[1].

In this chapter, we will explore the critical need for environment reproducibility in modern workflows. We will see why ad-hoc tools often fail, and how Nix's declarative approach and "component closures" provide a robust solution. We will also cover the core concepts of Nix (derivations, the store, and hermetic builds) that make this possible.

[1] https://repology.org/repositories/graphs

2.2 Why Reproducibility? Why Nix?

2.2.1 Motivation: Reproducibility in Scientific and Data Workflows

To ensure that a project is reproducible you need to deal with at least four things:

- Ensure the required version of your programming language (R, Python, etc.) is installed.
- Ensure the required versions of all packages are installed.
- Ensure all necessary system dependencies are installed (for example, a working Java installation for the `{rJava}` R package on Linux).
- Ensure you can install all of this on the hardware you have on hand.

But in practice, one or more of these bullet points are missing from projects. Our goal is to learn how to fulfil all the requirements to build reproducible projects.

The current consensus for tackling the first three points is often a mixture of tools: Docker for system dependencies, `{renv}` or `uv` for package management, and tools like the R installation manager (`rig`) for language versions. As for the last point, hardware architecture, the only way out is to be able to compile the software for the target platform. This involves a lot of moving parts and requires significant knowledge to get right.

2.2.2 Problems with Ad-Hoc Tools

Tools like Python's `venv` or R's `renv` only deal with some pieces of the reproducibility puzzle. Often, they assume an underlying OS,

do not capture system-level dependencies (like `libxml2`, `pandoc`, or `curl`), and require users to "rebuild" their environments from partial metadata. Docker helps but introduces overhead, security challenges, and complexity, and just adding it to your project doesn't make it reproducible if you don't explicitly take some precautionary steps.

Traditional approaches fail to capture the entire dependency graph of a project in a deterministic way. This leads to "it works on my machine" syndromes, onboarding delays, and subtle bugs.

2.2.3 Nix: A Declarative Solution

With Nix, we can handle all of these challenges with a single tool.

The first advantage of Nix is that its repository, `nixpkgs`, is humongous. As of this writing, it contains over 120,000 pieces of software, including the *entirety of CRAN and Bioconductor*. This means you can use Nix to handle everything: R, Python, Julia, their respective packages, and any other software available through `nixpkgs`, making it particularly useful for polyglot pipelines.

The second and most crucial advantage is that Nix allows you to install software in (relatively) isolated environments. When you start a new project, you can use Nix to install a project-specific version of R and all its packages. These dependencies are used only for that project. If you switch to another project, you switch to a different, independent environment. But this also means that all the dependencies of R and R packages, plus all of their dependencies and so on get installed as well. Your project's development environment will not depend on anything outside

of it (well, there are some caveats which we will explore as we move on).

This is similar to `{renv}`, but the difference is profound: you get not only a project-specific library of R packages but also a project-specific R version and all the necessary system dependencies. For example, if you need `{xlsx}`, Nix automatically figures out that Java is required and installs and configures it for you, without any intervention.

What's more, you can *pin* your project to a specific revision of the `nixpkgs` repository. This ensures that every package Nix installs will always be at the exact same version, regardless of when or where the project is built. The environment is defined in a simple plain-text file, and anyone using that file will get a byte-for-byte identical environment, even on a different operating system (Dolstra 2006).

2.3 Important Concepts

Before we start using Nix, it is important to spend some time learning about some Nix-centric concepts, starting with the *derivation*.

In `Nix` terminology, a derivation is *a specification for running an executable on precisely defined input files to repeatably produce output files at uniquely determined file system paths.* (source[2])

In simpler terms, a derivation is a recipe with precisely defined inputs, steps, and a fixed output. This means that given identical inputs and build steps, the exact same output will always

[2] https://nix.dev/manual/nix/2.25/language/derivations

be produced. To achieve this level of reproducibility, several important measures must be taken:

- All inputs to a derivation must be explicitly declared (and "inputs" here is meant in a very broad sense; for example, configuration flags are also inputs!).
- Inputs include not just data files but also software dependencies, configuration flags, and environment variables: essentially, anything necessary for the build process.
- The build process takes place in a *hermetic* sandbox to ensure the exact same output is always produced.

The next sections explain these three points in more detail.

2.3.1 Derivations

Here is an example of a *simple* `Nix` expression:

```
let
  pkgs = import (
   fetchTarball
    ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/2025
  ) {};
in
pkgs.stdenv.mkDerivation {
  name = "filtered_mtcars";
  buildInputs = [ pkgs.gawk ];
  dontUnpack = true;
  src = ./mtcars.csv;
  installPhase = ''
    mkdir -p $out
    awk -F',' 'NR==1 || $9=="1" { print }' $src >
 ↪  $out/filtered.csv
```

```
  '';
}
```

Without going into too much detail, this code uses `awk`, a common Unix data processing tool, to filter the `mtcars.csv` file. As you can see, a significant amount of boilerplate is required for this simple operation. However, this approach is completely reproducible: the dependencies are declared and pinned to a specific version of the `nixpkgs` repository. The only thing that could make this small pipeline fail is if the `mtcars.csv` file is not provided to it.

`Nix` builds the `filtered.csv` output file in two steps: it first generates a *derivation* from this expression, and only then does it build the output. For clarity, I will refer to code like the example above as a *derivation* rather than an expression, to avoid confusion with the concept of an *expression* in R.

The goal of the tool we will use in this book, T, is to help you create pipelines from such derivations without needing to learn the Nix language itself, while still benefiting from its powerful reproducibility features.

2.3.2 Dependencies of derivations

Nix requires that the dependencies of any derivation be explicitly listed and managed by Nix itself. If you are building an output that requires Quarto, then Quarto must be explicitly listed as an input, even if you already have it installed on your system. The same applies to Quarto's dependencies, and their dependencies, all the way down. To run a linear regression with R, you essentially need Nix to build the entire universe of software that R depends on first.

In Nix terms, this complete set of packages is what its author, Eelco Dolstra, refers to as a *component closure*:

> The idea is to always deploy component closures: if we deploy a component, then we must also deploy its dependencies, their dependencies, and so on. That is, we must always deploy a set of components that is closed under the 'depends on' relation.

(Dolstra, De Jonge, and Visser 2004)

Figure 4 of (Dolstra, De Jonge, and Visser 2004)

In the figure, `subversion` depends on `openssl`, which itself depends on `glibc`. Similarly, if you write a derivation to filter `mtcars`, it requires an input file, R, `{dplyr}`, and all of their respective dependencies. All of these must be managed by Nix. If any dependency exists "outside" this closure, the pipeline will only *work on your machine*, defeating the purpose of reproducibility.

2.3.3 The Nix store and hermetic builds

When building derivations, their outputs are saved into the **Nix store**. Typically located at `/nix/store/`, this folder contains all the software and build artefacts produced by Nix.

For example, the output of a derivation might be stored at a path like `/nix/store/81k4s9q652jlka0c36khpscnmr8wk7jb-filtered`. The long cryptographic hash uniquely identifies the build output and is computed based on the content of the derivation and all its inputs. This ensures that the build is fully reproducible.

As a result, building the same derivation on two different machines will yield the same cryptographic hash. You can substitute the built artefact with the derivation that generates it one-to-one, just as in mathematics, where writing $f(2)$ is the same as writing 4 for the function $f(x) := x^2$.

To guarantee that derivations always produce identical outputs, builds must occur in an isolated environment known as a **hermetic sandbox**. This process ensures that the build is unaffected by external factors, such as the state of the host system. This isolation extends to environment variables and even network access. If you need to download data from an API, for example, it will not work from within the build sandbox. This may seem restrictive, but it makes perfect sense for reproducibility: an API's output can change over time.

For a truly reproducible result, you should obtain the data once, version it, and use that archived data as an input to your analysis.

2.3.4 Other key Nix concepts

We've covered derivations, dependencies, closures, the Nix store, and hermetic builds. That's the core of what makes Nix tick. But there are a few more concepts worth knowing about before we move on:

- **Purity**: Nix tries very hard to keep builds "pure": the output only depends on what you explicitly list as inputs. If a build script tries to reach out to the internet or read some random file on your machine, Nix will block it. That can feel restrictive at first, but it's what guarantees reproducibility.

- **Binary caches**: You don't always need to build everything yourself. Think back to the math analogy: if you already know that $f(2) = 4$, there's no need to compute it again; just reuse the result! Nix does the same with binary caches: because every build is identified by its unique cryptographic hash made from its inputs, this means a prebuilt package fetched from `cache.nixos.org` (or your own cache you may want to set up) is bit-for-bit identical to what you would have built locally. This is why Nix is both reproducible and fast.

- **Garbage collection**: Since Nix never overwrites anything in the store, old packages can pile up. Running `nix-store --gc` will run the garbage collector to free up space.

- **Overlays**: If you want to tweak a package or add your own without forking all of `nixpkgs`, you can use overlays. They let you extend or override existing definitions in a clean, composable way.

- **Flakes**: The newer way to define and pin Nix projects. Flakes make it easier to share and reuse Nix setups across machines and repositories. There is a lot of discussion around flakes, as they're officially still considered not stable, even though they've been widely adopted by the community. But don't worry, this is not something you'll need to think about for this book.

Together, these features explain why Nix isn't *just another package manager*. It's more like a framework for reproducible environments that can scale from a single project to an entire operating system (called NixOS[3]).

[3] https://nixos.org/

As a little sidenote: I also want to highlight that Nix can even be used to declaratively and reproducibly configure your operating system (be it NixOS, macOS or other Linux distributions) using a tool that integrates with it called `home-manager`.[4] This is outside the scope of this book, but I wanted to highlight it, as it's extremely powerful. What this means in practice is that you could write a whole Nix expression that not only downloads and configures software, but even sets up users with their specific software, and preferences like wallpapers, colour schemes and so on.

2.4 Caveats

While Nix is powerful, there are some limitations and practical hurdles to be aware of if you plan to use it for actual work:

- **Hardware acceleration**: On non-NixOS systems, it can be difficult to set up GPU acceleration (CUDA, ROCm, OpenCL). Drivers are tightly coupled to the host kernel and libraries, while Nix builds aim for strict isolation. On NixOS this integration is smoother, but on macOS or other Linux distros you may encounter limitations or extra manual steps.
- **macOS-specific issues**: Reproducibility is harder to achieve on macOS (both Intel and Apple Silicon) than on Linux. Nix packages often rely on Apple system frameworks (e.g., CoreFoundation, Security) that live outside the Nix store and compromise hermetic builds[5]. The macOS sandbox is also weaker than Linux's and sometimes leaks

[4] https://github.com/nix-community/home-manager
[5] https://github.com/NixOS/nixpkgs/issues/67166

system tools like Xcode or Rosetta into builds[6]. Hydra cache coverage is thinner for Darwin platforms, especially Apple Silicon, so cache misses and local builds are much more common[7]. Finally, pinned environments can break after macOS or Xcode updates because of changes in system libraries or compiler flags[8].

That said, in practice most packages build fine on macOS, and the ecosystem continues to improve. When reproducibility problems do appear, the simplest fix is often just to use another nearby `nixpkgs` revision for pinning that is known to work. This makes the situation less fragile than it might first appear. Another solution would be to use Docker to deploy the Nix environment and use that as a dev container. All of this will be discussed in detail in this book.

- **Steep learning curve**: The Nix language and ecosystem (flakes, overlays, derivations) can be conceptually difficult if you come from traditional package managers. Even basic customizations require some ramp-up time. This was my main motivation to write `{rix}`, `{rixpress}` (for R) and `ryxpress` (for Python).

- **Disk space and builds**: Because Nix never mutates software in place, the store can accumulate large amounts of data. Cache misses sometimes force local builds, which can be slow and resource-intensive. However, it is of course possible to empty the Nix store to recover disk space, and it is also possible to set up your own project cache if you

[6] https://discourse.nixos.org/t/nix-macos-sandbox-issues-in-nix-2-4-and-later/17475

[7] https://www.reddit.com/r/NixOS/comments/17uxj6q/how_does_nix_package_manager_work_on_apple_silicon/

[8] https://github.com/NixOS/nix/issues/11679

so wish. Setting up your own cache will also be something that we will explore in this book.

These caveats don't diminish Nix's strengths but highlight that its guarantees are strongest on Linux (and thus WSL), and especially on NixOS. On macOS, reproducibility is possible but sometimes requires extra work, a bit of flexibility, and occasionally picking a different `nixpkgs` snapshot.

2.5 In summary

Nix makes it possible to *actually* build software reproducibly. To achieve this, it introduces several core concepts that are quite specific, and definitely worth taking the time to understand.

In the next chapter, we will learn how to install Nix, configure `cachix`, and set up Positron for a seamless development experience.

3 Setting Up Your Environment

In this chapter, we will install Nix, configure the `rstats-on-nix` binary cache to speed up installations, and set up our development environment (Positron, VS Code, or Emacs) to work seamlessly with reproducible Nix environments.

3.1 Installing Nix

3.1.1 For Windows Users: WSL2 Prerequisites

If you are on Windows, you need the Windows Subsystem for Linux 2 (WSL2) to run Nix. If you are on a recent version of Windows 10 or 11, you can simply run this as an administrator in PowerShell:

```
wsl --install
```

You can find further installation notes at this official MS documentation[1].

[1] https://learn.microsoft.com/en-us/windows/wsl/install

I recommend activating `systemd` in Ubuntu WSL2, mainly because this supports users other than `root` running Nix. To set this up, please follow this official Ubuntu blog entry[2]:

```
# in WSL2 Ubuntu shell

sudo -i
nano /etc/wsl.conf
```

This will open `/etc/wsl.conf` in nano, a command line text editor. Add the following line:

```
[boot]
systemd=true
```

Save the file with CTRL-O and then quit nano with CTRL-X. Then, type the following line in PowerShell:

```
wsl --shutdown
```

and then relaunch WSL (Ubuntu) from the start menu. For those of you running Windows, we will be working exclusively from WSL2 now. If that is not an option, then I highly recommend you set up a virtual machine with Ubuntu using VirtualBox[3] for example, or dual-boot Ubuntu.

[2] https://ubuntu.com/blog/ubuntu-wsl-enable-systemd
[3] https://www.virtualbox.org/wiki/Downloads

3.1.2 The Determinate Systems installer

Installing (and uninstalling) Nix is quite simple, thanks to the installer from Determinate Systems[4], a company that provides services and tools built on Nix, and works the same way on Linux (native or WSL2) and macOS.

Do not use your operating system's package manager to install Nix. Instead, simply open a terminal and run the following line (on Windows, run this inside WSL, and after the prerequisites listed above):

```
curl --proto '=https' --tlsv1.2 -sSf \
    -L https://install.determinate.systems/nix | \
    sh -s -- install --no-confirm --extra-conf "
trusted-users = root $USER
substituters = https://cache.nixos.org
 ↪  https://rstats-on-nix.cachix.org
trusted-public-keys =
 ↪  cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcD
 ↪  rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUrhi1Jz
```

This single command installs Nix and configures everything T needs:

- **`--no-confirm`** runs the installer non-interactively, so you don't have to answer any prompts.
- **`--extra-conf`** injects configuration directly into your Nix config during installation. The three settings it applies are:
- **`trusted-users = root $USER`**: Marks your user as a trusted Nix user. This is required for binary caches and certain Nix operations to work without permission errors.

[4] https://github.com/DeterminateSystems/nix-installer

- **`substituters = ...`**: Tells Nix to fetch pre-built packages from the NixOS cache and the `rstats-on-nix` Cachix cache (used by T for R and Python packages), avoiding long builds from source.
- **`trusted-public-keys = ...`**: The cryptographic keys Nix uses to verify the authenticity of packages from those caches.

Because the installer handles all of this in one step, there are no manual post-install configuration steps when using the Determinate Systems installer. If you installed Nix through other means, follow along below.

3.2 Already Have Nix?

If you installed Nix through a method other than the Determinate Systems installer (e.g., the official Nix installer[5], Homebrew, your Linux distribution's package manager, or you're on NixOS), you need to manually configure two more things.

3.2.1 Step 1: Add yourself as a trusted user

T uses binary caches that require your user to be in the `trusted-users` list. Without this, you'll get "ignoring untrusted substituter" errors.

If you're on NixOS, add this to your `/etc/nixos/configuration.nix`:

```
nix.settings.trusted-users = [ "root"
 ↪  "your-username" ];
```

[5] https://nixos.org/download/

Then rebuild:

```
sudo nixos-rebuild switch
```

On non-NixOS Linux, WSL or macOS, edit `/etc/nix/nix.conf`:

```
# Add your username to the trusted-users line
# If the line exists, append your username to it:
sudo sed -i 's/^trusted-users = .*/& your-username/'
 ↪  /etc/nix/nix.conf

# Or if no trusted-users line exists, add one:
echo "trusted-users = root $(whoami)" | sudo tee -a
 ↪  /etc/nix/nix.conf
```

Then restart the Nix daemon:

```
# Linux (systemd)
sudo systemctl restart nix-daemon

# macOS (launchd)
sudo launchctl kickstart -k
 ↪  system/org.nixos.nix-daemon
```

3.2.2 Step 2: Add the binary cache

T relies on pre-built R and Python packages from the `rstats-on-nix` Cachix cache. Without this cache, `nix develop` will try to build everything from source, which can take a very long time.

On NixOS, add this to your `/etc/nixos/configuration.nix`:

```
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://rstats-on-nix.cachix.org"
  ];
  trustedPublicKeys = [
    ↪ "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFG
    ↪ "rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUrh
  ];
};
```

Then rebuild:

```
sudo nixos-rebuild switch
```

On non-NixOS Linux, WSL or macOS, add these lines to `/etc/nix/nix.conf`:

```
echo "substituters = https://cache.nixos.org
 ↪ https://rstats-on-nix.cachix.org" | sudo tee -a
 ↪ /etc/nix/nix.conf
echo "trusted-public-keys =
 ↪ cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDS
 ↪ rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUrhi1JzhH
 ↪ | sudo tee -a /etc/nix/nix.conf
```

Then restart the Nix daemon (same commands as above).

3.3 Verifying installation with Temporary Shells

You now have Nix installed; before continuing, let's see if everything works (close all your terminals and reopen them) by dropping into a temporary shell with a tool you likely have not installed on your machine.

Open a terminal and run:

```
which sl
```

you will likely see something like this:

```
which: no sl in ....
```

now run this:

```
nix shell nixpkgs#sl
```

and then again:

```
which sl
```

this time you should see something like:

```
/nix/store/cndqpx74312xkrrgp842ifinkd4cg89g-sl-5.05/bin/sl
```

This is the path to the `sl` binary installed through Nix. The path starts with `/nix/store`: the *Nix store* is where all the software installed through Nix is stored. Now type `sl` and see what happens!

Temporary shells are quite useful, especially if you want to simply run a command using some tool that you only need infrequently. But this is not how we are going to be using Nix.

3.4 Configuring your IDE

3.4.1 Why a text editor (and not a notebook)?

Before we configure an IDE, a quick note on *what* we'll be writing in it. You might expect data science to happen in a Jupyter notebook. We'll mostly be working in plain-text files instead, for a few reasons.

A notebook is a recording of a conversation between you and the computer: run a cell, look at the output, change something, run it again, restart the kernel, run everything. That's a fine way to *explore*, but a poor way to *describe a system*. Notebooks are JSON rather than plain text, which makes version control, code review, and collaboration painful, and they encourage messy code that mixes data, logic, and results in one file.

Plain text is the opposite: the simplest, most enduring interface in computing. Git can diff it, and it is easy to review and collaborate on. It also matters for the AI agents we'll increasingly work with: notebooks carry hidden state, side effects, and non-linear execution, exactly what makes life hard for a model (more on this in Chapter 4).

3.4.2 Prerequisites

We now need to configure an IDE to use our Nix shells as development environments. You are free to use whatever IDE you want but the instructions below are going to focus on Positron, which is a fork of VS Code geared towards data science. It works well with both Python and R and makes it quite easy to choose the right R or Python interpreter (which you'll have to do to make sure you're using the one provided by Nix, see here[6]).

If you want to use VS Code proper, you can follow all the instructions here, but you need to install the REditorSupport[7] and the Python[8] extension. You will also need to add the `{languageserver}` R package to your Nix shells.

On Windows, you need to install Positron on Windows, not inside WSL.

If you want to use RStudio, you can, but you will need to install it through Nix: an RStudio installed through the usual means for your system is not going to be able to interact with Nix! This is a limitation of RStudio and there is currently no workaround. If you want to use RStudio, just skip the rest of the chapter, as it's irrelevant to you. Later, we'll see how to use Nix to install RStudio.

Other editors work well with Nix too. Emacs users can use the envrc[9] or emacs-direnv[10] packages to automatically load Nix

[6] https://positron.posit.co/managing-interpreters.html
[7] https://marketplace.visualstudio.com/items?itemName=REditorSupport.r
[8] https://marketplace.visualstudio.com/items?itemName=ms-python.python
[9] https://github.com/purcell/envrc
[10] https://github.com/wbolster/emacs-direnv

environments. Neovim users can use direnv.nvim[11]. The key is that any editor with `direnv` support will work with the setup described below.

3.4.3 direnv

Once Positron is installed, you need to install a piece of software called `direnv`: `direnv` will automatically load Nix shells when you open a project that contains a `flake.nix`, which will be generated by T. I haven't talked about `flake.nix` files but essentially, a `flake.nix` file is a file that contains the specification of our development environments. `direnv` works on any operating system and many editors support it, including Positron. If you're using Windows, install `direnv` in WSL (even though you've just installed Positron for Windows). To install `direnv` run this command:

```
nix profile install nixpkgs#direnv
```

This will install `direnv` and make it available even outside of Nix shells!

Then, I highly recommend installing the `nix-direnv` extension:

```
nix profile install nixpkgs#nix-direnv
```

It is not mandatory to use `nix-direnv` if you already have `direnv`, but it'll make loading environments much faster and seamless.

[11] https://github.com/direnv/direnv.vim

Finally, if you haven't used `direnv` before, don't forget this last step[12] to make your terminal detect and load `direnv` automatically.

Then, in Positron, install the direnv[13] extension. Finally, add a file called `.envrc` and simply write the following line in it (this `.envrc` file should be in the same folder as your project's `flake.nix`):

```
use flake
```

On Windows, *remotely connect to WSL* first, but on other operating systems, simply open the project's folder using `File > Open Folder...` and you will see a pop-up stating `direnv: /PATH/TO/PROJECT/.envrc is blocked` and a button to allow it. Click `Allow` and then open an R script. You might get another pop-up asking you to restart the extension, so click `Restart`. Be aware that at this point, `direnv` will use Nix to start building the environment. If that particular environment hasn't been built and cached yet, it might take some time before you will be able to interact with it. You might get yet another popup, this time from the R Code extension complaining that R can't be found. In this case, simply restart Positron and open the project folder again: now it should work every time.

3.4.4 In summary and next steps

We now have everything we need to start using T to set up reproducible data science projects. T, thanks to Nix, will handle the installation of all the required tools: R, Python, RStudio if you want it, and any other tool.

[12] https://direnv.net/docs/hook.html
[13] https://github.com/direnv/direnv-vscode

4 Reproducible Development Environments and Pipelines with T

4.1 Introduction

Now that we have Nix installed, and our IDE configured, we can actually tackle the reproducibility puzzle. This chapter will introduce T, a domain-specific language developed to build reproducible analytical pipelines using Python, R or Julia.

Before anything, I think it's important to see what an actual T pipeline looks like, to ground the rest of our discussion. Here is an example (you can find this example, and many others, in the T Demos Repository[1]):

```
p = pipeline {
  -- 1. Read gorilla image
  gorilla_pixels = pyn(
    command = <{
from PIL import Image
import numpy
```

[1] https://github.com/b-rodrigues/t_demos/tree/master/yanai_lercher_2020_t

```
def read_image(x):
    im = Image.open(x).convert("L")
    pixels = numpy.asarray(im)
    return pixels

gorilla_pixels =
 ↪  read_image("path/to/gorilla-waving-cartoon.jpg")
    }>,
    include = ["data/gorilla"]
  )

  -- 2. Threshold level
  threshold_level = pyn(command = <{ threshold_level
 ↪  = 50 }>)

  -- 3. Compute coordinates
  py_coords = pyn(
    command = <{
import numpy
py_coords =
 ↪  numpy.column_stack(numpy.where(gorilla_pixels <
 ↪  threshold_level))
    }>
  )

  -- 4. Convert to DataFrame for R (transfer via
 ↪  Arrow)
  raw_coords = pyn(
    command = <{
import pandas as pd
raw_coords = pd.DataFrame(py_coords, columns=["V1",
 ↪  "V2"])
    }>,
```

```
    serializer = ^ipc
  )

  -- 5. Clean coordinates in R
  coords = rn(
    command = <{
library(dplyr)
coords <- clean_coords(raw_coords)
    }>,
    functions = ["src/functions.R"],
    deserializer = ^ipc,
    serializer = ^ipc
  )

  -- 6. Gender distribution
  gender_dist = rn(
    command = <{
library(dplyr)
gender_dist <- gender_distribution(coords)
    }>,
    functions = ["src/functions.R"],
    deserializer = ^ipc
  )

  -- 7. Plots
  plot1 = rn(
    command = <{
library(dplyr)
library(ggplot2)
plot1 <- make_plot1(coords)
    }>,
    functions = ["src/functions.R"],
    deserializer = ^ipc
```

```
  )

  plot2 = rn(
    command = <{
library(dplyr)
library(ggplot2)
plot2 <- make_plot2(coords)
    }>,
    functions = ["src/functions.R"],
    deserializer = ^ipc
  )

  -- Render Quarto report
  report = node(script = "src/report.qmd", runtime =
  ↪  Quarto)
}

-- Materialize
populate_pipeline(p, build = true, verbose=1)
```

```
T execution failed:
Error running t (error code 1): <no output>
```

You do not need to understand every line yet, but let's walk through it together, because this single example contains the whole idea.

The big picture. `p = pipeline { ... }` defines a *pipeline*: a directed graph of named computation steps called *nodes*. Every name inside the braces is a node, and `p` itself is a first-class value you can inspect, transform, and pass around. Nothing runs yet. The code in the braces is a description. The

last line, `populate_pipeline(p, build = true, verbose = 1)`, is what actually executes the pipeline: it materializes the graph and builds every node in dependency order, printing each node's output as it goes.

The nodes. Each node is a computation in a specific runtime:

- `pyn(...)` declares a Python node, `rn(...)` an R node, and `node(script = ..., runtime = Quarto)` a Quarto node that renders the final report.
- The code inside `<{ ... }>` is the node's body. T stores it verbatim; it does not parse or execute it itself. When the node is built, the code runs inside a hermetic Nix sandbox provisioned for that runtime.
- `gorilla_pixels` opens the image and converts it to a grayscale array; `include = ["data/gorilla"]` copies the raw image into the sandbox so the code can find it.
- `threshold_level` is a node that produces a single number. It looks trivial, but that is the point: because the threshold is a node, it is part of the graph. Change it, and only the downstream nodes re-run.
- The rest follows the data: `py_coords` locates the dark pixels, `raw_coords` turns them into a pandas DataFrame, `coords` cleans them in R, and `gender_dist`, `plot1`, and `plot2` produce the final table and plots.

How the nodes are wired together. Notice that no node says "I depend on X". T infers the dependencies by scanning each node's code for names: `py_coords`'s body mentions `gorilla_pixels` and `threshold_level`, so T draws edges from those nodes into `py_coords`; `coords` mentions `raw_coords`, and so on. You can declare the nodes in any order and T assembles the graph, checks it for cycles, and builds in the right order.

Crossing the language boundary. The interesting part of this analysis is that data moves from Python to R. That is what `serializer` and `deserializer` are for. `serializer = ^ipc` tells T to write the node's output as an Arrow IPC file, a columnar format that both pandas and R's `{arrow}` package read natively, so the data crosses the language boundary without a lossy CSV round-trip. The R node's `deserializer = ^ipc` reads it back. And `functions = ["src/functions.R"]` sources a file of R helpers (`clean_coords()`, `make_plot1()`, …) into the sandbox before the node's command runs.

Why this is reproducible. Every node runs in its own Nix sandbox, with exactly the packages declared for the project, so no machine-specific state can leak in. And every node's output is cached by content: re-run the pipeline and T rebuilds only the nodes whose inputs actually changed. The whole analysis (image, threshold, cleaning, plots, report) is now a single machine-readable description that anyone can execute from raw inputs to final outputs.

We will come back to each of these pieces: T's syntax in the next section, the project's environment in the rest of this chapter, and building your first pipeline in Chapter 5. For now, hold onto the core idea: *a pipeline is a named graph of computations, and the dependencies are the names you use.*

Let's go back to some theory before moving on.

4.1.1 The Reproducibility Puzzle

Reproducibility in research and data science exists on a continuum. At one end, authors might only describe their methods in prose. Moving along the spectrum, they might share code, then

data, and finally what we call a *computational environment*: the complete set of software required to execute an analysis.

Even when researchers share code and data, they rarely specify the full software stack: the exact version of R, all package versions, and crucially, the system-level dependencies. Yet differences in any of these can lead to divergent results from the same code.

The stakes are well documented. Large-scale replication studies have consistently found that a substantial fraction of published results cannot be reproduced Brodeur et al. (2026), and reproducibility has become a first-class concern for computational science (Stodden et al. 2016).

Tools like `{renv}` address part of the puzzle: they capture R package versions in a lockfile. But `{renv}` does not manage the R version itself (you need `rig` for that), and neither handles system libraries. If `{sf}` requires GDAL 3.0 but your system has 2.4, `{renv}` can't help. And if your project uses both R and Python? Now you're coordinating multiple package managers, each with its own configuration.

4.1.2 Reproducibility Is Not Just About the Environment

Here is a subtlety that most reproducibility tools miss: **reproducing the environment is necessary but not sufficient**. The code also has to *run*.

Consider what typically happens. A researcher shares a `renv.lock`, or even a Nix expression pinning every dependency to the exact version used. A colleague clones the repository, reconstructs the environment perfectly, and… opens the project to find a folder full of scripts with no obvious order of execution.

Which script comes first? Which outputs feed into which inputs? Are there manual steps in between? The README might answer some of these questions, or it might not. Reproducing the environment got them 80% of the way there, but that last 20% is still a puzzle they have to solve by reading someone else's code.

This is the difference between *environment reproducibility* and *computational reproducibility*. The former means the same packages are available. The latter means the same computation can be re-executed, from raw inputs to final outputs, by anyone with access to the code, without prior knowledge of the project.

Computational reproducibility requires something the environment alone cannot provide: an explicit description of the *pipeline*. Not prose in a README. Not conventions like "always run `01-clean.R` before `02-model.R`". An actual, machine-readable graph of computations with explicit dependencies, where running one command re-executes the whole analysis in the correct order.

4.1.3 The Mindset Shift: From Tasks to Rules

There is a deeper point lurking here, one that goes beyond the technical details of package managers and lockfiles.

Think about two data scientists on the same team. Both write R or Python. Both know their tools. But they have fundamentally different mental models.

The first thinks in **tasks**: *I need to clean this dataset. I need to fit this model. I need to produce this report.* He writes a script for each task, runs them manually in the right order, and the

analysis is done. If someone else needs to reproduce it, our friend shares the scripts and a README and hopes for the best.

The second thinks in **rules**: *What are the general rules that govern this analysis? What are the inputs? What are the outputs? What depends on what? If the data changes, which steps need to re-run? If a colleague on a different machine runs this, will he get the same result?* He writes code that describes the *system*, not just the individual actions.

This is not a question of technical sophistication. It is a question of mental model. The first person sees the computer as a place where tasks happen. The second sees the computer as a machine that can be taught to follow rules repeatably, consistently, and at scale.

The shift matters enormously for reproducibility. If your analysis is a sequence of manual tasks, it is inherently fragile: it depends on you remembering the order, running the right scripts, and not making any ad hoc changes along the way. If your analysis is a system of explicit rules (a formal description of what produces what), then it can be re-executed by anyone (including AI agents), on any machine, at any point in the future, with a single command.

This framing is not unique to data science. Institutional economics argues that the rules of the game (formal and informal) shape collective outcomes more than individual talent (North 1990). Behavioural economics shows that the design of the choice environment steers behaviour as much as persuasion does (Thaler and Sunstein 2008). Legal scholars have noted that code itself regulates, because constraints written in software are harder to ignore than rules written in prose (Lessig 2006). And design research has catalogued how the affordances of everyday tools

silently determine what their users can and cannot do (Norman 2013).

A pipeline is the same idea applied to an analysis: the institutional design of the computation, a choice architecture that makes the reproducible path the path of least resistance.

This is the insight that motivates the pipeline-first approach in this book. Before we even talk about packages and environments, we are asking: *what is the structure of this computation?* Making that structure explicit, formal, and machine-readable is the only approach that delivers genuine computational reproducibility, and, as we will see, it is also the approach that works best with AI-assisted development.

4.1.4 Thinking in Systems in an AI-First World

If the previous section sounded abstract, consider what is happening right now to the practice of data science.

AI coding assistants have become genuinely capable. For many routine tasks such as data cleaning, visualisation, standard modelling, an LLM can write solid R or Python code faster than most humans. The cost of *writing* code is collapsing. This means the skill of typing correct syntax is becoming less valuable, and the skill of knowing *what to ask for* is becoming more valuable.

But here is the catch: knowing what to ask for requires domain knowledge that no LLM reliably has, or rather, doesn't reliably surface to you. Take a simple example. You are working with time series data on prices. Ask an LLM to clean and graph the data, and it will almost certainly produce competent code. But will it suggest deflating nominal prices to make them comparable across time? Maybe, maybe not. That depends on whether

the LLM has encountered enough economic methodology in its training data, and whether *you* thought to ask. And crucially, will you recognise when the LLM's answer is subtly wrong? You don't know what you don't know. And neither does the LLM, but, unlike you, it won't flag its own blind spots… or perhaps it will, depending on its training. But you can't rely on LLMs *perhaps* telling you what you need to pay attention to.

This points to a skill that is going to matter more, not less, as AI matures: the ability to interrogate and challenge an LLM's output. An AI assistant is confident by default. It will produce plausible-sounding analysis regardless of whether the underlying reasoning is sound, and even when it does provide some points you need to pay attention to: how can you be sure those are the right things to pay attention to? The person who gets value from it is not the one who accepts the first answer, but the one who can ask *"wait, did you account for inflation here?"*, or *"why did you choose this model over that one?"*, or *"what assumptions are you making about the data generating process?"*. That requires knowing enough to ask those questions.

This has an interesting implication for how we should think about expertise. The traditional advice has been to specialize: know one language deeply, one domain deeply, one tool deeply. That advice was right when expertise was expensive to acquire and hard to substitute. But in a world where an LLM can write the implementation for you, having some knowledge across many areas may become more valuable than deep specialisation in one. A data scientist who knows a little economics, a little software engineering, a little domain-specific methodology, and can draw on all of it to design and audit what the LLM produces, may outperform a narrower specialist who can no longer leverage their deep implementation skills because the LLM already has them.

I will close this section with a personal anecdote that I think illustrates the point well. Alongside this book, I have been building a Game Boy game using LLMs to write essentially all the code. I know nothing about Game Boy development: I do not know the hardware architecture, I have never written a line of assembly, and I have certainly never shipped a cartridge. But I approached the project exactly as I would approach a data science project.

The first thing I did was set up a reproducible development environment. The second was to make the LLM build tooling to *query the game's state programmatically*: not just run the game, but interrogate it. What is the player's position? What are the active sprites? What is the current game phase? With that infrastructure in place, I could instruct the LLM to run test scenarios from any point in the game, with any parameters, and get structured feedback. The game could be developed and debugged in a principled loop rather than by manually playing through it each time.

The result: an actual, running Game Boy game (well, more of a demo really, but still), built by someone who cannot write the underlying code. What I contributed was the *systems thinking*: insisting on reproducibility, insisting on programmable state inspection, insisting on a feedback loop that could be automated. The LLM contributed the implementation. Neither of us could have done it alone.

This is the approach that this book is trying to teach for data science. The specific tools matter less than the underlying mental model: make the structure of your computation explicit, build in the ability to inspect and query it at every stage, and create feedback loops that you and your AI collaborators can iterate on together.

4.1.5 The Polyglot Challenge

Modern data science is increasingly polyglot. Research shows that data scientists use, on average, nearly two programming languages in their work, with R and Python being the most common combination (see Chen et al. 2025). Python dominates machine learning, R excels at statistical modelling, and Julia offers high-performance numerics. Projects increasingly combine these strengths.

This creates a reproducibility challenge: a project using R, Python, and Quarto requires coordinating multiple package managers. Nix solves this by providing a unified framework for all languages and system tools. But there is still a missing piece: something to *orchestrate* all these moving parts within a single, coherent and reproducible workflow.

4.1.6 Enter T

However, Nix alone has a steep learning curve. Its functional programming language can be daunting for researchers focused on their analysis, not system administration.

I know this problem well. My first attempt at solving it was `{rix}`: an R package that generates Nix expressions from intuitive R function calls. You describe *what* you want, and `{rix}` figures out *how* to express it in Nix. I then built `{rixpress}` on top of it, an R package for defining reproducible, polyglot analytical pipelines. Both packages solved real problems, but they were fundamentally R-focused. `{rix}` needed R to run, and the pipeline syntax of `{rixpress}` was R code. If you worked in Python or Julia primarily, you were a second-class citizen.

T is what I built next, and it goes far beyond both. T is a reproducibility-first domain-specific language (DSL) for polyglot data science. It is *not* an R package. It is its own language, with its own runtime, its own REPL, and its own ecosystem of packages, built from the ground up to treat R, Python, Julia, and Shell as equals. T is meant to orchestrate R, Python and Julia, but critically, T does not just *orchestrate* environments defined elsewhere: it *is* the environment definition system. Every T project is a Nix flake, and T manages the entire stack: language runtimes, system dependencies, and pipeline execution under one roof.

T provides a functional, immutable language for constructing composable *micropipelines*: first-class, introspectable computation graphs that coordinate R, Python, Julia, Quarto, and Shell execution within a unified system. Pipelines in T are not configuration artifacts but executable program structures with explicit dataflow, typed nodes, and content-addressed outputs. And unlike any other language used for data science, where reproducibility is more often than not bolted on, T makes reproducibility impossible to opt out of: every node in a T pipeline runs in its own hermetic Nix sandbox, and every output is content-addressed by design. As far as I know, T is literally the first reproducibility by design programming language.

There is a broader current here. The history of computing is a history of raising the level of abstraction: from telling the CPU exactly what to do, to describing data transformations, to, with AI, describing the *system*, its *constraints*, and your *intent*. We are rediscovering that the oldest Unix philosophy was right all along: everything should have an explicit, textual representation that can be composed into larger systems. And once the design of your project is explicit, whether the implementation is R, Python, or Julia becomes almost an implementation detail. That

is the main design principle behind T.

And here is an important reassurance if you already live in R, Python, or Julia: **you do not have to use T's own built-in functions at all.** T is a superset, not a replacement. You can write every node's logic in the language you already know and let T do exactly what it is best at: orchestrating those nodes into a reproducible pipeline. T's data-manipulation verbs and other builtins are there when you want them, but for pure orchestration they are entirely optional.

A crucial point to internalise: **the primary way to work with T is to write Pipelines.** T is not, first and foremost, a scripting language. If you write a plain script full of commands and try to execute it with `t run`, it will not run at all: non-interactive execution requires a pipeline. You can override this with the `--unsafe` flag (`t run --unsafe script.t`), but that is really not recommended. Think of it as a workaround for little helper scripts, for example on CI.

That said, T does come with a REPL. In the REPL you can run commands interactively, exactly the way you would in R or Python. Later in the book (Chapter 14), we will see how T's own data-manipulation verbs let you wrangle data, and you can do all of that right inside the REPL.

T is also designed to treat pair-programming with LLMs, or even delegating the entire pipeline-writing process to an LLM, as a first-class programming model. To support this, T provides built-in mechanisms for LLMs to introspect and query build artifacts with ease. Each T project also includes a dedicated `AGENTS.md` file that guides the LLM on how to properly write and work with T pipelines. Pair programming with LLMs will be studied later, though. A plain-text, explicit pipeline is what makes this work: it is far easier for a model to read, write, and

verify a diffable description of the computation than to reason about the hidden state of an interactive notebook (see Chapter 3).

The workflow is simple:

1. **Bootstrap** a T project using `t init`
2. **Declare** your dependencies in `tproject.toml`
3. **Define** your R, Python or Julia functions to perform the actual analysis
4. **Write** (or better yet, have your AI agent write for you) your reproducible pipeline as a T pipeline in `src/pipeline.t`
5. **Build** and run with `t run`

4.1.7 A candid note: who this is for (and who it isn't)

Before we get hands-on, let me be straight with you about where T stands.

T is young. It is new, still in beta, and honestly niche. Its API and some of its behaviour will keep changing as it matures. If you are looking for a battle-worn tool with a decade of production scars and a huge community, T is not it yet.

I built it for myself first. I designed T to solve my own problems: the polyglot reproducibility pain I kept hitting in my own work, where R, Python and Julia all had to coexist in one reproducible environment. That is also why I am so motivated to keep building it: it is the tool I wish I had had, and I use it every day. This book is, in a sense, the documentation I wanted to exist. I am writing it for myself first, and making it available in the hope that you find it useful too.

You do not have to switch. If `{targets}` (Landau 2021) (for pure-R work) or Snakemake (Köster and Rahmann 2012) (for polyglot work) already serve you well, that is a perfectly good choice. Both are robust, well documented, and backed by large communities. The core ideas in this book (Nix-managed environments, an explicit pipeline, and testing) apply no matter which orchestrator you pick. T is an option, not a requirement, but one I would urge you to consider, as I believe it tackles the problem of reproducibility *correctly*, thanks to being built on top of Nix.

What T adds is that the *environment* and the *pipeline* are the same language: one file defines your exact, bit-for-bit reproducible stack *and* the computation that runs on it, and the pipeline is a first-class value you can inspect and transform. It is also designed to be written and verified by LLMs. Here is a fair comparison:

	T	**targets**	**Snake-make**	**rix / rixpress**
Languages	R, Python, Julia, Shell	R (mostly)	Python rules + any via envs	R (Python via `ryxpress`)
Environment management	Yes: Nix-based, env + pipeline in one	No: pair with `{renv}`	Partial: conda envs	Yes: Nix-based
Pipeline as first-class data	Yes: query, filter, transform, compose	Limited	Limited	Limited

	T	**targets**	**Snake-make**	**rix / rixpress**
Maturity / commu-nity	Young, in beta, small	Very mature, large	Very mature, large	Moderate
Learning curve	Low for R users (new DSL)	Low for R users	Moderate (Snakefile)	Moderate
Designed for LLMs	Yes	No	No	No

So read this book as one person's answer to the reproducibility puzzle. Take what helps; and if T itself is not for you, the Nix and pipeline ideas still are.

This chapter covers everything you need to know to create project-specific, reproducible development environments and pipelines for your polyglot data science projects. The next section focuses on T, and its syntax; but I want to be very clear. You do not need to write your analyses in T! You can continue working with R, Python or Julia, and only use T for orchestration. But T also has some interesting features as a language, and you might want to take a look at its syntax and capabilities.

4.2 A Quick Tour of T Syntax

Before we build our first pipeline, here is a fast tour of T's syntax. T is a small, functional language, so the surface area is modest. You do not need to memorise any of this now, treat it as a map you can return to as the chapters unfold.

4.2.1 Variables and assignment

T binds variables with `=`. Values are immutable by default; to rebind an existing name (shadowing), use `:=`:

```
x = 10
name = "Alice"
x := 20    -- rebinds x to 20
```

4.2.2 Conditionals

Conditionals are expressions. They return values:

```
result = if (x > 5) "high" else "low"
```

4.2.3 The two pipes

T has two pipe operators, and the difference between them is one of the language's defining ideas: **errors are values**.

- `|>` passes the left-hand value to the right-hand function and **short-circuits on error**: if the left-hand value is an `Error`, the pipeline stops and the error is returned without calling the function.
- `?|>` (the maybe-pipe) **always forwards** the left-hand value to the right-hand function, even if it is an `Error`. This is how you write explicit recovery.

```
[1, 2, 3] |> map(\(v) v * v) |> sum -- 14
error("boom") |> double              -- Error:
 ↪  short-circuits

handle = \(x) if (is_error(x)) "recovered" else x
error("boom") ?|> handle                  --
 ↪  "recovered"
```

`double()` will not even run, since `|>` short-circuits. However, in the second example, `handle()` will get the error object passed to it because of `?|>` and will do something with it (whatever you programmed `handle()` to do with errors).

4.2.4 Imports

If you create a T package, you can import it in a pipeline, or import specific functions from it. You can also import functions defined in plain scripts:

```
import mypackage
import mypackage [fn1, fn2]
import "helpers.t" [clean_data, normalize]
```

Note that to use R, Python or Julia packages, the import statements of each respective language need to be written inside the required `<{...}>` foreign code blocks, as we shall see below.

4.2.5 Sequence generation

Inspired by R:

```
seq(5)                  -- [1, 2, 3, 4, 5]
seq(1, 10, by = 2)      -- [1, 3, 5, 7, 9]
```

4.2.6 Operators at a glance

The tour so far has met operators one at a time. Here is the whole set in one place, listed in order of precedence (loosest first), so a pipe chain reads left to right without parentheses, and `1 + 2 * 3` still means `1 + (2 * 3)`.

Operator	Job	Notes
`=`, `:=`	assign, rebind	`:=` shadows an existing name
`if (c) a else b`	conditional expression	always returns a value
`|>`, `?|>`	pipe, maybe-pipe	`|>` short-circuits on `Error`; `?|>` forwards it
`~`	formula	`y ~ x`; the language of models and `case_when`
`||`	scalar OR	both sides must be Bool
`&&`	scalar AND	both sides must be Bool
`|`	scalar bitwise OR	scalar operands only
`.|`	element-wise OR	Lists, Vectors, NDArrays
`&`	scalar bitwise AND	scalar operands only

Operator	Job	Notes
`.&`	element-wise AND	Lists, Vectors, NDArrays
`==`, `!=`, `<`, `>`, `<=`, `>=`	scalar comparison	`==` on a collection is a `TypeError`
`in`	membership	`x in [1, 2, 3]`; the right side is a list
`%name%`	custom infix operator	R-style; packages define them, you consume them
`.==`, `.!=`, `.<`, `.>`, `.<=`, `.>=`	element-wise comparison	`1 .== [1, 2]` is `[true, false]`
`+`, `-`	scalar add, subtract	int + float promotes to float
`.+`, `.-`	element-wise add, subtract	the workhorses of column math
`*`, `/`, `%`	scalar multiply, divide, modulo	
`.*`, `./`, `.%`	element-wise multiply, divide, modulo	
`.`, `()`	field access, function call	tightest of all

Unary operators sit above everything in the table: `-` (negation), `!` (logical not), and `!!` / `!!!` (unquote and unquote-splice, the metaprogramming tools from Chapter 15, heavily inspired by R's `{rlang}`).

Two distinctions matter more than any single operator. First, the **scalar-versus-broadcast split**: the plain operators (`+`, `==`, `&`, …) demand scalar operands and say so with a `TypeError`, while

the dotted operators (`.+`, `.==`, `.&`, …) do the work element-wise across Lists, Vectors, and NDArrays. No silent coercion, no surprises. Second, the pipes bind *loosest* of all, which is what lets you write long chains without parenthesising anything.

4.2.7 What can a T value be?

Every expression in T evaluates to a value, and the set of values is small enough to hold in your head:

Value	Example	Where it is covered
Int, Float, Bool, String	`42`, `3.14`, `true`, `"hi"`	everywhere
NA	`NA`	missing data, Chapter 14
Error	`error("boom")`	errors as values, Chapter 6
Symbol	a bare name captured as data	non-standard evaluation, Chapter 15
List / Dict	`[1, 2, 3]`, `[name: "Alice"]`	collections, Chapter 15
Vector	a typed numeric or text vector	columns, Chapter 14
NDArray	an n-dimensional array	Chapter 14
DataFrame	`read_csv("data.csv")`	Chapter 14
Factor	a categorical column	Chapter 14
Date / Datetime	`ymd("2024-01-01")`	Chapter 14
Formula	`mpg ~ wt`	models, Chapter 14

Value	Example	Where it is covered
Lens	`get(mtcars, col_lens("mpg"))`	Chapter 15
Lambda	`\(x) x + 1`	functions, Chapter 15
Pipeline / Node	`pipeline { ... }`	Chapters 5 and 8
Intent	`intent { ... }`	Chapter 15

Most of the rest of this book is a tour of this table, one row at a time.

4.2.8 How T evaluates

Three ideas make T's behaviour predictable once you see them.

The data mask. Inside a verb, column names resolve against the data the verb is operating on. That is why `df |> filter($age > 30)` works even though `age` is not a variable in scope: `filter` masks the columns of `df` into the expression, and `$age` picks one out. You write expressions *over columns*, not over R or Python objects.

Non-standard evaluation. Some verbs need the *expression*, not its value. `mutate($bonus = $salary * 0.1)` captures the right-hand side and evaluates it row-wise later; a bare name in that position is captured as a symbol, not looked up. The explicit tools for this (`quo`, `!!`, `!!!`) are discussed in Chapter 15; the verbs in Chapter 14 use it under the hood.

Errors as values. A failing expression does not crash the program; it produces an `Error` value you can test with `is_error`, branch on with `if` or `match`, or route with `?|>`. That is what

makes the two pipes possible, and it is why a long pipeline can degrade gracefully instead of dying at the first bad row.

4.2.9 Types, briefly

You will meet annotations like `\(x: Int, y: Int -> Int)` in later chapters. T has a light but real type system: parameter and return annotations, generic type variables, a strict mode that requires full signatures on top-level functions, and semantic types like `DataFrame[schema]`. We take it apart in Chapter 15; for now, read an annotation as a promise about what a function accepts and returns.

T's functions (how to define them, closures, and higher-order functions) are a topic in their own right, and we take them up properly in Chapter 15.

4.3 Transitioning to Nix-Managed R

Now that Nix is installed, I strongly recommend uninstalling any system-wide R installation and removing the packages in your user library (typically found in `~/R` on Linux or `~/Library/R` on macOS). From this point forward, let T and Nix handle everything. If you are using Windows, you can keep your Windows-specific R installation, since Nix will not interfere with it (remember, Nix is installed inside WSL and Positron will automatically load Nix environments installed in WSL as well).

If you are not ready to take this step, you can still use T: it manages its own Nix-sandboxed R environment per project node, so it will not interfere with any existing R installation. However,

for the cleanest experience and to avoid potential subtle conflicts, I recommend fully committing to Nix.

4.3.1 Bootstrapping T without a local installation

T is distributed exclusively via Nix. You don't need to install it in the traditional sense. Instead, you launch a temporary shell that provides the `t` executable, use it to scaffold a new project, and then let the project itself manage the T version it uses, which is pinned in the project's `flake.lock`.

Running the following line in a terminal will drop you into an ephemeral shell with `t` available:

```
nix shell --accept-flake-config
 ↪  github:b-rodrigues/tlang
```

This gives you a temporary shell with `t` ready to use. From here, you can scaffold any new project. For example, navigate to your projects directory and initialize a new project:

```
t init --project my-analysis
```

This creates a new directory `my-analysis/` containing the necessary project files. When prompted, you will be asked for basic project information and an **AI Agent Context Level** (Small, Medium, Full, or Huge), which generates tailored reference documentation for LLMs. More on this in a later chapter.

You can then leave the temporary shell and enter the project's own reproducible environment:

```
exit
cd my-analysis
nix develop
```

That last command, `nix develop`, is going to be very important. It is the command that enters the project's development shell, which provides the pinned version of `t` alongside all declared R, Python, and Julia runtimes. All subsequent commands should be run inside this shell. To leave that shell, type `exit` just like we did above to leave the temporary shell to bootstrap our project.



Getting LLM assistance with T

As already mentioned, T is designed from the ground up for AI-assisted development. When you initialize a new T project, two files are automatically generated in the project root:

- **`AGENTS.md`**: A project-specific onboarding guide that tells LLMs how to work within your project's architecture.
- **`T-LANGUAGE-REFERENCE.md`**: A tiered technical reference for the AI to read, tuned to the context level you chose at `t init`.

With these files, any AI agent you pair-program with has immediate access to the exact technical context it needs to write correct T pipelines. You can also supply additional context by pointing your LLM at the T documentation website[2].

[2]https://tstats-project.org

4.4 The T Project

Every T project has the following directory structure:

```
my-analysis/
  tproject.toml       # Project configuration and
dependencies
  flake.nix           # Reproducible environment
definition
  flake.lock          # Locked dependency versions
  README.md           # Project overview
  AGENTS.md           # Onboarding guide for AI
Agents
  T-LANGUAGE-REFERENCE.md # Tiered language
reference for LLMs
  src/
     pipeline.t      # Your main analysis script
  data/               # Place your raw data files
here
  outputs/            # Output directory for
results
  tests/              # Unit tests for your
analysis
```

The two most important files are `tproject.toml` and `src/pipeline.t`.

4.4.1 Declaring Dependencies with `tproject.toml`

T projects are explicit: R, Python, and Julia packages belong in `tproject.toml`, not in ad-hoc `install.packages()`, `pip`

`install`, or `Pkg.add()` calls. Open `tproject.toml` and add the runtime packages you need:

```
[r-dependencies]
packages = ["dplyr", "ggplot2"]

[py-dependencies]
version = "python313"
packages = ["polars", "scikit-learn"]

[jl-dependencies]
version = "lts"
packages = ["DataFrames", "CSV", "GLM"]
```

For Julia, the `version` field defaults to `"lts"` (the current Julia long-term-support release). To pin a specific Julia version, use formats like `"1.10"` or `"1.11"`, which map to `julia_1_10` or `julia_1_11` in Nixpkgs.

After editing `tproject.toml`, sync the project environment:

```
t update
```

You might get the following error:

```
Error: Git working directory is not clean. Commit or
 ↪  stash changes first.
```

This is because each T project also gets automatically tracked by Git. I won't talk about how to set up Git, but this is another way that T is a reproducibility-first language: the projects must be version controlled! Running `t update` will generate the

`flake.nix` and `flake.lock` files. These files are what define the computational environment your pipeline will run in. These also need to be tracked by Git. You will get this error again:

```
bash-5.3$ t update

No T package dependencies declared in tproject.toml;
 ↪  project defines 3 R
dependencies, 4 Python dependencies and 2 additional
 ↪  tools.

Syncing 0 T dependencies, 3 R dependencies, 4 Python
 ↪  dependencies, 0 Julia
dependencies and 2 additional tools from
 ↪  tproject.toml → flake.nix... Running
nix flake update... Error: Failed to update
 ↪  dependencies: Command 'nix flake
update --accept-flake-config' failed (exit 1):
 ↪  error: Path 'basic_t/flake.nix'
in the repository "/home/user/projects/project-1" is
 ↪  not tracked by Git.

       To make it visible to Nix, run:

       git -C "/home/user/projects/project-1" add
        ↪  "flake.nix"
```

Follow the instructions and then run `t update` again. This is a one-time setup, and if everything goes well, you should see this:

```
Running nix flake update...
flake.lock changes:
  - flake.lock created.
Updated.
```

Then exit the temporary shell, and re-enter the development shell so the updated package set is active:

```
exit
nix develop
```

4.4.1.1 Why NOT to use `install.packages()` / `pip install` / `Pkg.add()`

It's crucial to understand: **never call imperative installation commands from within a T project**. Here's why:

1. **Technically, it won't work:** Computations defined in your pipeline run inside hermetic Nix sandboxes. Anything not declared through Nix simply will not be available during pipeline execution.
2. **Conceptually, it defeats reproducibility:** The whole point of T's Nix integration is that your environment is fully defined by `tproject.toml` and `flake.lock`. Ad-hoc installations break this guarantee.

Instead, add packages to `tproject.toml` and run `t update`. Calling `install.packages()` (or equivalent commands for Python and Julia) will raise an error.

4.4.2 Working with R, Python, and Julia

Declaring dependencies in `tproject.toml` is not just bookkeeping: it gives you working R, Python, and Julia environments. Once you are inside the development shell, typing `R`, `python`, or `julia` starts the corresponding interpreter, with every package you declared already installed.

This makes the shell a natural place to do interactive work. You can develop small functions in R, Python, or Julia, test them, and then have your LLM agent wire them into the T pipeline. A setup that works well: start an editor such as Positron from inside the shell (for example, by typing `positron`), and work on your functions there while a terminal sits next to it, where your LLM agent builds the pipeline incrementally, node by node. Because the editor inherits the shell's environment, its R, Python, and Julia sessions use exactly the same versions and packages as your pipeline.

4.4.3 Alternative installation methods

T provides several other ways to install packages, using `uv` for Python or `{renv}` for R. By default, T uses Nix to resolve packages directly. We recommend sticking to the default resolver as much as possible. The default approach works exceptionally well for **Julia** and **R** because virtually all packages in their ecosystems are available through Nix. This is not the case for **Python**.

Python's PyPI contains hundreds of thousands of packages of varying quality. If you use Python for data analysis, you've likely hit this issue: you try to install one package that requires `numpy < 2`, and another that requires `numpy >= 2`. You're *cooked*, as

the youths say. The resolver can't help you because the requirements are literally incompatible. No amount of Rust-written package managers can solve this. The underlying issue is PyPI's ecosystem model.

In R, this situation rarely happens. CRAN enforces a system where packages are continuously tested against their reverse dependencies. If `{ggplot2}` or `{dplyr}` updates in a way that breaks other packages, CRAN catches it. Authors have two weeks to fix the issue, or their package gets archived. As a result, if a package is on CRAN, it works. CRAN manages this consistency across ~27,000 packages.

PyPI does not do this. It hosts packages without global consistency checks. If Package A and Package B declare mutually exclusive requirements, PyPI hosts them both anyway. Nix tries to curate Python packages, but because of PyPI's constraints, the entirety of PyPI cannot be made available through Nix in an automated way. Occasionally, you can patch a package's `pyproject.toml` in Nix to relax incompatible constraints (e.g., changing `numpy = "^1"` to `numpy = ">=1"`), but patching isn't a universal solution.

This is why T provides alternative resolvers (`uv` for Python and `{renv}` for R): * For Python (`uv`): Allows Nix to build the environment while fetching packages directly from PyPI via `uv`. This is especially useful if you are collaborating with non-T/Nix users who rely on standard `pyproject.toml` and `uv.lock` files. * **For R (`renv`):** Allows you to reuse existing `renv.lock` files for collaboration or legacy workflows.

4.4.3.1 Alternative Python Resolver: `uv`

Instead of declaring Python packages directly in `tproject.toml`, you can delegate dependency management to `uv`.

First, configure `tproject.toml`:

```
[py-dependencies]
resolver = "uv"
workspace = "python"
```

The workspace directory must contain your `uv` project metadata and lock file:

```
python/
  pyproject.toml
  uv.lock
```

The `version` field in `tproject.toml` is **optional** when using the `uv` resolver. If omitted, T infers the Nixpkgs Python attribute (e.g., `python312`) from the `requires-python` field in `python/pyproject.toml`.

The inference accepts specifiers that constrain to a single minor version (`==3.12`, `==3.12.*`, `~=3.12`, `>=3.12,<3.13`) and errors on open-ended ranges (`>=3.12`). If an explicit `version` conflicts with `requires-python`, T prints a warning and uses the explicit version.

When using `resolver = "uv"`, do **not** set `[py-dependencies].packages`. Python dependencies are declared exclusively in `pyproject.toml` and locked by `uv.lock`. Running `t update` generates `uv2nix/pyproject.nix` inputs in `flake.nix` and builds the Python environment as a Nix virtual environment.

If you don't have `uv` installed locally and lack existing metadata:

1. Edit `tproject.toml` to set the resolver:

```
[py-dependencies]
resolver = "uv"
workspace = "python"
```

2. Create a minimal `pyproject.toml` inside the `python/` directory:

```
# python/pyproject.toml
[project]
name = "my_project_python_env"
version = "0.1.0"
requires-python = ">=3.12,<3.13"
dependencies = [
    "pandas",
]
```

3. Enter a temporary Nix shell to generate the lock file using `uv`:

```
nix shell nixpkgs#uv nixpkgs#python3
uv lock --project python
exit
```

4. Re-enter your T development shell and update:

```
nix develop
t update
```

This reads the `uv` workspace, adds `pyproject-nix`, `uv2nix`, and `pyproject-build-systems` inputs to `flake.nix`, and configures the Python environment to use `pyProject.mkVirtualEnv`.

4.4.3.2 Alternative R Options: {renv} & Git Repositories

If your project already contains an `renv.lock` file, set:

```
[r-dependencies]
resolver = "renv"
```

When `resolver = "renv"`, T automatically discovers all R dependencies from `renv.lock`:

- **CRAN packages** are read from each entry's `Repository` or `Bioconductor` field and mapped to `pkgs.rPackages.*`.
- **GitHub/GitLab packages** are fetched via `builtins.fetchGit` using the `RemoteHost`, `RemoteUsername`, `RemoteRepo`, and `RemoteSha` fields (supports `api.github.com` and `gitlab.com`).
- **Remotes** are parsed and injected as `buildInputs` for dependent Git packages.
- Base R packages (`R`, `methods`, `stats`, etc.) are automatically filtered out.

No `packages` list is required in `tproject.toml`; `renv.lock` serves as the single source of truth. Run `t update` to regenerate `flake.nix`.

> **Note:** This does not install the *exact* package versions locked in `renv.lock`. Instead, it installs the package versions available in Nixpkgs at the date defined in your `tproject.toml` (`[nixpkgs].date`).

To install R packages directly from remote Git repositories (e.g., GitHub), declare them directly in `tproject.toml`:

```
[r-dependencies]
packages = ["dplyr"]
my_pkg = { git = "https://github.com/user/my-pkg",
 ↪  rev = "abc123def456" }
```

The `rev` field must be a full Git commit hash. Each Git package is injected into every R pipeline node's `buildInputs`. When using `resolver = "renv"`, Git packages declared in `tproject.toml` are automatically merged with those in `renv.lock`.

4.4.4 System Dependencies and LaTeX

Beyond language packages, you can declare system tools and LaTeX packages required by your project.

4.4.4.1 Additional Tools

Use `[additional-tools]` to make CLI utilities, compilers, or system libraries available in your shell and pipeline sandboxes:

```
[additional-tools]
packages = ["git", "jq", "gawk", "pandoc", "typst"]
```

4.4.4.2 LaTeX Support

If your project generates PDFs or reports, use the `[latex]` section. T automatically provides a `texlive` environment (starting from `scheme-small`). Simply list any extra LaTeX packages needed:

```
[latex]
packages = ["amsmath", "blindtext", "physics",
 ↪  "hyperref"]
```

4.5 Summary

Traditional tools like `{renv}` or Python's `venv` only capture part of the reproducibility puzzle. They track package versions but not the language version itself, nor system-level dependencies like GDAL or Java. This means your project can still break on a different machine, or even on your own machine after a system update.

Nix solves this by managing *everything*: R, Python, all packages, and all system dependencies. T builds on top of Nix to solve the *orchestration* problem: it provides a language for describing *how* computations in different runtimes interact, and it enforces reproducibility at every step.

When you define a pipeline with T, you get a complete, self-contained specification that anyone can use to recreate the exact same results, on any machine, at any point in the future.

In the next chapter, we will get our hands on the machinery and **build our first pipeline**.

5 Your First T Pipeline

5.1 Introduction

In the previous chapter we set up a T project: we bootstrapped T, declared dependencies in `tproject.toml`, and entered the project's `nix develop` shell. Now it is time to actually build something.

This chapter is deliberately hands-on. We will write a pipeline, build it, break it, inspect it, fix it, and extend it. Theory comes later; Chapter 6 explains *why* pipelines work the way they do. Here the goal is to get your hands on the machinery and get comfortable with the core development loop.

5.2 How a T Pipeline Executes

Before writing code, it helps to understand the five stages every T pipeline goes through from declaration to result:

1. **Declare.** You define nodes inside a `pipeline { ... }` block. Each node gets a name, a command, a runtime, and a serializer.

2. **Analyze.** T scans the DAG: it resolves which node depends on which, detects cycles, and extracts identifier references from foreign-code blocks for dependency tracking.
3. **Emit.** T generates one Nix derivation per node. Each derivation bundles the right runtime (R, Python, Julia, Quarto, or shell), the packages from `tproject.toml`, and the serializer/deserializer glue code.
4. **Build.** Nix executes each derivation in a hermetic sandbox. Upstream artifacts are already materialized in the Nix store, so the deserializer reads them in and the serializer writes the node's output, no shared memory, no runtime coupling.
5. **Read back.** After `build_pipeline(p)` completes, use `read_node(p.name)` to load any node's artifact back into T for inspection.

Key insight: A pipeline is a *declarative build graph*, not a script. Nodes describe *what* to produce, not *when* to compute. T handles the ordering, caching, and reproducibility: the same pipeline produces the same results on any machine.

5.3 The Simplest Possible Pipeline

Open `src/pipeline.t` and write:

```
p = pipeline {
  x = 10
  y = 20
  total = x + y
}
```

```
build_pipeline(p)
```

This pipeline doesn't use any R, Python or Julia code yet, but is written in pure T. As shown in the previous chapter, T comes with many builtin functions, and even data manipulation verbs inspired by `{dplyr}`! We return to them in Chapter 14.

Now let's run the pipeline. There are two ways of doing it. From the terminal (inside of the environment):

```
nix develop
t run src/pipeline.t
```

or inside an interactive T session, using `t_make()`.

T resolves the dependency graph (`total` depends on `x` and `y`), builds each node, and caches the results. Access any node's value with dot notation in the REPL:

```
p.x      -- 10
p.y      -- 20
p.total  -- 30
```

The pipeline itself prints as:

```
Pipeline(3 nodes: [x, y, total])
```

Note that the order of node declarations does **not** matter. T infers the execution order from the data dependencies. You could write `total` before `x` and `y` and the result would be identical.

5.4 The Core Development Loop

A productive way to build pipelines is incrementally: start small, build, inspect, extend:

```
t check src/pipeline.t   # validate cheaply, catches
 ↪  errors in milliseconds
t run src/pipeline.t     # build when clean
```

Inside the REPL (`t` or `t repl`) you can inspect results immediately after a build:

```
read_node(p.total) -- reads the serialized artifact
 ↪  back from the Nix store
show_plot(p)       -- visualise the DAG
```

Add a node and rebuild:

```
p = pipeline {
  x     = 10
  y     = 20
  total = x + y
  label = str_sprintf("Sum is %d", total)
}
build_pipeline(p)
```

Because `x`, `y`, and `total` were already built and cached, Nix skips them and only computes the new `label` node. Run `build_pipeline(p, dry_run = true)` to preview which nodes will be rebuilt versus served from cache before committing to a full build.

This **check → build → inspect → extend** loop is the core iterative development cycle in T. Get comfortable with it before moving on. It is also possible to interactively inspect R, Python or Julia nodes. More on this later.

Once the pipeline is built, you still need to get to the build artifacts. For this, use `pipeline_copy()`. This copies all computed node artifacts from the Nix store to a local directory for easy access. By default, it copies all nodes from the latest build to the "pipeline-output/" folder. You can specify a custom folder using the `target_dir` argument: `pipeline_copy(target_dir = "outputs")`

5.5 Node Types: Calling R, Python, and Julia

The power of T is that nodes can run in any runtime. The node wrapper functions make this explicit:

Function	Runtime	Use case
`node()`	T native	Data manipulation, logic, native T operations (optional)
`rn()`	R	Statistical modelling, tidyverse, Bioconductor
`pyn()`	Python	Machine learning, scikit-learn, PyTorch

Function	Runtime	Use case
`jln()`	Julia	High-performance numerics, Flux.jl
`shn()`	Shell	File processing, calling external CLIs

Foreign-language code lives inside `<{...}>` raw code blocks, which pass code verbatim to the target runtime without T parsing it:

```
p = pipeline {
  -- Load data in T
  data = read_csv("data/mtcars.csv", separator =
 ↪  "|")

  -- Summarise in R using dplyr
  summary_r = rn(
    command   = <{
      data |>
      dplyr::group_by(cyl) |>
      dplyr::summarize(avg_mpg = mean(mpg))
    }>,
    deserializer = ^ipc,
    serializer   = ^ipc
  )

  -- Count groups in Python using pandas
  counts_py = pyn(
    command   = <{ summary_r.groupby("cyl").size()
 ↪  }>,
    deserializer = ^ipc,
```

```
    serializer   = ^json
  )
}

build_pipeline(p)
```

T reads the R node's output (`summary_r`, serialized as `^ipc` Arrow IPC) and makes it available inside the Python node's sandbox under the same name. Serialization and deserialization happen automatically, you never write glue code to convert between R data frames and Python DataFrames.

5.6 Serializers

Every node in a T pipeline produces an artifact. The `serializer` argument controls the format of that artifact, and the `deserializer` argument controls how an upstream artifact is loaded into the current node's sandbox. Both use the `^` prefix:

Serializer	T Type	Best for
`^csv`	`DataFrame`	Portable tabular data between nodes
`^ipc`	`DataFrame`	Fast Arrow IPC hand-off between pipeline nodes
`^parquet`	`DataFrame`	Durable storage, archival, external analytics

Serializer	T Type	Best for
`^json`	`Dict` / `List`	Structured data interchange
`^pmml`	`Model`	R/Python models, native T evaluation
`^onnx`	`Model`	Python ML models, portable evaluation
`^text`	`String`	Plain text and shell output

Rule of thumb: use `^ipc` to pass DataFrames *between nodes while a pipeline runs* (fastest, uncompressed), and `^parquet` for anything you *persist, ship, or store* (4–10× smaller, compatible with DuckDB, Spark, pandas, etc.). Both work symmetrically across T, R, Python, and Julia.

When no serializer is specified, T picks a sensible default. When you are passing a DataFrame from an R node to a Python node, set `serializer = ^ipc` on the R node and `deserializer = ^ipc` on the Python node.

5.7 A Complete Worked Example

Let's build a small but complete polyglot pipeline: load data in T, fit a logistic regression in R, and read the results back.

First, make sure `tproject.toml` has the R packages we need:

```
[r-dependencies]
packages = ["dplyr"]
```

Then run `t update && exit && nix develop` to sync the environment.

Now write the pipeline:

```
-- src/pipeline.t
p = pipeline {
  -- Node 1: load data in T
  data = node(
    command    = to_dataframe([
      x = [1.0, 2.0, 3.0, 4.0, 5.0],
      y = [0L,  0L,  1L,  1L,  1L]
    ]),
    serializer = ^ipc
  )

  -- Node 2: fit a logistic regression in R
  model = rn(
    command = <{
      data$y <- as.factor(data$y)
      glm(y ~ x, data = data, family = binomial(link
 ↪  = "logit"))
    }>,
    deserializer = ^ipc,
    serializer   = ^pmml
  )
}

build_pipeline(p, verbose = 1)

-- Read results back into T and inspect
raw = read_node(p.data)
fitted = read_node(p.model)
```

```
print("Data shape:")
print(str_sprintf("%d rows × %d cols", nrow(raw),
 ↪  length(colnames(raw))))

print("Model coefficients:")
print(fitted.coefficients)
```

Run it:

```
t run src/pipeline.t
```

You should see the build log, then the data shape and model coefficients printed. Try changing the data values and re-running; only the affected nodes rebuild.

5.8 Inspecting the Pipeline

After a build, several tools let you look inside the pipeline from the REPL:

```
-- See the full DAG as a table
pipeline_to_frame(p)

-- See which node depends on which
pipeline_deps(p)

-- Visualise the dependency graph
show_plot(p)

-- Deep-dive into a specific node's metadata
```

```
inspect_node(p.model)
-- Returns: runtime, serializer, path in Nix store,
 ↪  dependencies, warnings

-- Read the raw Nix build log for a node
read_log("model")
```

`explain()` gives a quick summary of any node:

```
explain(p.model)
-- {
--   `kind`: "computed_node",
--   `name`: "model",
--   `runtime`: "R",
--   `path`: "/nix/store/...-model/artifact",
--   `serializer`: "pmml",
--   `class`: "glm",
--   `dependencies`: ["data"]
-- }
```

The `path` field is the escape hatch: the absolute path to the node's output in the Nix store. You can pass this to any external tool, or use the companion helper packages for R or Python to read T artifacts outside of T.

`explain()` reaches further than nodes: it describes any value, including DataFrames, errors, formulas, and whole pipelines. Chapter 15 takes it apart, alongside the `intent` blocks that record *why* a piece of code exists.

5.9 Comparing Builds with `t diff`

After a build you can compare it against any previous build to see exactly what changed:

```
t diff src/pipeline.t
```

This compares the most recent build against the one before it:

```
Name           Status    Class_a  Class_b
data           Unchanged T        T
model          Changed   T        T
```

The `Status` column tells you the blast radius of your last edit. `Changed` means the node's content-addressed hash is different: the artifact actually changed, not just the code. `Added` means a new node. `Removed` means a node was deleted from the pipeline definition.

This is more reliable than diffing source code: T diffs *outputs*, not inputs. If you refactored a node's code but the result is bit-for-bit identical (e.g. you renamed an internal variable), `t diff` shows `Unchanged`. If a small change cascaded through several downstream nodes, you see the full chain marked `Changed`.

5.10 What `t check` Catches

Before building, `t check` validates the pipeline structure without triggering any Nix builds. Validation runs in three tiers, each more expensive than the last:

Tier	Flag	What it checks	Cost
1	`t check`	Parse, DAG structure, node references, serializer coherence	milliseconds
2	`t check --schema`	Column names and types propagated across nodes	milliseconds
3	`t check --env`	Nix environment can be built for each node	seconds

Start with tier 1 and only escalate when you want deeper validation. Tier 2 is particularly valuable: it catches column-name typos in downstream nodes *before* you pay for a full Nix build:

```
t check --schema src/pipeline.t
```

Pass `--json` at any tier for structured output that agents and CI scripts can consume:

```
t check --json src/pipeline.t
```

```
{
  "error_class": "schema_mismatch",
  "node": { "id": "model", "lang": "R" },
  "message": "Column 'xval' not found. Did you mean
   ↪  'x'?",
  "caused_by": ["data"],
  "suggested_fix": { "kind": "rename_column",
   ↪  "old_name": "xval", "new_name": "x" }
}
```

Notice the `suggested_fix` field: when T can infer the correct action, it says so explicitly. `t fix` applies these mechanical fixes automatically:

```
t fix --dry-run src/pipeline.t   # preview what
 ↪  would change
t fix src/pipeline.t             # apply
```

Always preview before applying: a rename fix changes column references throughout the file. The `--dry-run` output shows exactly which lines are affected.

For continuous validation while actively editing, run `t check` in watch mode in a separate terminal:

```
t check --watch --schema src/pipeline.t
```

It re-runs automatically on every file save. Exit codes are consistent and scriptable:

Exit code	Meaning
0	Clean, no errors
1	Structural / DAG problem
2	Column / type mismatch
3	Nix environment problem

Make it a habit: **`t check` first, `t run` only when clean.**

5.11 Interactive Development with the T REPL in Positron

Non-interactive batch execution via shell commands like `t run src/pipeline.t` is perfect for automated builds, reproducible pipelines, and CI/CD runners. But humans rarely write data science workflows as single monolithic batch runs. Data analysis is inherently iterative: we need to explore intermediate outputs, check data shapes, inspect model coefficients, test function logic, and troubleshoot errors in real time. Thankfully, T provides a REPL (Read-Eval-Print Loop).

When working in Positron (or VS Code with the T extension), you can open an interactive `t` session directly in your editor terminal. Running `t` (or `t repl`) inside Positron gives you a live, interactive environment where you can execute pipeline definitions, evaluate `build_pipeline(p)` on the fly, and inspect nodes immediately. For this, you do need to configure your editor. Please follow the instructions over at https://github.com/b-rodrigues/tlang/blob/main/docs/editors.md to correctly configure your editors.

5.11.1 Inspecting Live Nodes: `read_node()`

Once a pipeline is built in your session, you can inspect any node's computed result using `read_node()`. Suppose that we are working on the pipeline from before. To run it from the REPL, use `t_make()`:

```
t_make()
```

You can now read the contents of individual nodes:

```
read_node(p.data)
```

Note that `read_node()` strictly expects a `ComputedNode` reference (such as `p.data` or `p.stats`), not a raw string name.

5.11.2 Inspecting Historical Builds: `read_past_node()`

What if you want to inspect a node's output from a previous build, or examine artifacts when the pipeline object `p` is no longer in your active REPL memory?

T provides `read_past_node()` for temporal introspection:

```
read_past_node(p.data, which_log = "2026-08-14")
```

Unlike `read_node()`, which looks up live in-scope nodes from the active pipeline object, `read_past_node()` captures the node identifier non-evaluatively and queries T's historical build logs in `_pipeline/logs/`. The mandatory `which_log` parameter specifies a timestamp or log identifier pattern. This allows you to inspect past run outputs, audit how dataset values evolved over time, or inspect artifacts from previous sessions without re-running the pipeline.

5.11.3 High-Fidelity Representation of Foreign Objects

One of the greatest benefits of the T REPL is how it displays results from foreign runtimes. Whether a node was computed

in pure T, R (`rn`), Python (`pyn`), or Julia (`jln`), `read_node()` automatically deserializes the underlying artifact (Parquet, Arrow IPC, JSON, PMML, etc.) and converts it into a native, high-fidelity T representation.

When you call `read_node()` on a foreign node from the T REPL, T formats the output cleanly:

- **DataFrames** (from R `tibble`/`data.frame`, Python `pandas`/`polars`, or Julia `DataFrame`) print with column names, data types, and row/column counts.
- **Models** (such as an R `glm` serialized as PMML or a Python `scikit-learn` model) display structured model metadata and tidy coefficient tables via `fitted.coefficients`.
- **Structured collections** (Lists and Dicts) display as clean, readable trees without low-level string formatting artifacts.

You get rich, instant feedback inside your interactive Positron session without writing custom glue code or manually serializing objects to temporary files. If the node produces an artifact that T doesn't know how to deserialize, it is still possible to start a temporary R or Python (or Julia) shell to inspect that object, using `debug_node()`. Keep reading.

5.11.4 Stepping into Guest Runtimes: `debug_node()` (A Debugging Appetizer)

Sometimes inspecting an output from the T REPL isn't enough, especially when a foreign script throws an unexpected runtime error or requires step-by-step code execution.

T provides `debug_node()` to drop you directly from the T REPL into an interactive guest subshell for that specific language:

```
debug_node(p.stats)
```

Calling `debug_node()` automatically:

1. Resolves all upstream dependency artifacts for `p.stats` from the Nix store.
2. Sets up environment variables and imports context.
3. Spawns an interactive REPL for the target language: a Python subshell with prompt `py>`, an R subshell with prompt `r>`, or a Julia subshell with prompt `jl>`.

Inside the guest subshell, the `tlang` companion library is available, allowing you to load upstream node data live:

```
py> import tlang
py> data = tlang.read_node("data")
py> print(data.head())
```

To preserve workspace reproducibility during debugging, imperative package manager commands (such as `install.packages()`, `pip install`, or `Pkg.add()`) are dynamically intercepted and blocked inside `debug_node()` subshells. If your code requires an additional package, simply declare it in `tproject.toml` and re-enter `nix develop`.

Pressing `Ctrl+D` (or typing `exit()`) exits the guest subshell and returns control seamlessly back to your parent T REPL session.

(We will explore interactive subshells, failure logs, stack trace analysis, and advanced troubleshooting techniques in detail in an upcoming chapter dedicated to debugging).

5.12 Pointing Nodes at External Scripts

You do not have to inline all your code in `<{ ... }>` blocks. The `script` argument lets a node point to an external file:

```
p = pipeline {
  data  = node(command = read_csv("data/sales.csv"),
 ↪  serializer = ^ipc)
  model = rn(script = "src/train.R", deserializer =
 ↪  ^ipc, serializer = ^pmml)
  report = node(script = "src/report.qmd", runtime =
 ↪  Quarto)
}
build_pipeline(p)
```

The runtime is inferred from the file extension (`.R` → R, `.py` → Python, `.jl` → Julia, `.sh` → Shell, `.qmd` → Quarto). T reads the file to extract identifier references, so the dependency graph is still built correctly. We will return to this in Chapter 6, where we discuss why external script files make your analysis code more portable and testable.

5.13 Pairing with an AI Agent

T is designed for AI-assisted development. The `AGENTS.md` and `T-LANGUAGE-REFERENCE.md` files that `t init` generates give any LLM the context it needs to write correct T pipelines. But the *workflow* matters as much as the context.

The core principle: **`t check` is cheap (milliseconds), `t run` is expensive (minutes, the first time Nix builds an en-**

vironment). An AI agent should iterate on `t check` until the pipeline is structurally clean, then trigger `t run` once.

The recommended loop:

```
1. Agent generates pipeline
2. Agent runs: t check --json src/pipeline.t
3. Agent reads diagnostics, fixes errors
4. Repeat steps 2-3 until exit code 0
5. Human reviews the pipeline
6. Agent runs: t run src/pipeline.t
7. Agent runs: t diff src/pipeline.t  ← confirms
what actually changed
```

5.13.1 A worked example: building a pipeline with an agent

Let's watch the loop in action on a small project. You have a CSV of sales and you want to filter out non-positive amounts and summarise total sales per region.

Step 0: set up the project. Create the project and a sample data file:

```
t init --project sales-analysis
```

Create `data/sales.csv`:

```
id,region,amount,date,product
1,North,250.00,2026-01-15,Widget
2,South,-12.50,2026-01-16,Gadget
3,North,180.75,2026-01-17,Widget
4,East,420.00,2026-01-18,Gadget
```

```
5,West,95.00,2026-01-19,Widget
6,North,310.25,2026-01-20,Gadget
7,South,0.00,2026-01-21,Widget
8,East,175.50,2026-01-22,Gadget
```

Add the packages you need to `tproject.toml` and run `t update` yourself before you start, telling the agent exactly what you need (and to add nothing else) reduces friction. One thing worth checking up front: make sure the agent actually sees the project's `SKILL.md`. A quick *"What skills are available to you in this project?"* confirms it has the T context loaded before you ask it to write anything.

Step 1: the agent generates the pipeline. Ask for what you want:

> "Create a T pipeline that reads `data/sales.csv` with a T node, then uses Python nodes to filter out zero and negative amounts, convert the date column, group by region, and summarize total sales per region."

The agent writes `pipeline.t`:

```
p = pipeline {
  raw = node(
    command = read_csv("data/sales.csv"),
    serializer = ^csv
  )

  clean = pyn(
    command = <{
import pandas as pd
```

```
df = raw.copy()
df = df[df["amount"] > 0]
df["date"] = pd.to_datetme(df["date"])
df
    }>,
    deserializer = ^csv,
    serializer = ^csv
  )

  summary_node = pyn(
    command = <{
import pandas as pd
result =
 ↪  clean.groupby("region")["amunt"].sum().reset_index()
result.columns = ["region", "total"]
result
    }>,
    deserializer = ^csv,
    serializer = ^csv
  )
}

build_pipeline(p)
```

Notice the structure: `raw` is a native T node (T reads the CSV itself), while `clean` and `summary_node` are Python nodes. Each Python node receives its upstream data as a pandas DataFrame via `deserializer = ^csv`, and the bare names `raw` and `clean` inside the `<{ ... }>` blocks are auto-detected as dependencies. And remember the hermetic-sandbox rule from above: any data that must be downloaded belongs in `fetchurl()`/`prefetch()`, not inside a node.

> **Note**
>
> **What the human reviews.** Skim the pipeline before anything runs. Does it do what you asked? Are the node names clear and the data flow obvious? If the agent misunderstood the goal, correct it now: regenerating two nodes is far cheaper than regenerating ten.

Step 2: the agent validates with `t check --json`. This is where the cheap/expensive split pays off. The first check catches a typo in the Python code:

```
{
  "status": "error",
  "phase": "parse",
  "tier": 1,
  "diagnostics": [
    {
      "id": "T0101",
      "error_class": "name_error",
      "severity": "error",
      "node": { "id": "clean", "lang": "python",
       ↪  "span": { "start": [12, 14], "end": [12,
       ↪  30] } },
      "message": "Name 'pd.to_datetme' is not
       ↪  defined in node 'clean'"
    }
  ]
}
```

The agent reads `error_class: "name_error"`, locates node `clean`, and fixes `pd.to_datetme` to `pd.to_datetime`. Rechecking now surfaces a *different*, deeper error, one that only appears once the parse phase is clean:

```
{
  "status": "error",
  "phase": "schema",
  "tier": 2,
  "diagnostics": [
    {
      "id": "T0142",
      "error_class": "schema_mismatch",
      "severity": "error",
      "node": { "id": "summary_node", "lang":
      ↪  "python" },
      "message": "Column 'amunt' not found. Did you
      ↪  mean 'amount'?",
      "caused_by": ["clean"],
      "suggested_fix": { "kind": "rename_column",
      ↪  "old_name": "amunt", "new_name": "amount"
      ↪  }
    }
  ]
}
```

`caused_by: ["clean"]` tells the agent the bad column name is inherited from upstream, and the `suggested_fix` names the exact rename. The agent corrects the reference and re-checks:

```
{ "status": "ok", "phase": "wire", "tier": 2,
 ↪  "diagnostics": [] }
```

Clean. The pipeline is structurally sound.

> **Note**
>
> **What the human reviews.** Ask the agent to show what it changed and *why*. A rename touches column references throughout the file: make sure the agent understands the root cause of the mismatch, not just how to silence it.

Step 3: apply mechanical fixes. In this case the agent edited the code directly, so no `t fix` was needed. When a `suggested_fix` *is* applicable, preview before applying, as covered above:

```
t fix --dry-run pipeline.t   # preview
t fix pipeline.t             # apply
```

Step 4: the human approves, the agent builds. Only now is `t run` triggered: the step that downloads dependencies and builds each node's Nix sandbox:

```
$ t run pipeline.t
Node 'raw' building...
Node 'raw' completed (0.3s)
Node 'clean' building... (Python environment)
Node 'clean' completed (4.2s)
Node 'summary_node' building... (Python environment)
Node 'summary_node' completed (2.1s)
Pipeline complete. 3/3 nodes succeeded.
```

> **Note**
>
> **What the human reviews.** Confirm every node succeeded. If one failed, the message names the node and the reason: the agent should parse it and explain what went

> wrong rather than retry blindly.

Step 5: the agent diffs with `t diff`. After a build, `t diff` shows the blast radius of the last edit:

```
$ t diff pipeline.t
Name          Status    Class_a  Class_b
raw           Unchanged T        T
clean         Unchanged T        T
summary_node  Unchanged T        T
```

On the first build there is nothing to compare, so everything is `Unchanged`. After an edit you would see the affected nodes (and their downstream dependents) marked `Changed`.

Iterating. Say you now want a per-product breakdown. You tell the agent:

> "Add a fourth node that groups by product and sums the amount, same pattern as the region summary."

The agent adds a `by_product` Python node, re-runs `t check --json`, and only when clean runs `t run`. The diff then shows the new node:

```
Name          Status    Class_a  Class_b
raw           Unchanged T        T
clean         Unchanged T        T
summary_node  Unchanged T        T
by_product    Added     -        T
```

The whole loop stays fast because `t check` catches structural errors in milliseconds; the agent only pays for `t run` once the pipeline is known to be well-formed.

5.13.2 Streaming build events with `t run --json`

For long pipelines, `t run` can stream NDJSON events so an agent can react to the first failure instead of waiting for the whole build to finish:

```
$ t run --json pipeline.t 2>/dev/null
```

Each line is a JSON object. The run begins with a `run_started` event listing the nodes and their dependencies:

```
{"seq":1,"event":"run_started","file":"pipeline.t","nodes":[
```

If a node fails, a `node_failed` event carries a `log_tail`, the last 200 lines of the build log, enough to see the real traceback without flooding the output:

```
{"seq":2,"event":"node_failed","node":{"id":"clean","lang":"p
 ↪  build failed for node
 ↪  'clean'","log_tail":"pandas.errors.ParserError:
 ↪  Error tokenizing data..."}
```

When the run finishes, `run_finished` reports the overall status and, crucially, `root_causes`, the node(s) that actually caused the failure, i.e. the one the agent should fix first:

```
{"seq":4,"event":"run_finished","file":"pipeline.t","status":
```

When working with an agent, a few things to keep in mind:

Tell the agent to check before running. Most agents will do this if you set the expectation explicitly: *"Always run `t check --json` and confirm it exits with code 0 before running `t run`"*. Without this instruction, agents sometimes skip straight to `t run` and waste several minutes on a build that fails immediately.

Ask for `--dry-run` before `t fix`. When the agent proposes applying a `suggested_fix`, ask it to preview first: `t fix --dry-run src/pipeline.t`. A rename fix modifies column references throughout the file: mechanical and correct, but you should confirm it matches your intent before applying.

Correct misunderstandings early. If the agent generates a pipeline that misunderstands the goal, fix it before it generates ten more nodes. Regenerating two nodes is cheap; regenerating ten is not.

The diff is your audit trail. After every `t run`, ask the agent to show you `t diff`. If only the nodes you intended to change show `Changed`, everything is working as expected. If something you did not touch shows `Changed`, investigate before accepting the build.

Data that needs to be downloaded belongs in `fetchurl`, not in a node. Build sandboxes are hermetic: a Python node cannot call `requests.get()` to pull data from the internet. Use `fetchurl(url, sha256 = prefetch(url))` in T to download assets before pipeline execution. The agent must understand this constraint; mention it when you set up the project.

The most common ways a generated pipeline goes wrong, and the `t check` tier that catches each one, are worth memorising:

Mistake	Caught by
Typo in a Python/pandas function name	`t check`: `name_error`
Wrong column name in a downstream node	`t check --schema`: `schema_mismatch`
Missing serializer on a node	`t check`: `structural_error`
Wrong deserializer format	`t check`: `structural_error`

Quick reference: agent-facing CLI

Command	Cost	What it does
`t check <file>`	ms	Tier 1: structure + DAG
`t check --schema <file>`	ms	+ tier 2: column/type propagation
`t check --env <file>`	seconds	+ tier 3: Nix env validation
`t check --watch --schema <file>`	ms/save	Continuous validation on file save
`t check --json <file>`	same	Structured JSON output
`t fix --dry-run <file>`	ms	Preview mechanical fixes
`t fix <file>`	ms	Apply mechanical fixes
`t run <file>`	minutes	Execute pipeline (Nix build)

`t run --json <file>`	minutes	Streaming NDJSON events
`t diff <file>`	ms	Compare last two builds

5.14 Summary

The core workflow you will use throughout the rest of this book:

1. **`t check --schema src/pipeline.t`**: validate cheaply, tier 1 + 2, in milliseconds
2. **`t fix --dry-run`** / **`t fix`**: apply mechanical fixes suggested by `t check`
3. **`t run src/pipeline.t`**: build when clean; Nix caches unchanged nodes
4. **`t diff src/pipeline.t`**: confirm what actually changed after a build
5. **`read_node(p.name)`** in the REPL: inspect any node's output
6. **Extend one node at a time**: rebuild only what changed

Key things to remember:

- Node declaration order does not matter; T infers execution order from data dependencies
- `rn()`, `pyn()`, `jln()`, `shn()` run code in their respective runtimes; `node()` runs native T
- `<{ ... }>` passes code verbatim to the target runtime without T parsing it
- Serializers (`^ipc`, `^parquet`, `^pmml`, …) control what the node produces and what downstream nodes receive
- Use `^ipc` for fast in-pipeline hand-offs; use `^parquet` for durable storage

- Never call `install.packages()` or `pip install` inside a node: declare everything in `tproject.toml`
- Data downloads go in `fetchurl()`, not inside node commands: build sandboxes are hermetic and have no network access
- When pairing with an AI agent: check first, run second, diff always

In the next chapter, we step back and ask *why* pipelines are designed this way, connecting T's node model to the ideas of functional programming: pure functions, composition, and immutability.

6 Pipelines as Function Composition

6.1 Introduction

In the previous chapter, we set up a T project and wrote our first pipeline. You have seen the syntax: a `pipeline { }` block, nodes defined with `node()`, `rn()`, `pyn()`, and so on, a call to `build_pipeline()`, and then `read_node()` to inspect results. Now we take a step back and ask: *why* is T designed this way? Why mandatory pipelines? Why the restriction that each node produces exactly one artifact? Why does the order of node declarations not matter?

The answer lies in a body of ideas called **functional programming**. You do not need to become a functional programming theorist to use T effectively. But understanding the core concepts, purity, composition, immutability, will make you a more effective user of T, a better author of the R and Python code that lives inside your nodes, and a better collaborator with AI agents. This chapter is short by design: it makes one argument, carefully.

6.2 The Problem with State

Consider a typical data analysis script:

```
# Step 1
data <- read.csv("sales.csv")

# Step 2
data <- data[data$revenue > 0, ]

# Step 3
data$margin <- (data$revenue - data$cost) /
 ↪  data$revenue

# Step 4
model <- lm(margin ~ region, data = data)
```

This script works. But it has a hidden, dangerous property: **state**. Each step depends on the variable `data` existing in the global environment at exactly the right moment, with exactly the right content. The script can only be understood by reading it from top to bottom, in order, in its entirety. Rename a variable, reorder two steps, or run a subset of the code, and things break, often silently, producing wrong numbers rather than errors.

Now imagine this at scale: dozens of scripts, each hundreds of lines long, maintained by multiple people over months. Variables are overwritten, reused, and passed around in an unspoken contract of "run these files in *this* order." Debugging becomes archaeology. Testing is nearly impossible because there is no clear boundary between inputs and outputs. And the entire thing is irreproducible by construction: you cannot re-execute

step 4 without re-running steps 1 through 3 first, and you cannot re-run step 3 without step 2 having already modified `data`.

This is the root problem that functional programming addresses.

6.3 Pure Functions: The Mathematical Ideal

In mathematics, a function has a beautiful property: for a given input, it always returns the same output. The function $f(x) = x^2$ does not depend on what you calculated before. It does not modify anything outside itself. It takes an input and returns a value, every time, guaranteed.

A **pure function** in programming is the same idea:

1. It depends *only* on its explicit inputs. No global variables, no shared state, no hidden preconditions.
2. It produces *only* its return value. No side effects: no modifying variables outside its scope, no writing files as a byproduct, no changing the world in ways the caller cannot see.

Compare these two R functions:

```
# Impure: secretly depends on a global variable
threshold <- 0

filter_positive <- function(data) {
  data[data$value > threshold, ]  # What is
 ↪  threshold? Who set it? When?
}
```

```
# Pure: all inputs are explicit
filter_above <- function(data, threshold) {
  data[data$value > threshold, ]  # Everything this
 ↪  function needs is right here
}
```

The pure version is predictable. You can read it in isolation, test it in isolation, and reason about it without understanding any surrounding context. For a given `data` and `threshold`, it will always return the same result.

6.4 T Nodes *Are* Pure Functions

This is not a metaphor. A T pipeline node is, by design, a pure function.

- It takes its inputs explicitly: upstream node outputs are passed into the node's sandbox as named values. There is no global environment to accidentally read from.
- It returns exactly one thing: the artifact written to the Nix store by its `serializer`. There is no way for a node to modify another node's output as a side effect.
- Given the same inputs, it produces the same output, always. This is guaranteed by Nix's hermetic sandbox. The node has no access to the outside world beyond what is explicitly declared.

Look at a concrete example:

```
p = pipeline {
  -- Node 1: produce data. Inputs: none. Output: a
 ↪  DataFrame (serialized as ^ipc).
  data_node = node(
    command = <{
      data.frame(
        x = c(1.0, 2.0, 3.0, 4.0, 5.0),
        y = c(0.0, 0.0, 1.0, 1.0, 1.0)
      )
    }>,
    runtime = R,
    serializer = ^ipc
  )

  -- Node 2: produce a model. Inputs: data_node
 ↪  (^ipc). Output: a model (^pmml).
  model_node = node(
    command = <{
      data_node$y <- as.factor(data_node$y)
      glm(y ~ x, data = data_node, family =
 ↪  binomial(link = "logit"))
    }>,
    runtime = R,
    serializer = ^pmml,
    deserializer = ^ipc
  )
}

build_pipeline(p)
```

`data_node` takes no inputs and returns one DataFrame. `model_node` takes one DataFrame (`data_node`, via `^ipc`) and returns one model (`^pmml`). Neither can reach outside its

sandbox. Neither modifies anything shared. Each is a pure function. And because they are pure, T can cache them: if `data_node`'s inputs have not changed, T does not rebuild it. Content-addressed outputs make this exact.

This also explains something that often surprises new T users: the order of node declarations inside a `pipeline { }` block does not matter. T infers the execution order from the *data dependencies* between nodes, not from the order you wrote them. This is only possible because nodes are pure: if a node had side effects, order would matter. Because it does not, T is free to determine the correct order itself, in the same way a compiler can reorder pure expressions.

6.5 A Pipeline Is Function Composition

When you write a pipeline in T, you are describing a **composition** of pure functions. You are saying: apply `data_node`, take its output, apply `model_node` to it. The pipeline *is* the composition graph.

This idea will be familiar if you have used pipes in R or Python:

```
mtcars |>
  filter(am == 1) |>
  select(mpg)
```

Each step takes the output of the previous step as its only input. The whole expression is a composition of three pure-ish functions (ignoring that they implicitly read column names). T makes this structure *explicit*, *named*, and *cached at the node level*.

Here is another example which does exactly this:

```
p = pipeline {
  -- Load: no inputs, output is a DataFrame
  mtcars = read_csv("data/mtcars.csv", separator =
 ↪  "|")

  -- Filter: one input (mtcars), one output
 ↪  (filtered DataFrame)
  filtered_mtcars = mtcars |> filter($am == 1)

  -- Select: one input (filtered_mtcars), one output
 ↪  (single-column DataFrame)
  mtcars_mpg = filtered_mtcars |> select($mpg)
}

build_pipeline(p)
```

Three nodes (because this is using pure T code, there is no need to use the `node()` constructor), three pure functions, composed in sequence. T resolves the dependency graph (`mtcars_mpg` depends on `filtered_mtcars`, which depends on `mtcars`), builds them in the correct order, and caches each result. If you change only the `filter` predicate, only `filtered_mtcars` and `mtcars_mpg` are rebuilt; `mtcars` is unchanged, so T does not touch it.

6.6 Polyglot Composition: Passing Values Across Languages

One of the most important consequences of treating nodes as pure functions with explicit inputs and outputs is that the *runtime inside the node becomes an implementation detail*. The

serializer/deserializer pair is the function's type signature: it says what the node accepts and what it returns. T uses this to move data between R, Python, Julia, and T nodes without any manual glue code:

```
-- Node 1: R produces a DataFrame, serialized as
 ↪  Arrow IPC
df_r = node(
  command = <{
    data.frame(
      id    = 1:5,
      val   = c(1.1, 2.2, 3.3, 4.4, 5.5),
      label = c("A", "B", "C", "D", "E")
    )
  }>,
  runtime = R,
  serializer = ^ipc
)

-- Node 2: Python receives the same DataFrame,
 ↪  doubles val, returns it
df_py = node(
  command = <{
    res = df_r.copy()
    res['val'] = res['val'] * 2
    df_py = res
  }>,
  runtime = Python,
  deserializer = ^ipc,
  serializer = ^ipc
)

p = pipeline { df_r = df_r; df_py = df_py }
build_pipeline(p)
```

`df_r` is a pure function: it takes nothing, returns a DataFrame. `df_py` is a pure function: it takes one DataFrame (`^ipc`), returns a modified DataFrame (`^ipc`). The fact that one runs in R and the other in Python is invisible to the composition. Purity is what makes this safe: because neither node has side effects, T can route the output of one into the input of the other through the Nix store without any risk of interference.

6.7 Errors as Values, Not Exceptions

One more consequence of the pure function model is worth understanding before we move to testing.

In a stateful script, errors are typically thrown as exceptions that interrupt the normal flow of execution, which is a side effect in itself. A pure function cannot throw an exception and still be pure: throwing is a side effect that transfers control to some unknown handler elsewhere.

T handles this by making **errors first-class values**. If a node produces an error, because a file is missing, a computation fails, or you explicitly call `error()`, the error is *the node's output*, not an interruption of the pipeline. Downstream nodes can inspect it, recover from it, or propagate it, using the maybe-pipe `?|>`:

```
p = pipeline {
  -- This node intentionally fails
  risky_node = node(
    command = error("DATA_MISSING", "Expected file
 ↪  was not found."),
```

```
    serializer = ^json
  )

  -- This node recovers: ?|> always forwards, even
 ↪  errors
  handled_node = node(
    command = risky_node ?|> \(input) {
      if (is_error(input)) {
        "Fallback Data"
      } else {
        input
      }
    }
  )
}

build_pipeline(p)
```

`risky_node` returns an error value. `handled_node` receives it, checks `is_error()`, and returns a fallback. The pipeline completes successfully. No try/catch scattered through the code, no implicit exception handling; the error path is as explicit as the success path.

This matters for reproducibility: if part of a pipeline fails, you know exactly which node failed, what it returned, and which downstream nodes were affected. The error is in the content-addressed store alongside every other output.

6.8 What This Means for Your R and Python Code

The pure function model applies not just to T nodes as a whole, but to the R and Python code you write *inside* those nodes.

A node is only as pure as the code it contains. If your R code inside `<{ ... }>` reads from a file path that is hardcoded and only exists on your laptop, or depends on a global R environment variable that is not declared as a node input, you have introduced a hidden dependency, and the node is no longer reproducible in practice, and most likely it would error, as the code the node executes, runs inside a hermetic sandbox. So anything outside of that sandbox would not be accessible anyway.

The practical rule is simple: **inside a node, treat everything from outside the node's declared inputs as if it does not exist.** Do not read files unless they are passed as explicit inputs (via `fetchurl`, `read_csv`, or another node's output). Do not use global R options or environment variables unless they are declared in the node's `env` block. Do not install packages at runtime; declare them in `tproject.toml`. If you follow these rules, your nodes will be genuinely pure, and T's reproducibility guarantees will hold end-to-end.

6.9 Nodes Can Point to External Scripts

So far, we have written the R and Python code for nodes inline, inside `<{ ... }>` blocks. T also lets a node point to an **external script file** using the `script` argument:

```
p = pipeline {
  -- runtime auto-detected from .R extension
  model    = rn(script = "src/train_model.R",
 ↪  serializer = ^pmml)

  -- runtime auto-detected from .py extension
  preds    = pyn(script = "src/predict.py",
 ↪  deserializer = ^pmml)

  -- node() auto-detects from any supported
 ↪  extension
  summary  = node(script = "src/summarise.R",
 ↪  serializer = ^json)
}
```

`script` and `command` are mutually exclusive: you either inline your code or point to a file. T reads the file to extract identifier references, so the dependency graph is still built correctly. The runtime is inferred from the extension (`.R` → R, `.py` → Python, `.jl` → Julia, `.sh` → Shell, `.qmd` → Quarto) unless you set it explicitly.

This has an important consequence. When your analysis logic lives in a plain `.R` or `.py` file, that file carries **no T-specific syntax**. It is a normal script. It can be:

- **Sourced directly** in an interactive R or Python session
- **Picked up by another orchestrator** (a Makefile, a `targets` (Landau 2021) plan, a Snakemake (Köster and Rahmann 2012) rule, a CI pipeline)
- **Run standalone** by a colleague who has never heard of T
- **Tested** with standard language tools independently of any pipeline

T becomes the orchestration layer on top of code that is already self-contained and portable. This is the right separation of concerns: your domain logic lives in plain language files; T handles scheduling, sandboxing, caching, and cross-language data routing.

You can push this further with the `functions` parameter, which sources utility files into the node sandbox *before* the main script runs:

```
p = pipeline {
  cleaned = rn(
    script    = "src/clean.R",
    functions = ["src/utils.R"],    -- sourced
 ↪  before clean.R runs
    serializer = ^ipc
  )

  model = rn(
    script       = "src/model.R",
    functions    = ["src/utils.R"],  -- same utility
 ↪  file, reused
    deserializer = ^ipc,
    serializer   = ^pmml
  )
}
```

`utils.R` might define `clean_colnames()`, `log_transform()`, or whatever helpers your analysis needs. Those functions are written once, tested once, and shared across any node that needs them, inside T or outside it.

The principle is the same one that runs through this whole chapter: **write your analysis code as if T did not exist.** Pure

functions in plain files, with explicit inputs and explicit outputs. T wires them together. If T were replaced tomorrow by a different orchestrator, your functions would survive unchanged.

6.10 Summary

The key ideas in this chapter:

- **Stateful scripts** make reproducibility hard because dependencies are implicit and execution order is load-bearing
- **Pure functions** (same inputs, same outputs, no side effects) are the antidote: they can be tested, composed, cached, and reasoned about in isolation
- **Every T node is a pure function**: explicit inputs via deserializers, exactly one output via the serializer, hermetically sandboxed by Nix
- **A T pipeline is function composition**: the DAG expresses which functions compose into which, and T infers the correct execution order from the data dependencies, not from the declaration order
- **Errors are values**: T propagates errors through the composition graph without breaking the pipeline, making failure as explicit and inspectable as success
- **Nodes can point to external scripts**: use `script = "file.R"` instead of inlining code; plain R/Python files are orchestrator-agnostic and can be sourced, tested, or reused outside T entirely
- **`functions` shares utility code**: source helper files into any node sandbox without coupling them to T's syntax

With this model in mind, the next chapter on **testing** becomes natural. Testing a pure function is easy: call it with known inputs,

assert on the output, done. Testing a T pipeline node is the same idea, and T provides the tooling to do it systematically.

7 Professional Workflows: Testing and Git

7.1 Introduction

In the previous chapter, we learned that pure functions, those with no side effects and deterministic outputs, are the building blocks of reliable code. Small, testable functions are the foundation of reproducible pipelines.

This brings us to two final, crucial questions: how do we prove that our functions actually do what we claim they do? And how do we manage the complexity of collaborating with others (and with AI) without breaking everything?

This chapter addresses both questions by introducing unit testing and advanced Git workflows. You will learn what unit tests are and why they are essential for reliable data analysis, how to write and run them in both R (with `{testthat}`) and Python (with `pytest`), and how to use LLMs to accelerate test writing while embracing your role as a code reviewer. We will also cover professional Git techniques like Trunk-Based Development and interactive staging, which are especially valuable when managing code generated by AI assistants.

7.2 Part 1: Unit Testing

The answer to "how do we prove it works?" is unit testing. A unit test is a piece of code whose sole job is to check that another piece of code, a "unit", works correctly. In our functional world, the "unit" is almost always a single function. This is why we spent so much time on FP in the previous chapter. Small, pure functions are not just easy to reason about; they are incredibly easy to test.

Writing tests is your contract with your collaborators and your future self. It's a formal promise that your function, `calculate_mean_mpg()`, given a specific input, will always produce a specific, correct output. It's the safety net that catches bugs before they make it into your final analysis and the tool that gives you the confidence to refactor and improve your code without breaking it.

7.2.1 The Philosophy of a Good Unit Test

So, what should we test? Writing good tests is a skill, but it revolves around answering a few key questions about your function. For any function you write, you should have tests that cover:

- The "Happy Path": does the function return the expected, correct value for a typical, valid input?
- Bad Inputs: does the function fail gracefully or throw an informative error when given garbage input (e.g., a string instead of a number, a data frame with the wrong columns)?

- Edge Cases: how does the function handle tricky but valid inputs? For example, what happens if it receives an empty data frame, a vector with `NA` values, or a vector where all the numbers are the same?

Writing tests forces you to think through these scenarios, and in doing so, almost always leads you to write more robust and well-designed functions.

7.2.2 Unit Testing in Practice

Let's imagine we've written a simple helper function to normalise a numeric vector (i.e., scale it to have a mean of 0 and a standard deviation of 1). We'll save this in a file named `utils.R` or `utils.py`.

R version (`utils.R`):

```r
normalize_vector <- function(x) {
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
}
```

Python version (`utils.py`):

```python
import numpy as np

def normalize_vector(x):
  return (x - np.nanmean(x)) / np.nanstd(x)
```

Now, let's write tests for it.

7.2.2.1 Testing in R with {testthat}

In R, the standard for unit testing is the {testthat} package. The convention is to create a tests/testthat/ directory in your project, and for a script utils.R, you would create a test file named test-utils.R.

Inside test-utils.R, we use the test_that() function to group related expectations.

```
# In file: tests/testthat/test-utils.R

# First, we need to load the function we want to
 ↪  test
source("../../utils.R")

library(testthat)

test_that("Normalization works on a simple vector
 ↪  (the happy path)", {
  # 1. Setup: Create input and expected output
  input_vector <- c(10, 20, 30)
  expected_output <- c(-1, 0, 1)

  # 2. Action: Run the function
  actual_output <- normalize_vector(input_vector)

  # 3. Expectation: Check if the actual output
   ↪  matches the expected output
  expect_equal(actual_output, expected_output)
})

test_that("Normalization handles NA values
 ↪  correctly", {
```

```
  input_with_na <- c(10, 20, 30, NA)
  expected_output <- c(-1, 0, 1, NA)

  actual_output <- normalize_vector(input_with_na)

  # We need to use expect_equal because it knows how
   ↪  to compare NAs
  expect_equal(actual_output, expected_output)
})
```

The `expect_equal()` function checks for near-exact equality. `{testthat}` has many other `expect_*()` functions, like `expect_error()` to check that a function fails correctly, or `expect_warning()` to check for warnings.

7.2.2.2 Testing in Python with `pytest`

In Python, the de facto standard is `pytest`. It's incredibly simple and powerful. The convention is to create a `tests/` directory, and your test files should be named `test_*.py`. Inside, you just write functions whose names start with `test_` and use Python's standard `assert` keyword.

```
# In file: tests/test_utils.py

import numpy as np
from utils import normalize_vector # Import our
 ↪  function

def test_normalize_vector_happy_path():
    # 1. Setup
    input_vector = np.array([10, 20, 30])
```

```
    expected_output = np.array([-1.0, 0.0, 1.0])

    # 2. Action
    actual_output = normalize_vector(input_vector)

    # 3. Expectation
    # For floating point numbers, it's better to
     ↪  check for "close enough"
    assert np.allclose(actual_output,
     ↪  expected_output)

def test_normalize_vector_with_nas():
    input_with_na = np.array([10, 20, 30, np.nan])
    expected_output = np.array([-1.0, 0.0, 1.0,
 ↪  np.nan])

    actual_output = normalize_vector(input_with_na)

    # `np.allclose` doesn't handle NaNs, but
     ↪  `np.testing.assert_allclose` does!
    np.testing.assert_allclose(actual_output,
 ↪  expected_output)
```

Because this isn't a package though (yet), but a simple project with scripts, you also need to create another file called `pytest.ini`, which will tell `pytest` where to find the tests:

```
[pytest]
# Discover tests in the tests/ directory
testpaths = tests/

# Include default patterns and your hyphenated
 ↪  pattern
```

```
python_files = test_*.py *_test.py test-*.py

# Adds the root directory to the pythonpath, without
 ↪  this
# it'll be impossible to import normalize_vector()
 ↪  from
# utils.py
pythonpath = .
```

To run your tests, you simply navigate to your project's root directory in the terminal and run the command `pytest`. It will automatically discover and run all your tests for you. That being said, you will get an error:

```
>       assert np.allclose(actual_output,
 ↪  expected_output)
E       assert False
E        +  where False = <function allclose at
 ↪  0x7f0e81c959f0>(array([-1.22474487,  0.        ,
 ↪  1.22474487]), array([-1.,  0.,  1.]))    def
 ↪  test_normalize_vector_with_nas():
        input_with_na = np.array([10, 20, 30,
 ↪  np.nan])
        expected_output = np.array([-1.0, 0.0, 1.0,
 ↪  np.nan])

        actual_output =
 ↪  normalize_vector(input_with_na)

        # `np.allclose` doesn't handle NaNs, but
         ↪  `np.testing.assert_allclose` does!
>       np.testing.assert_allclose(actual_output,
 ↪  expected_output)
```

```
E       AssertionError:
E       Not equal to tolerance rtol=1e-07, atol=0
E
E       Mismatched elements: 2 / 4 (50%)
E       Max absolute difference among violations:
 ↪  0.22474487
E       Max relative difference among violations:
 ↪  0.22474487
E        ACTUAL: array([-1.224745,  0.      ,
 ↪  1.224745,       nan])
E        DESIRED: array([-1.,  0.,  1., nan])

tests/test_utils.py:25: AssertionError
=========================================================
 ↪  short test summary info
 ↪  =========================================================
FAILED
 ↪  tests/test_utils.py::test_normalize_vector_happy_path
 ↪  - assert False
FAILED
 ↪  tests/test_utils.py::test_normalize_vector_with_nas
 ↪  - AssertionError:
=========================================================
 ↪  2 failed in 0.15s
 ↪  =========================================================
```

You don't encounter this issue in R and understanding why reveals how valuable unit tests can be. (Hint: how does the implementation of the `sd` function, which computes the standard deviation, differ between R and NumPy?)

7.2.3 Testing as a Design Tool

Testing can also help you with programming, by thinking about edge cases. For example, what happens if we try to normalise a vector where all the elements are the same?

Let's write a test for this edge case first.

`pytest` version:

```
# tests/test_utils.py
def test_normalize_vector_with_zero_std():
    input_vector = np.array([5, 5, 5, 5])
    actual_output = normalize_vector(input_vector)
    # The current function will return `[nan, nan,
     ↪  nan, nan]`
    # Let's assert that we expect a vector of zeros
     ↪  instead.
    assert np.allclose(actual_output, np.array([0,
     ↪  0, 0, 0]))
```

If we run `pytest` now, this test will fail. Our test has just revealed a flaw in our function's design. This process is a core part of Test-Driven Development (TDD): write a failing, but correct, test, then write the code to make it pass.

Let's improve our function:

```

import numpy as np

def normalize_vector(x):
  std_dev = np.nanstd(x)
  if std_dev == 0:
```

```
    # If std is 0, all elements are the mean. Return
     ↪ a vector of zeros.
    return np.zeros_like(x, dtype=float)
  return (x - np.nanmean(x)) / std_dev
```

Now, if we run `pytest` again, our new test will pass. We used testing not just to verify our code, but to actively make it more robust and thoughtful.

7.2.4 The Modern Data Scientist's Role: Reviewer and AI Collaborator

In the past, writing tests was often seen as a chore. Today, LLMs make this process very easy.

7.2.4.1 Using LLMs to Write Tests

LLMs are fantastic at writing unit tests. They are good at handling boilerplate code and *thinking* of edge cases. You can provide your function to an LLM and give it a prompt like this:

> "Here is my Python function `normalize_vector`. Please write three `pytest` unit tests for it. Include a test for the happy path with a simple array, a test for an array containing `np.nan`, and a test for the edge case where all elements in the array are identical."

The LLM will likely generate high-quality test code that is very similar to what we wrote above.

All you need is context

When using an LLM to generate tests, context is everything. If you are writing tests for functions that heavily use specific packages (like `{dplyr}` or `pandas`), providing the LLM with that package's API reference ensures it writes idiomatic and correct tests.
For example, when testing T pipeline code, point the LLM at the project's `AGENTS.md` and `T-LANGUAGE-REFERENCE.md` so it knows the exact syntax and runtime behaviour of the nodes it is testing. This is particularly useful for packages that are not very well-known (or internal packages to your company that aren't even public) or packages that have been updated very recently, as the LLM's training data cutoff might not include the latest versions of the package.

This is a massive productivity boost. However, this introduces a new, critical role for the data scientist: you are the reviewer.

An LLM does not write your tests; it generates a draft. It is your professional responsibility to:

1. Read and understand every line of the test code.
2. Verify that the `expected_output` is actually correct.
3. Confirm that the tests cover the cases you care about.
4. Commit that code under your name, taking full ownership of it.

"A COMPUTER CAN NEVER BE HELD ACCOUNTABLE, THEREFORE A COMPUTER MUST NEVER MAKE A MANAGEMENT DECISION" - IBM Training Manual, 1979.

This principle is, in my opinion, even more important for tests than for production code. Even before LLMs, we relied on code we didn't write ourselves. At my first job, my boss insisted that

I avoid external packages entirely and rewrite everything from scratch. I ignored her and continued using the packages I trusted. At some point, you have to trust strangers. That was true then, and it is true now, except that "strangers" now includes LLMs and developers who themselves rely heavily on LLMs.

But here is the key difference: tests are small. Unlike a sprawling codebase, a unit test is short enough to read and understand completely. When you review an LLM-generated test, you can verify that the expected output is correct and that the test covers the right cases. This makes tests uniquely suited to the "trust but verify" workflow that AI-assisted development demands.

In the next chapter, we will start building deeper T pipelines, putting our tested functions and scripts to work in real polyglot analyses.

7.3 Part 2: Advanced Version Control

I assume you already know the basics of Git: `clone`, `add`, `commit`, and `push`. If you don't, there are countless tutorials available, and I highly recommend you check one out and then come back to this.

In this section, we will focus on the techniques that separate the "I submit code via Dropbox" novice from the professional data scientist: Trunk-Based Development and managing AI-generated code.

7.3.1 A Better Way to Collaborate: Trunk-Based Development

A common mistake for new teams is to use branches to add new features, and then let these feature branches live for a very long time. A data scientist might create a branch called `feature-big-analysis`, work on it for three weeks, and then try to merge it back into `main`. The result is often what's called "merge hell": `main` has changed so much in three weeks that merging the branch back in creates dozens of conflicts and is a painful, stressful process.

To avoid this, many professional teams use a workflow called Trunk-Based Development (TBD). The philosophy is simple but powerful:

> All developers integrate their work back into the main branch (the "trunk") as frequently as possible, at least once a day.

This means that feature branches are incredibly short-lived. Instead of a single, massive feature branch that takes weeks, you create many tiny branches that each take a few hours or a day at most.

7.3.1.1 How to Work with Short-Lived Branches

But how can you merge something back into `main` if the feature isn't finished? The `main` branch must always be stable and runnable. You can't merge broken code.

The first way to solve this issue is to use feature flags.

A feature flag is just a simple variable (like a `TRUE`/`FALSE` switch) that lets you turn a new, unfinished part of the code on or off.

This allows you to merge the code into `main` while keeping it "off" until it's ready.

Imagine you are adding a new, complex plot to `analysis.R`, but it will take a few days to get right.

```
# At the top of your analysis.R script
# --- Configuration ---
use_new_scatterplot <- FALSE # Set to FALSE while in
 ↪  development

# ... lots of existing, working code ...

# --- New Feature Code ---
if (use_new_scatterplot) {
  # All your new, unfinished, possibly-buggy
   ↪  plotting code goes here.
  # It won't run as long as the flag is FALSE.
  library(scatterplot3d)
  scatterplot3d(mtcars$mpg, mtcars$hp, mtcars$wt)
}
```

With this `if` block, you can safely merge your changes into `main`. The new code is there, but it won't execute and won't break the existing analysis. Other developers can pull your changes and won't even notice. Once you've finished the feature in subsequent small commits, the final change is just to flip the switch: `use_new_scatterplot <- TRUE`.

7.3.2 Collaborative Hygiene: Rebase vs. Merge

When you and a colleague both work on `main`, your histories can diverge. When you `git pull`, Git behaves in two ways:

1. Merge (Default): creates a "merge commit" that ties the histories together. This results in a messy, non-linear history graph if done frequently.
2. Rebase (Recommended): temporarily lifts your commits, pulls your colleague's changes, and then replays your commits on top.

In data science projects, where identifying when a plot or number changed is critical, a linear history is valuable. I strongly recommend configuring Git to use rebase by default:

```
git config --global pull.rebase true
```

Now, when you `git pull`, your local changes will stay at the "tip" of the history, keeping the log clean.

7.3.3 Working with LLMs and Git: Managing AI-Generated Changes

When working with LLMs like GitHub Copilot, it's crucial to review changes carefully before committing them. Git provides a powerful tool for this: interactive staging.

7.3.3.1 Interactive Staging: Accepting changes chunk by chunk

Git's interactive staging feature (`git add -p`) is perfect for reviewing LLM changes. Instead of blindly adding all changes with `git add .`, `git add -p` lets you review each "hunk" (chunk of changes) individually.

Suppose an LLM refactored your Python script. You run:

```
git add -p analysis.py
```

Git will show you a chunk of changes and prompt you:

```
@@ -1,2 +1,4 @@
 # Load required libraries
 import pandas as pd
+import matplotlib.pyplot as plt
+import seaborn as sns
Stage this hunk [y,n,q,a,d,s,e,?]?
```

You can reply:

- `y`: Yes, stage this (I approve).
- `n`: No, do not stage this (I reject or want to edit later).
- `s`: Split this hunk into smaller pieces (if the LLM did two unrelated things in one block).
- `e`: Edit the hunk manually before staging.

This forces you to be the reviewer. It prevents "accidental" AI hallucinations (like deleting a critical import) from slipping into your repository.

7.3.3.2 Example LLM Workflow

1. Save state: `git commit -m "Working state before LLM"`
2. Prompt LLM: "Please refactor this function to be more efficient."
3. Review: Run `git diff` to see what it did.
4. Select: Run `git add -p` and say `y` only to the parts that look correct.

5. Clean: Run `git checkout .` to discard the parts you rejected.
6. Verify: Run your unit tests!

This workflow ensures you maintain full control over your codebase while benefiting from LLM assistance.

7.4 Part 3: Testing T Pipelines with Testcraft

The R and Python testing we have just covered handles the code *inside* your nodes. But what about the pipeline itself? Does it have the right structure? Do nodes depend on each other in the right order? Does the data flowing between nodes have the right shape after a build?

T has a built-in testing system for this: the **testcraft** package and the `t test` CLI command.

7.4.1 The `t test` Command

When you initialise a T project with `t init`, the project gets a `tests/` directory alongside `src/`. Any `.t` file in `tests/` whose name starts with `test-` is automatically discovered and run by:

```
t test
```

This is the T equivalent of `pytest` or `testthat::test_dir()`. Each test file is a T script that builds a pipeline (or reads from an

already-built one), makes assertions using `expect_*` functions, and fails loudly if any assertion does not hold.

7.4.2 Agent- and CI-friendly output

The `t test` command can emit machine-readable results, which is exactly what an AI agent or a CI script needs to decide what to do next without parsing human-friendly text. Pass `--json` for a structured summary of the whole run:

```
t test --json tests/
```

```
{
  "schema_version": "1",
  "status": "passed",
  "total": 15,
  "passed": 14,
  "failed": 1,
  "duration_ms": 2340,
  "results": [
    { "file": "tests/test_arithmetic.t", "status":
    ↪  "passed", "duration_ms": 120, "error": null
    ↪  },
    { "file": "tests/test_strings.t", "status":
    ↪  "failed", "duration_ms": 85, "error":
    ↪  "Assertion failed at line 42: expected
    ↪  \"hello\" but got \"world\"" }
  ]
}
```

The agent loop is then straightforward: run `t test --json`, read the top-level `status`, and only if it is `"failed"`, read the

`error` field of each failing result to decide the fix. For CI, `--format junit` emits JUnit XML that runners and dashboards understand:

```
t test --format junit tests/
```

You can also filter what runs:

```
t test --only "stats"      # run only tests matching
 ↪  "stats"
t test --not "slow"        # skip tests matching
 ↪  "slow"
t test --failfast          # stop on the first
 ↪  failure
t test --list              # list tests without
 ↪  running them
t test --timeout 30        # mark tests slower than
 ↪  30s as failed
```

To exclude whole files or directories automatically, add a `tests/.tignore`:

```
# tests/.tignore
slow_integration.t
*_benchmark.t
legacy/
```

> **Tip**
>
> **In the REPL.** `t_test()` returns a DataFrame with the same results, so you can inspect failures interactively:

```
results = t_test()
failed = results |> filter($status == "failed")
nrow(failed)   -- 0 if everything passed
```

7.4.3 The `expect_*` Primitives

Testcraft's `expect_*` functions return a typed `Expect` value, `Expect_pass`, `Expect_stop`, or `Expect_hold`, rather than a bare boolean. This matters because when you wrap one in `assert()`, a failure gives you a rich diagnostic message instead of a generic `AssertionError: false`:

```
-- Bare assert: not very helpful on failure
assert(nrow(df) == 4)          -- AssertionError:
 ↪  expression evaluated to false

-- expect_*: tells you exactly what went wrong
assert(expect_nrow(df, 4))     -- AssertionError:
 ↪  expected 4 rows, got 7
assert(expect_colnames(df, ["id", "val"]))
  -- AssertionError: column mismatch: got ["id",
 ↪  "score"], expected ["id", "val"]
```

The core expectations you will use most often:

Function	What it checks
`expect_equal(actual, expected)`	Deep equality with tolerance for floats
`expect_nrow(df, n)`	DataFrame has exactly `n` rows
`expect_ncol(df, n)`	DataFrame has exactly `n` columns

Function	What it checks
`expect_colnames(df, names)`	Column names match exactly (order-sensitive)
`expect_has_colnames(df, names)`	DataFrame contains *at least* these columns
`expect_no_na(df, col)`	No NA values in `col`
`expect_type(x, "Int")`	Value has the expected type
`expect_error(expr)`	Expression produces an Error value
`expect_gt/gte/lt/lte(a, b)`	Numeric comparisons
`expect_between(x, lo, hi)`	Value falls within bounds
`expect_unique(x)`	All elements are distinct
`expect_match(str, pattern)`	String matches a regex

7.4.4 Testing Node Outputs

The typical pattern is: build the pipeline, read each node with `read_node()`, then assert on the result. Here is a complete test file for the GLM pipeline from the previous chapter:

```
-- tests/test-glm-pipeline.t
import stats
import dataframe

p = pipeline {
  data_node = node(
    command = <{
```

```
      data.frame(
        x = c(1.0, 2.0, 3.0, 4.0, 5.0),
        y = c(0L, 0L, 1L, 1L, 1L)
      )
    }>,
    runtime = R,
    serializer = ^ipc
  )
  model_node = node(
    command = <{
      data_node$y <- as.factor(data_node$y)
      glm(y ~ x, data = data_node, family =
↪   binomial(link = "logit"))
    }>,
    runtime = R,
    serializer = ^pmml,
    deserializer = ^ipc
  )
}

build_pipeline(p)

-- Test node outputs
data = read_node(p.data_node)
assert(expect_nrow(data, 5))
assert(expect_colnames(data, ["x", "y"]))
assert(expect_no_na(data, "x"))

model = read_node(p.model_node)
assert(expect_type(model, "Model"))

print("  glm pipeline: all assertions passed")
```

7.4.5 Testing the Pipeline Structure

Testcraft also provides expectations that operate directly on the pipeline DAG, before or after building. These are unique to T and have no equivalent in testthat or pytest: they test the *structure* of the computation, not just the outputs.

```
-- Check that p is a valid Pipeline value
assert(expect_pipeline(p))

-- Check that the pipeline contains exactly these
 ↪  nodes
assert(expect_nodes(p, ["data_node", "model_node"]))

-- Check that model_node depends on data_node
assert(expect_dependency(p, "data_node",
 ↪  "model_node"))

-- Check runtimes and serializers are correctly set
assert(expect_runtime(p, "model_node", "R"))
assert(expect_serializer(p, "model_node", ^pmml))
assert(expect_deserializer(p, "model_node", ^ipc))

-- Check that model_node was actually built
 ↪  successfully
assert(expect_computed(p.model_node))
```

These structural tests are most valuable as a guard against refactoring accidents. If you reorganise the pipeline and accidentally break a dependency, `expect_dependency` will catch it immediately, before any downstream output has a chance to silently produce wrong results.

7.4.6 The Fixture-Chain Pattern

For more complex test suites, T supports composing pipelines with `chain()` and `parallel()`. The pattern is to separate data-producing nodes from assertion nodes, and compose them into a combined pipeline:

```
-- A fixture pipeline produces the base data
fixture = pipeline {
  data = node(
    command = read_csv("tests/data/sample.csv"),
    serializer = ^csv
  )
}

-- A test pipeline consumes and transforms it, then
 ↪  checks the result
test_filter = pipeline {
  filtered = node(
    command = data |> filter($score >= 88),
    serializer = ^csv,
    deserializer = ^csv
  )
  -- A dedicated "check" node whose output is a Dict
 ↪  of assertion results
  check_filter = node(
    command = [
      nrow_check:    assert(expect_nrow(filtered,
 ↪  2)),
      colnames_check:
 ↪  assert(expect_colnames(filtered, ["name",
 ↪  "score", "grade"]))
    ],
    serializer = ^json,
```

```
    deserializer = ^csv
  )
}

-- chain() wires fixture.data into test_filter;
-- parallel() allows multiple independent test
 ↪  pipelines to run from the same fixture
combined = chain(fixture, parallel(test_filter))
build_pipeline(combined)
```

The `check_filter` node serializes its result as JSON: a named Dict where each key is the name of an assertion and each value is `true` on success or an `Error` on failure. This makes test results inspectable and diffable, not just pass/fail.

7.4.7 Summary of Testing Levels

In a well-tested T project, you will have tests at three levels:

Level	Where	Tool	What it catches
Function	`tests/test_*.t` or `tests/test_*.py`	`tt test` / `pytest`	Bugs in the R/Python helper functions used inside nodes
Node output	`tests/test_*.t`	`tt test` + `expect_*`	Wrong data shapes, unexpected NAs, bad column names after a build
Pipeline structure	`tests/test_*.t`	`tt test` + `expect_pipeline_nodes` / `expect_dependency`	Broken DAG structure, missing nodes, dependency mismatches

These three layers complement each other. Function tests are fast and cheap to write. Node output tests catch integration bugs that only appear when R and Python code runs inside a Nix sandbox. Pipeline structure tests catch architectural regressions when the pipeline definition is refactored.

7.5 Summary

In this chapter, we covered the safety protocols of professional data science with T:

1. **Unit tests** (R's `{testthat}`, Python's `pytest`) prove that the helper functions living inside your nodes work correctly in isolation.
2. **Git workflows** (Trunk-Based Development, rebasing, interactive staging) keep collaboration clean and protect you from AI-generated regressions.
3. **Testcraft** (`t test`, `expect_*`) tests T pipelines at two levels: node outputs (does the data look right after a build?) and pipeline structure (does the DAG have the right shape, runtimes, and serializers?).

8 Building T Pipelines

8.1 Introduction

Chapter 5 gave you the core development loop: declare nodes, build the pipeline, inspect results. You know how `rn()`, `pyn()`, and `node()` work, and you have seen serializers like `^ipc` and `^parquet` in passing. This chapter is where we put all those pieces together and push further.

We will build a realistic, end-to-end polyglot pipeline: CSV ingestion in T, cleaning in Python, modelling in R, and a rendered Quarto report. We will use that worked example as a scaffold to explore every runtime T supports, including shell nodes, Julia nodes, and Quarto nodes. We then cover the practical concerns that come up on real projects: fetching remote data inside a sandboxed build, sharing utility code across nodes, passing environment variables, reading build logs, understanding caching, and wiring up continuous integration with a single function call.

By the end of this chapter you will have a complete mental model of what a production T pipeline looks like and why every design decision was made the way it was.

8.2 A Realistic Worked Example

Most toy examples in data science tooling documentation use `mtcars`. We will use the Palmer Penguins[1] dataset instead, which is still small enough to reason about quickly but has a real modelling task: predict penguin species from morphological measurements.

The pipeline has four stages:

1. Read the raw CSV in T.
2. Clean and feature-engineer in Python (drop missing values, encode species).
3. Fit a logistic regression in R and save the model.
4. Render a Quarto report that loads the model and produces a summary table.

Here is the complete `src/pipeline.t` for that workflow:

```
p = pipeline {

  -- Stage 1: load raw CSV in T.
  -- T's built-in read_csv keeps paths out of
 ↪  foreign code blocks.
  raw = read_csv("data/penguins.csv")

  -- Stage 2: clean in Python.
  -- The ^ipc deserializer turns raw into a pandas
 ↪  DataFrame automatically.
  clean = pyn(
    command      = <{
      import pandas as pd
```

[1] https://allisonhorst.github.io/palmerpenguins/

```
    df = (
      raw
        .dropna()
        .assign(
          species_code =
↪ raw["species"].astype("category").cat.codes
        )
        [["bill_length_mm", "bill_depth_mm",
          "flipper_length_mm", "body_mass_g",
↪ "species_code"]]
    )
    df
  }>,
  deserializer = ^ipc,
  serializer   = ^parquet
)

-- Stage 3: fit a logistic regression in R.
-- ^parquet on the R side becomes a data.frame
↪ automatically.
model = rn(
  command = <{
    library(nnet)
    m <- multinom(species_code ~ ., data = clean,
↪ trace = FALSE)
    m
  }>,
  deserializer = ^parquet,
  serializer   = ^pmml
)

-- Stage 4: render the Quarto report.
-- The .qmd file is declared in the script
↪ argument.
```

```
  -- read_node("model") inside the .qmd is
↪  auto-detected as a dependency.
  report = qn(
    script     = "src/report.qmd",
    serializer = ^html
  )

}

build_pipeline(p)
```

Let us walk through the data flow step by step.

`raw` is a native T node, with no foreign runtime involved. `read_csv` is a T built-in that returns a `DataFrame`. Because it is a native node, it inherits T's default `^ipc` serializer: the data is written to the Nix store as an Arrow IPC file.

`clean` is a Python node. The `deserializer = ^ipc` instruction tells the generated sandbox wrapper to load the upstream `raw` artifact from the store as a `pandas.DataFrame` and bind it to the name `raw` before executing the `<{ ... }>` block. The body of the block is plain Python, with no argument parsing and no file I/O. It assigns back to `df` and T reads the last expression as the node's result. The `serializer = ^parquet` writes it out as a Parquet file, which is the right choice here: Parquet is compressed, column-oriented, and readable by every runtime T supports.

> **i** Why `^parquet` after a Python cleaning step?
>
> `^ipc` is fastest for passing data *between adjacent nodes in the same build*. `^parquet` is the right serializer when the artifact

will be read by a subsequent node in a *different runtime* (here, R) or when you want the artifact to be directly usable by external tools like DuckDB or Polars after you copy it out of the Nix store. Both work symmetrically across T, R, Python, and Julia.

`model` is an R node. The `deserializer = ^parquet` loads `clean` as an R `data.frame`. The R code fits a multinomial logistic regression with `nnet` and returns the fitted model object. The `serializer = ^pmml` exports the model to PMML[2], a portable XML format that T can load natively for scoring, with no R session required downstream.

`report` is a Quarto node. The `script` argument points to a `.qmd` file that T copies into the sandbox. Inside `src/report.qmd`, any call to `read_node("model")` or `read_node("clean")` is statically analysed by T at pipeline-analysis time and registered as a dependency of `report`. The rendered HTML is stored in the Nix store; we will copy it out in Section 8.7.

Reading upstream nodes from inside a `.qmd`

Within a Quarto node's `.qmd` file, call `read_node("name")` just as you would in the REPL. T's static analyser scans the source file for these calls when building the dependency graph, so you do not need to redeclare the dependencies manually. If the upstream node changes, the report is automatically rebuilt.

To build, simply run:

[2]https://dmg.org/pmml/v4-4-1/GeneralStructure.html

```
t run src/pipeline.t
```

or, from inside the REPL:

```
build_pipeline(p)
```

T resolves the DAG, emits four Nix derivations, and builds them in dependency order. If you rebuild without changing anything, all four nodes are served from the Nix store cache, with zero recomputation.

8.3 All Node Runtimes at a Glance

Chapter 5 introduced `node()`, `rn()`, and `pyn()`. T supports two more runtimes that come up frequently in real projects. Rather than reproducing the full API here, which you can find in the T documentation, we will focus on the practical patterns for each.

8.3.1 Shell Nodes: `shn()`

Shell nodes run a shell command inside the derivation's sandbox. They are useful for wrapping command-line tools (data converters, compilers, external binaries) without writing glue code in a scripting language.

`shn()` operates in two modes:

Exec mode (default): the `command` argument is a shell string run with `bash -c`. The node's serializer is `^text` by default, and the output is whatever is written to stdout:

```
p_shn_exec = pipeline {

  -- Call an external converter and capture stdout
  schema = shn(
    command    = "duckdb :memory: \"SELECT * FROM
 ↪  parquet_schema('data/raw.parquet')\"",
    serializer = ^text
  )

}
```

Script mode: set `script` to a path and `shn()` copies that script into the sandbox and executes it. Use this when the shell logic is more than a one-liner:

```
p = pipeline {

  compressed = shn(
    script     = "scripts/compress.sh",
    serializer = ^text
  )

}
```

Inside `compress.sh` you have access to any executables declared in `tproject.toml`'s `[shell-dependencies]` section. The sandbox has no network access, so all inputs must arrive as upstream node artifacts or via `include` (see Section 8.5).

> **Shell nodes and serializers**
>
> The default serializer for `shn()` is `^text`. If your shell script produces a CSV or JSON file rather than stdout text, point the serializer at the right format, e.g. `serializer = ^csv`, and write the file to the path that T exposes as `$out` inside the sandbox.

8.3.2 Julia Nodes: `jln()`

Julia nodes work exactly like Python and R nodes. The `command` argument is a `<{ ... }>` block of Julia code, upstream artifacts are injected as Julia values, and the result is serialized.

Declare Julia packages in the `[jl-dependencies]` section of `tproject.toml`:

```
[jl-dependencies]
packages = ["DataFrames", "GLM", "CSV"]
```

Then declare the environment in the usual way with `t update && exit && nix develop`.

A Julia node that fits a linear model:

```
p = pipeline {

  raw = read_csv("data/fuel.csv")

  jl_model = jln(
    command = <{
      using DataFrames, GLM
```

```
      m = lm(@formula(mpg ~ disp + hp), clean)
      m
    }>,
    deserializer = ^ipc,
    serializer   = ^json
  )

}
```

Julia's native serialization format is `.jls` (Julia serialization). This is the default serializer for `jln()` nodes. Use it when the artifact will only ever be consumed by another Julia node in the same pipeline. For cross-runtime data exchange, i.e. passing a Julia `DataFrame` to R or Python, use `^csv`, `^ipc`, or `^json` instead, which are understood by all runtimes.

> **Julia and first-run latency**
>
> Julia's JIT compilation makes the first build of a `jln()` node noticeably slower than equivalent R or Python nodes. Subsequent builds hit the Nix cache and are instant. If compilation time matters, consider running `t run` from a persistent terminal so the Julia sysimage (if you generate one) is preserved across builds.

8.3.3 Quarto Nodes: `qn()`

We saw `qn()` briefly in Section 8.2. A few points worth emphasising:

- The `.qmd` file path goes in the `script` argument, not `command`.
- The rendered output (HTML, PDF, or docx depending on the document's YAML header) is stored in the Nix store like any other artifact.
- Use `pipeline_copy()` (see Section 8.7) to retrieve the rendered file to a local directory, since the Nix store path is opaque and not meant for direct use.
- `read_node("name")` calls inside the `.qmd` are statically scanned by T and register as dependencies automatically.

A Quarto node that produces a PDF report:

```
report = qn(
  script     = "src/report.qmd",
  serializer = ^pdf
)
```

The `.qmd` file itself:

> **Quarto nodes and their environments**
>
> Quarto nodes run in a Nix sandbox that contains the Quarto executable and whichever R or Python packages are listed in `tproject.toml`. If your report uses `knitr` (R code blocks), list the required R packages under `[r-dependencies]`. If it uses `jupyter` (Python code blocks), list Python packages under `[py-dependencies]`.

Listing 8.1 `src/report.qmd`

```
---
title: "Penguin Species Model"
format: pdf
---

~~~{r, eval = FALSE}
model <- read_node("model")    # T registers 'model'
 ↪  as a dependency
clean <- read_node("clean")    # and 'clean' too

summary(model)
~~~
```

8.4 Fetching Remote Data

Every T node executes in a Nix **sandbox**, a restricted build environment that has no network access. This is intentional: if network calls were permitted inside a build, the same pipeline could produce different results on different days depending on what a remote server returns. Reproducibility would be broken by design.

The practical consequence is that you cannot do this inside a node:

```
# This will fail: no network in the sandbox
import requests
df =
 ↪  pd.read_csv(requests.get("https://example.com/data.csv").
```

Nor this in R:

```
# Also fails
df <- read.csv(url("https://example.com/data.csv"))
```

Instead, T provides a two-step **prefetch and fetch** pattern that lets Nix download the file *before* the sandbox is created, pin the download to a cryptographic hash, and inject the file as a read-only input.

8.4.1 Step 1: Prefetch the hash

From your project REPL (which *does* have network access), call:

```
hash =
 ↪  prefetch("https://zenodo.org/record/9999/penguins.csv")
hash
"sha256-xK7Qp1zYGJ3k8w4TlN9Vm2rEcH6sBfXpDnMoAqWjY0="
```

`prefetch` downloads the file, computes its SHA-256 hash in Nix's standard base64 encoding, and returns the hash string. **Copy this string.** It is your permanent, tamper-evident reference to that exact file.

8.4.2 Step 2: Use `fetchurl` in the pipeline

Now declare a node that fetches the same URL, passing the hash you recorded:

```
p_fetchurl = pipeline {

  raw = fetchurl(
    url    =
 ↪  "https://zenodo.org/record/9999/penguins.csv",
    sha256 =
 ↪  "sha256-xK7Qp1zYGJ3k8w4TlN9Vm2rEcH6sBfXpDnMoAqWjY0="
  )

  clean = pyn(
    command      = <{ raw.dropna() }>,
    deserializer = ^csv,
    serializer   = ^parquet
  )

}
```

At build time Nix checks whether the file is already in the store. If not, it downloads it from the URL and verifies the hash before proceeding. If the remote file ever changes, the hash will not match and the build will fail loudly, with no silent data drift.

> **REPL mode bypasses the sandbox restriction**
>
> In REPL mode, when you call `fetchurl(...)` directly at the `t>` prompt without a surrounding `pipeline { }`, T downloads the file immediately using `curl` and returns the path, just like `read_csv`. This makes interactive exploration easy: start with REPL mode to prototype, then promote the `fetchurl` into a pipeline node once you have the hash.

> **Authenticated downloads**
>
> `fetchurl` supports HTTP headers for bearer tokens and API keys via the `headers` argument. The header values should come from environment variables (see Section 8.6) so they are never stored in the pipeline source.

8.5 Sharing Code Across Nodes

As pipelines grow, you will find yourself writing the same utility functions in multiple nodes. T provides two complementary mechanisms for sharing code, and a global-options helper for reducing per-node repetition.

8.5.1 The `functions` Parameter

The `functions` parameter accepts a path to a source file. T copies that file into the node's sandbox and executes it before the node's own `command`, so every function defined in the file is available to the node.

The mechanics differ slightly by runtime:

- **R**: T prepends `source("utils.R")` before the command.
- **Python**: T prepends `exec(open("utils.py").read())` before the command.
- **Julia**: T prepends `include("utils.jl")` before the command.

A concrete example with Python cleaning and R modelling sharing utility functions:

```
p = pipeline {

  raw = read_csv("data/penguins.csv")

  clean = pyn(
    command      = <{ clean_penguins(raw) }>,
    deserializer = ^ipc,
    serializer   = ^parquet,
    functions    = "src/py_utils.py"   -- defines
 ↪  clean_penguins()
  )

  model = rn(
    command      = <{ fit_model(clean) }>,
    deserializer = ^parquet,
    serializer   = ^pmml,
    functions    = "src/r_utils.R"     -- defines
 ↪  fit_model()
  )

}
```

`src/py_utils.py` is a plain Python file:

```
# src/py_utils.py

def clean_penguins(df):
    return (
        df
        .dropna()

 ↪  .assign(species_code=df["species"].astype("category").cat
        [["bill_length_mm", "bill_depth_mm",
```

```
          "flipper_length_mm", "body_mass_g",
           ↪  "species_code"]]
    )
```

And `src/r_utils.R` is plain R:

```
# src/r_utils.R

fit_model <- function(df) {
  nnet::multinom(species_code ~ ., data = df, trace
 ↪  = FALSE)
}
```

The key benefit is that your node commands become one-liners that delegate to tested, version-controlled utility functions. When the function changes, every node that lists it under `functions` is automatically rebuilt by Nix.

8.5.2 The `include` Parameter

`functions` is designed for executable source files. If your node needs additional *non-executable* resources, such as a configuration YAML, a lookup table, or a template file, use the `include` parameter instead. T copies the listed paths into the sandbox without attempting to source or execute them:

```
report = qn(
  script  = "src/report.qmd",
  include = ["src/custom.scss", "src/logo.png",
 ↪  "data/lookups.csv"]
)
```

8.5.3 Global Pipeline Options

Repeating `functions = "src/utils.R"` on every R node is tedious and error-prone. The `set_pipeline_global_options` function lets you declare defaults once for all nodes of a given runtime:

```
set_pipeline_global_options(
  r      = [functions = "src/r_utils.R"],
  python = [functions = "src/py_utils.py"],
  julia  = [functions = "src/jl_utils.jl"]
)

p = pipeline {

  raw = read_csv("data/penguins.csv")

  -- r_utils.R is automatically sourced in every
 ↪  rn() node
  clean_r = rn(
    command      = <{ prepare(raw) }>,
    deserializer = ^ipc,
    serializer   = ^parquet
  )

  model = rn(
    command      = <{ fit_model(clean_r) }>,
    deserializer = ^parquet,
    serializer   = ^pmml
  )

}

build_pipeline(p)
```

You can also set global `include` and `env_vars` here. Node-level settings take precedence over globals: if a node declares its own `functions`, it overrides the global default for that node (rather than appending to it), so you retain full per-node control.

8.6 Environment Variables in Nodes

The `env_vars` argument passes named environment variables into a node's sandbox. This is the correct way to inject secrets, feature flags, and run-mode switches without hardcoding them in the pipeline source.

```
p_env_vars = pipeline {

  model = rn(
    command  = <{
      mode <- Sys.getenv("MODEL_MODE", unset =
↪   "production")
      if (mode == "debug") {
        message("Debug mode: using 100 trees")
        ntree <- 100L
      } else {
        ntree <- 500L
      }
      randomForest::randomForest(species ~ ., data =
↪   clean, ntree = ntree)
    }>,
    deserializer = ^parquet,
    serializer   = ^pmml,
    env_vars     = [MODEL_MODE: "debug", SEED: "42"]
```

```
  )

}
```

In Python nodes, read variables with `os.environ`:

```
import os
mode = os.environ.get("MODEL_MODE", "production")
```

In Julia nodes, use `ENV["MODEL_MODE"]`.

> **Secrets and reproducibility**
>
> Environment variables passed via `env_vars` are baked into the derivation's input hash. Two builds with different `env_vars` values will produce different Nix derivations and will not share cached results. This is the correct behaviour: a model trained with `MODEL_MODE=debug` and one trained with `MODEL_MODE=production` are genuinely different artifacts.
> For *secrets* that should not appear in the pipeline source (API keys, database passwords), do not hardcode them in `env_vars`. Instead, set them in your shell before running `t run`, and reference them with the `env_from_shell` argument; see the T documentation for details.

8.7 Build Logs and Artifacts

After `build_pipeline(p)` completes, T provides several tools for inspecting what was built, how long it took, and where the artifacts live.

8.7.1 Reading the Build Log

`build_log(p)` returns the raw log for the most recent build as a T value. The companion `build_log_to_frame(log)` materialises it as a `DataFrame` with four columns:

Column	Type	Contents
`name`	`String`	Node name
`status`	`String`	`"built"`, `"cached"`, or `"failed"`
`duration_s`	`Float`	Wall-clock build time in seconds
`path`	`String`	Nix store path of the artifact

```
log   = build_log(p)
frame = build_log_to_frame(log)
frame

name        status    duration_s  path
raw         cached    0.00
 ↪  /nix/store/abc…-raw
clean       built     3.21
 ↪  /nix/store/def…-clean
model       built     8.74
 ↪  /nix/store/ghi…-model
report      built     12.05
 ↪  /nix/store/jkl…-report
```

A `status` of `"cached"` means Nix found an existing derivation with the same input hash and served it from the store without rerunning anything. `"built"` means the node was (re)computed in this run.

8.7.2 Copying Artifacts Out of the Store

Nix store paths are read-only and content-addressed, so they are not meant to be worked with directly. Use `pipeline_copy()` to copy artifacts to a local directory:

```
-- Copy a single node's artifact
pipeline_copy(p.report, dest = "outputs/")

-- Copy every artifact in the pipeline
pipeline_copy(p, dest = "outputs/")
```

After copying, `outputs/report.html` (or whatever format the Quarto node produced) is a regular file you can open, commit to Git, or publish.

8.7.3 Garbage Collection

Over time the Nix store accumulates artifacts from old builds. Two functions help manage this:

`pipeline_gc(p)` removes store paths that belong to *this pipeline's* previous builds but are no longer referenced by the current build graph. Pass `dry_run = true` first to see what would be deleted:

```
pipeline_gc(p, dry_run = true)

Would remove:
  /nix/store/xyz…-clean  (superseded by def…-clean)

pipeline_gc(p)
```

`t_gc()` runs a full Nix garbage collection: it removes every store path that is not referenced by any active GC root on the system. Use this periodically on development machines or CI runners to reclaim disk space:

```
t_gc()
```

> **GC roots and the current pipeline**
>
> `build_pipeline(p)` automatically registers the current build as a GC root, so running `t_gc()` will not remove artifacts from the most recent successful build. Artifacts from *previous* builds of the same pipeline are not registered as roots and will be collected. If you need to compare results across builds, copy the artifact you want to keep out of the store with `pipeline_copy()` before running GC.

8.8 Incremental Builds and Caching

You have already seen that T caches nodes by input hash: if nothing changes, nothing is rebuilt. There is an important subtlety worth understanding clearly.

The pipeline is a child of the environment.

When Nix builds a node, the derivation's input hash includes not just the node's code and upstream data, but also the entire closure of packages and tools that the node's sandbox contains. Those packages come from the date pin and package list in `tproject.toml`. If you update the date pin, say from `2026-03-01` to `2026-06-01`, the environment closure changes, which

changes the hash of every derivation in the pipeline, which means the entire pipeline rebuilds from scratch.

This is not a bug. It is the only way to guarantee that the pipeline's results are reproducible against the *current* environment. If cached results from an old environment were reused after a package update, you could never be certain that a non-backwards-compatible change in a dependency had not silently altered your results. By rebuilding everything when the environment changes, T ensures that what you see is what the *declared* environment actually produces.

In practice, this means:

- Pin the date in `tproject.toml` deliberately. Do not bump it casually.
- When you do need to update the environment, treat it as an event: rebuild, review the log, commit the new `tproject.toml` and the updated results together.

8.8.1 Previewing Cache Hits

Before committing to a full build, you can ask T to report which nodes would be rebuilt versus served from cache by passing `dry_run = true` to `build_pipeline`:

```
build_pipeline(p, dry_run = true)

Would build:  [model, report]
Would cache:  [raw, clean]
```

This is useful when iterating on a slow node (a model fit, a long render) and you want to confirm that only the nodes downstream of your change will run.

The equivalent from the shell:

```
t run src/pipeline.t --dry-run
```

8.9 CI/CD with `pipeline_to_ga()`

A reproducible pipeline is most valuable when it runs automatically: on every push, on a schedule, or when upstream data changes. T ships with a function that generates a complete GitHub Actions workflow YAML from your pipeline definition:

```
pipeline_to_ga(p, file =
 ↪  ".github/workflows/pipeline.yml")
```

That single call writes a workflow file that:

1. Installs Nix on the runner using the Determinate Systems installer.
2. Restores the Nix store cache from the `t-runs` branch of your repository (a dedicated branch that stores serialised store paths between runs).
3. Runs `t run src/pipeline.t` inside the project's Nix shell.
4. Uploads any artifacts you have declared as outputs.
5. Saves the updated Nix store cache back to `t-runs`.

The `t-runs` branch caching approach avoids the GitHub Actions cache size limit by persisting the Nix store as a Git repository. On a typical pipeline, the first CI run downloads and builds everything; subsequent runs restore from cache and only rebuild what changed, the same incremental behaviour you get locally.

A minimal end-to-end CI setup for our penguin pipeline:

```
-- In src/pipeline.t, add at the bottom:
pipeline_to_ga(
  p,
  file     = ".github/workflows/pipeline.yml",
  on_push  = ["main"],
  schedule = "0 6 * * 1"   -- Monday mornings at
 ↪  06:00 UTC
)
```

The generated YAML looks roughly like this:

```
name: T Pipeline

on:
  push:
    branches: [main]
  schedule:
    - cron: "0 6 * * 1"

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses:
 ↪  DeterminateSystems/nix-installer-action@main

      - name: Restore t-runs cache
```

```
        uses: actions/cache@v4
        with:
          path: /nix/store
          key: t-runs-${{ hashFiles('tproject.toml')
↪  }}
          restore-keys: t-runs-

      - name: Build pipeline
        run: nix develop --command t run
↪  src/pipeline.t

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: pipeline-outputs
          path: outputs/
```

Committing `pipeline.yml` to version control

Commit the generated `.github/workflows/pipeline.yml` to your repository. It is generated deterministically from your pipeline definition, so it will be regenerated identically by anyone who runs `pipeline_to_ga(p, ...)`. Treat it like any other generated file: regenerate it when the pipeline changes, review the diff, and commit.

Other CI providers

`pipeline_to_ga()` targets GitHub Actions because it is the most common platform in the community, but the underlying Nix build is CI-agnostic. For GitLab CI, Forgejo Actions, or Jenkins, the pattern is the same: install Nix, restore the

store cache if available, run `t run src/pipeline.t`. The T documentation includes template snippets for the major providers.

8.10 Summary

We have covered a lot of ground in this chapter. Here is what we built:

- A four-stage polyglot pipeline (T → Python → R → Quarto) as a concrete working reference (Section 8.2).
- The full set of node runtimes: `node()`, `rn()`, `pyn()`, `jln()`, `shn()`, and `qn()`, with the practical usage patterns for each (Section 8.3).
- The `fetchurl` / `prefetch` pattern for safely pulling remote data into a sandboxed build (Section 8.4).
- The `functions`, `include`, and `set_pipeline_global_options` mechanisms for sharing code without repetition (Section 8.5).
- The `env_vars` argument for passing flags and secrets into node sandboxes (Section 8.6).
- `build_log_to_frame`, `pipeline_copy`, `pipeline_gc`, and `t_gc` for inspecting and managing build artifacts (Section 8.7).
- Why the pipeline rebuilds when `tproject.toml` changes, and why that is the correct behaviour (Section 8.8).
- `pipeline_to_ga()` for generating a GitHub Actions workflow in three lines (Section 8.9).

Between Chapter 5 and this chapter you now have a complete picture of the T pipeline model for single-machine analytical

workflows. The next chapter takes the pipeline abstraction further: we will look at *pipeline manipulation*: composing pipelines, branching on parameter grids, mapping nodes over collections, and building reusable pipeline templates that can be shared across projects.

9 Advanced Pipeline Patterns

By this point you can declare a T pipeline, build it, inspect its outputs, and wire it into a CI job. That covers the majority of day-to-day work. This chapter goes further: it treats pipelines themselves as data.

In most pipeline tools, `targets` (Landau 2021), Snakemake (Köster and Rahmann 2012), and Airflow, the pipeline *definition* is code that runs once, produces a build graph, and is then discarded. The graph is an internal implementation detail you cannot touch. T takes a different view. A pipeline value is a first-class object you can query, filter, transform, and compose using the same functional idioms you apply to any other value. This opens up meta-programming patterns that simply are not possible in other tools: running only a subgraph, swapping one node's implementation for another, generating dozens of branches from a list, or assembling a production pipeline from tested sub-components.

We cover these patterns in order of complexity, starting with the simplest read-only queries and working up to dynamic branching and composable meta-pipelines. All examples build on the project layout introduced in Chapter 8.

9.1 Pipelines as Data

The pipeline value you construct inside a `pipeline { ... }` block is an ordinary T record. It carries the DAG structure, the node metadata, and enough information for T's build engine to emit Nix derivations. Because it is a record, all the standard higher-order functions, `filter`, `map`, `fold`, and `select`, work on it just as they do on any other collection.

This has a practical consequence: you write *pipeline transformations* in the same language as the pipeline itself, without shelling out to a separate tool or reading a build-graph dump. The chapter illustrates this with a running example: a shared ETL pipeline wired to two alternative model pipelines, one linear, one a random forest. We will build it up gradually.

```
etl = pipeline {
  raw        = rn(<{ read.csv("data/sales.csv") }>)
  validated  = rn(<{ validate_schema(raw) }>)
  cleaned    = rn(<{ clean_missing(validated) }>)
  features   = rn(<{ engineer_features(cleaned) }>)
}
```

Keep this pipeline in mind; we will manipulate it throughout the chapter.

9.2 Querying the DAG

9.2.1 `pipeline_to_frame`

`pipeline_to_frame(p)` converts a pipeline into a data frame whose rows are nodes. The columns are:

Column	Type	What it holds
`name`	string	Node name
`runtime`	string	`"r"`, `"python"`, `"julia"`, `"shell"`, `"quarto"`
`serializer`	string	Encoder used when writing the node's output
`deserializer`	string	Decoder used when reading upstream artifacts
`noop`	bool	If `true`, the node is skipped at build time
`deps`	list	Immediate dependency names
`depth`	int	Distance from any root node
`command_type`	string	`"expr"`, `"file"`, or `"shell"`

You do not need to memorise those columns; the point is that they are queryable with ordinary data-frame operations.

```
df = pipeline_to_frame(etl)
print(df)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

```
name        runtime  serializer   noop  depth
raw         r        rds          false  0
validated   r        rds          false  1
cleaned     r        rds          false  2
features    r        rds          false  3
```

From here, any standard filter or select applies directly. To find every Python node:

```
py_nodes = filter(df, \(row) row.runtime ==
 ↪  "python")
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

To find every node at depth 2 or deeper, which is useful when you want to know which steps are non-trivial downstream work:

```
deep_nodes = filter(df, \(row) row.depth >= 2)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

9.2.2 Structural queries

Four functions give you structural information without materialising a full frame:

- `pipeline_roots(p)`: nodes with no dependencies (the entry points)
- `pipeline_leaves(p)`: nodes on which nothing else depends (the outputs)
- `pipeline_edges(p)`: every `(from, to)` dependency pair as a frame
- `pipeline_depth(p)`: a named record mapping each node name to its depth

```
roots  = pipeline_roots(etl)   -- ["raw"]
leaves = pipeline_leaves(etl)  -- ["features"]
edges  = pipeline_edges(etl)

-- edges:
-- from         to
-- raw          validated
-- validated    cleaned
-- cleaned      features
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

9.2.3 Column projection with `select_node`

`select_node` lets you project specific fields using non-standard evaluation (the `$field` syntax). It returns a frame restricted to the named columns, which is handy when you want to pass a compact summary to a report or a CI log:

```
etl_summary = select_node(etl, $name, $runtime,
 ↪  $depth)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

> **Note**
>
> `select_node` is a read-only view; it does not modify the pipeline. For modifications, see `mutate_node` below.

9.3 Modifying Nodes Surgically

Queries tell you what is in a pipeline. The mutating functions let you change it, always producing a *new* pipeline value and leaving the original intact, in keeping with T's pervasive immutability.

9.3.1 `filter_node`

`filter_node(p, pred)` returns a new pipeline containing only the nodes that satisfy the predicate. DAG validity is *not* checked until `build_pipeline` is called, so you can remove nodes freely and let T report missing-dependency errors only when you actually try to build.

```
-- Keep only the ETL steps that are not "raw"
 ↪  (already built, skip ingestion)
no_ingest = filter_node(etl, $name != "raw")
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

This is the simplest way to produce a lighter pipeline for a fast check run.

9.3.2 `mutate_node`

`mutate_node(p, pred, field = value)` modifies a field on all nodes matching `pred`. Without a predicate, every node is affected.

Skipping expensive nodes. Set `$noop = true` to mark a node as a no-op at build time. T will resolve the node's output by reading the most recent cached artifact instead of rebuilding it, so downstream nodes can still reference it. This is the canonical way to skip slow steps during development:

```
-- Skip the feature-engineering step while iterating
 ↪  on the model
fast_etl = mutate_node(etl, $name == "features",
 ↪  $noop = true)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

Swapping runtimes. You can change the runtime of a node to test whether a Python reimplementation of an R step produces the same result:

```
-- Run "cleaned" in Python instead of R, without
 ↪  touching anything else
py_etl = mutate_node(etl, $name == "cleaned",
 ↪  $runtime = "python")
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

> **Tip**
>
> `mutate_node` *replaces* the field entirely. This differs from `set_pipeline_global_options`, which *prepends* to option lists. Use `mutate_node` when you want a clean override; use `set_pipeline_global_options` when you want to augment existing options.

9.3.3 `rename_node`

`rename_node(p, old, new)` renames a node and automatically rewires every dependency edge that referenced the old name. You do not have to hunt through the pipeline looking for references; T handles the rewriting:

```
-- Rename "features" to "feature_matrix" for a
 ↪  cleaner downstream API
etl2 = rename_node(etl, "features",
 ↪  "feature_matrix")
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
```

9.3.4 `swap`

`swap(p, name, new_node)` replaces a node's implementation while keeping all its dependency edges intact. The new node inherits the same name and position in the DAG; only its command changes. This is the right tool when you want to test an alternative algorithm without restructuring the pipeline:

```
glm_node = r { glm(target ~ ., data = features,
 ↪  family = gaussian()) }

model_pipeline_glm = swap(model_pipeline,
 ↪  "linear_model", glm_node)
```

9.3.5 `rewire`

`rewire(p, name, new_deps)` reroutes a node's dependency edges. The node's command stays the same; only *what it reads from* changes. Use it when you want to connect the same computation to a different upstream:

```
-- Point "linear_model" at "features_v2" instead of
 ↪  "features"
rewired = rewire(model_pipeline, "linear_model",
 ↪  ["features_v2"])
```

9.3.6 A realistic "what-if" workflow

Putting this together: suppose you have a model step that takes twenty minutes to run, and you want a quick sanity check that only reruns the feature engineering and report steps.

```
-- 1. Build the full pipeline once
res = build_pipeline(full_pipeline)

-- 2. For the next iteration, skip the model and go
 ↪  straight to the report
fast_check = full_pipeline
  |> mutate_node($name == "model", $noop = true)
  |> build_pipeline()
```

Because T cached the model node on step 1, step 2 reads that cache and immediately builds the report. No data is lost; no special configuration is required.

9.4 Set Operations

T exposes four set operations on pipeline values. They compose naturally: the output of any set operation is itself a pipeline, so you can chain them.

9.4.1 `union`

`union(p1, p2)` merges two pipelines. If both contain a node with the same name, T raises an error at construction time. Use `rename_node` on one of the pipelines first if you need to resolve the collision:

```
full = union(etl, model_pipeline)
```

9.4.2 `difference`

`difference(p1, p2)` keeps the nodes in `p1` that do not appear in `p2`. This is useful when you want to strip out a phase you no longer need:

```
-- Keep only the modelling steps; drop the ingestion
 ↪  phase
ingest_only = pipeline {
  raw       = rn(<{ read.csv("data/sales.csv") }>)
  validated = rn(<{ validate_schema(raw) }>)
}
trimmed = difference(etl, ingest_only)
print(pipeline_nodes(trimmed))
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
```

> **Note**
>
> After removing nodes with `difference`, you may be left with leaf nodes that have dangling dependencies. Run `prune` (see below) to strip them automatically.

9.4.3 `intersect`

`intersect(p1, p2)` keeps only the nodes that appear in *both* pipelines, using the definitions from `p1`. This is handy when you want to extract the shared core of two related pipelines:

```
shared_core = intersect(full_pipeline_v1,
 ↪  full_pipeline_v2)
```

9.4.4 `patch`

`patch(p, overrides)` updates existing nodes in `p` using definitions from `overrides`, but ignores any node in `overrides` that does not already exist in `p`. This is the safe way to apply configuration overrides: new nodes in the override pipeline cannot accidentally sneak into the production pipeline.

```
-- Override the "cleaned" node without risk of
 ↪  adding unreviewed steps
prod = patch(prod_pipeline, dev_overrides)
```

9.4.5 `prune`

`prune(p)` removes all leaf nodes, i.e. nodes that nothing else depends on. It is typically used after `difference` to clean up dangling tails:

```
-- Remove the report node, then prune anything that
 ↪  became orphaned
no_report = difference(full_pipeline, ["report"])
              |> prune()
```

9.4.6 A realistic use-case: shared ETL, swapped model

Here is the pattern you will reach for most often in a production setting: a shared ETL pipeline assembled once, then forked into two model pipelines that differ only in their model node.

```
-- Shared ETL (defined once, potentially sourced
 ↪  from its own file)
etl = pipeline {
  raw      = r { read.csv("data/sales.csv") }
  cleaned  = r { clean_missing(raw) }
  features = r { engineer_features(cleaned) }
}

-- Linear model pipeline
linear = pipeline {
  linear_model = r { lm(target ~ ., data = features)
 ↪  }
  report       = qmd { "reports/linear.qmd" }
}

-- Random forest pipeline: same structure, different
 ↪  model node
rf_node = r { randomForest::randomForest(target ~ .,
 ↪  data = features) }
rf      = swap(linear, "linear_model", rf_node)
          |> rename_node("report", "report_rf")

-- Assemble both into one build
full = union(etl, union(linear, rf))
res  = build_pipeline(full)
```

We build one ETL phase and two model branches in a single call. Nix caches the ETL outputs and reuses them for both branches.

9.5 DAG-Aware Transformations

The set operations above operate on names. The DAG-aware functions operate on *structure*: they follow dependency edges to extract connected subgraphs.

9.5.1 `upstream_of`

`upstream_of(p, name)` returns a new pipeline containing the named node and all of its transitive ancestors:

```
-- Everything needed to produce "features", and
 ↪  nothing more
etl_only = upstream_of(full, "features")
build_pipeline(etl_only)
```

9.5.2 `downstream_of`

`downstream_of(p, name)` returns the named node and everything that directly or transitively depends on it:

```
-- Re-run the model and every downstream step after
 ↪  changing a hyperparameter
model_onwards = downstream_of(full, "linear_model")
build_pipeline(model_onwards)
```

This is the most common use of DAG-aware transformations: you change a parameter, extract everything that is sensitive to that parameter, and rebuild only that subgraph.

9.5.3 `subgraph`

`subgraph(p, names)` returns the full connected component reachable from any of the named nodes, traversing both upstream and downstream edges. Use it when you want to isolate a middle section of a complex pipeline for testing:

```
-- Isolate the cleaning and feature-engineering
 ↪  stages
mid = subgraph(full, ["cleaned", "features"])
```

> **Tip**
>
> Combine `downstream_of` with `mutate_node` for a surgical rebuild after a parameter change: extract the affected subgraph, mark the changed node as non-noop (or simply rebuild it), and leave everything upstream cached.

9.6 Pipeline Composition

So far we have been working with flat pipelines assembled by hand. T provides higher-level composition tools for when you want to treat pipelines as modules.

9.6.1 `chain`

`chain(p1, p2)` wires two pipelines together explicitly: it requires that `p2` references at least one node from `p1` by name. This is stricter than `union` (which silently accepts unconnected pipelines) and is the right choice when you want T to enforce that the wiring is intentional.

```
chained = chain(etl, model_pipeline)
```

If `p2` contains no reference to any node in `p1`, `chain` raises an error at construction time.

```
chained = chain(etl, model_pipeline)
```

The stub node costs nothing at build time, since T sees it as a no-op alias, but it satisfies the dependency declaration that `chain` requires.

9.6.2 `parallel`

`parallel(p1, p2)` combines two independent pipelines that share no nodes and need no wiring. T checks for name collisions and raises an error if it finds any:

```
report_en  = pipeline { report =
 ↪  qn("reports/report_en.qmd") }
report_fr  = pipeline { report =
 ↪  qn("reports/report_fr.qmd") }

-- Name collision! Both have a node called "report"
```

```
-- parallel(report_en, report_fr)  -- ERROR

report_fr2 = rename_node(report_fr, "report",
 ↪  "report_fr")
both       = parallel(report_en, report_fr2)
print(pipeline_nodes(both))
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-75cac4eeb890d239/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

9.6.3 `pipeline_of` meta-pipelines

`pipeline_of` lets you compose multiple named pipelines into a higher-order DAG, where each sub-pipeline becomes a namespace. Nodes are auto-namespaced with the sub-pipeline name as a prefix: a node `summary` inside a sub-pipeline called `stats` becomes `stats.summary`. You can build, read, and inspect the meta-pipeline exactly as you would a flat pipeline:

```
meta = pipeline_of {
  etl   = etl_pipeline
  stats = stats_pipeline
  plots = plots_pipeline
}
```

```
-- Build the whole thing
res = build_pipeline(meta)

-- Read a namespaced node
read_node("stats.summary")
```

> **Note**
>
> Dependencies between sub-pipelines still need to be declared. If `stats` references `etl.features`, that reference must appear in the node's command, or you can use a T-stub as described above.

9.6.4 Pipeline templates via lambdas

Because pipelines are values, you can wrap them in functions to produce parameterised templates. This is the T idiom for the factory pattern:

```
make_model_pipeline = \(multiplier: Float ->
 ↪  Pipeline) pipeline {
  scaled_features = rn(<{ features * multiplier }>)
  model           = rn(<{ lm(target ~ ., data =
 ↪  scaled_features) }>)
  report          = qn("reports/model.qmd")
}

p1   = make_model_pipeline(1.0)
p2   = make_model_pipeline(0.5)

both = parallel(p1 |> rename_node("report",
 ↪  "report_1x"),
```

```
                p2 |> rename_node("report",
 ↪  "report_half"))
print(pipeline_nodes(both))
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-5cd2a9653fe5535a/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-5cd2a9653fe5535a/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

9.7 Dynamic Branching

Static pipelines describe a fixed graph. Dynamic branching generates nodes at build time: one node per element of a list, or one per combination of parameter values. The resulting graph can be large, but you declare it concisely.

9.7.1 `map_pattern`

`map_pattern(dep)` generates one branch per element of an upstream dependency. You pass it as the `pattern` argument to `node()`, and the node's `command` references the dependency by name. T substitutes each element in turn:

```
p_map = pipeline {
  params = [10, 20, 30]
  results = node(
    command = <{ compute(params) }>,
    pattern = map_pattern(params)
  )
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-f0a036b347b17629/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-f0a036b347b17629/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

This expands into three nodes, `results_branch_1`, `results_branch_2`, and `results_branch_3`, where branch `i` runs the command with element `i` of `params`. Multiple dependencies can be mapped at once; they must all have the same length, and branch `i` receives element `i` from each:

```
p_map2 = pipeline {
  xs = [1, 2, 3]
  ys = [10, 20, 30]
  z = node(command = <{ xs + ys }>, pattern =
 ↪  map_pattern(xs, ys))
```

```
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-0e6d03120adfba82/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-0e6d03120adfba82/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

9.7.2 `cross_pattern`

`cross_pattern(sub1, sub2, ...)` generates a Cartesian product of `map_pattern` sub-patterns: one branch per combination of values:

```
p_cross = pipeline {
  radii  = [1, 2]
  cycles = [3, 4]
  spiro = node(
    command = <{ spirograph(radii, cycles) }>,
    pattern = cross_pattern(map_pattern(radii),
 ↪  map_pattern(cycles))
  )
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6fdc8030400597be/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6fdc8030400597be/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

This produces four branches, `spiro_branch_1` through `spiro_branch_4`, covering `(1,3)`, `(1,4)`, `(2,3)`, and `(2,4)`. The spirograph example in the T demos does exactly this: three values of angular frequency crossed with three values of petal count produce nine data-generation branches and nine R-based plot branches, each built in parallel.

9.7.3 Selector patterns

When a map or cross generates more branches than you want, the selector patterns reduce the set. Each takes a dependency and an integer parameter and is passed as the `pattern` argument, just like the others:

- `slice_pattern(dep, [i, j, ...])`: branch on the elements at the given 0-based indices.
- `head_pattern(dep, n)`: branch on the first `n` elements.
- `tail_pattern(dep, n)`: branch on the last `n` elements.

- `sample_pattern(dep, n)`: branch on `n` randomly chosen elements.

```
p_select = pipeline {
  x = [10, 20, 30, 40, 50]
  first_two = node(command = <{ x }>, pattern =
 ↪  head_pattern(x, 2))
  last_two  = node(command = <{ x }>, pattern =
 ↪  tail_pattern(x, 2))
  odd       = node(command = <{ x }>, pattern =
 ↪  slice_pattern(x, [0, 2, 4]))
  random    = node(command = <{ x }>, pattern =
 ↪  sample_pattern(x, 2))
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-7adb73e9b1894cb3/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-7adb73e9b1894cb3/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

`head_pattern` and `tail_pattern` cap at the dependency length, so `head_pattern(x, 10)` on a five-element list still yields five branches. `sample_pattern` is deterministic: it seeds a partial Fisher–Yates shuffle from the node name, so re-expanding the

same pipeline always makes the same draw, while different node names make different draws.

9.7.4 Expanding and inspecting

Patterns expand automatically when you call `build_pipeline()` or `populate_pipeline()`, so you can build a patterned pipeline directly. To inspect the expanded structure first, or to write it out as an explicit script, call `expand_pipeline()`:

```
demo = pipeline {
  x = [10, 20, 30, 40, 50]
  first_two = node(command = <{ x }>, pattern =
 ↪  head_pattern(x, 2))
}
expanded = expand_pipeline(demo)
pipeline_nodes(expanded)
-- ["x", "first_two_branch_1", "first_two_branch_2"]

expanded.first_two_branch_1  -- the first branch's
 ↪  node
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6e8c4435d9a1bab8/cl
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6e8c4435d9a1bab8/cl
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

```
node<T>(...)
```

Each branch is a regular node named `<node>_branch_<i>` with a 1-based index, so you can read it with the usual `read_node()` or dot access.

> **Tip**
>
> During development, use `head_pattern(dep, 1)` to prototype your command on a single branch before committing to a full grid build.

9.8 Cross-Node Artifact Retrieval

Patterns fan a node out into branches, but sometimes a node needs the *deserialized* output of another node in the same pipeline, not just its declared dependencies passed through as raw store paths. T exposes every node's materialized artifact to the Nix sandbox through an environment variable, and a small `get()` helper makes it easy to pull one in.

In a Nix sandbox, each dependency `dep` is available as the environment variable `T_NODE_<dep>`, which holds the path to that node's store directory. To read a sibling node's deserialized value from inside a node's command, use `get()` with a `node_lens` locator:

```
p_lens = pipeline {
  node_a = node(command = <{ make_a() }>)
  node_b = node(
    command = <{ combine(get(node_lens("node_a")),
↪   make_b()) }>,
```

```
    deps = [node_a]
  )
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-e759ccc0f6e34140/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-e759ccc0f6e34140/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

`get(node_lens("node_a"))` deserializes `node_a`'s artifact and returns it as a T value, so `node_b` can use it directly even though the value is produced elsewhere in the pipeline. This is the canonical way to reach across the dependency graph when a plain `deps` list is not enough.

9.9 Static Conditionals

Dynamic branching generates nodes from data. Static conditionals *exclude* nodes from the pipeline entirely at construction time, based on conditions evaluated before any build starts. This is the right tool for environment-specific toggles.

9.9.1 `node_when`

`node_when(condition, node)` includes `node` in the pipeline only if `condition` is truthy. If the condition is falsy, the node is simply absent: it does not appear in the DAG at all, and any downstream node that references it will report a missing-dependency error (which is the correct behaviour: you should not silently skip a node that something else needs).

The idiomatic pattern reads environment variables via `env()`:

```
p_when = pipeline {
  raw      = rn(<{ read.csv("data/sales.csv") }>)
  cleaned  = rn(<{ clean_missing(raw) }>)
  features = rn(<{ engineer_features(cleaned) }>)

  -- Only run the expensive validation step outside
 ↪  CI
  deep_validation = node_when(env("CI") == "",
    rn(<{ run_expensive_validation(cleaned) }>))
}
```

In a CI environment where `CI` is set (GitHub Actions sets it to `"true"`), `deep_validation` is simply absent. In a local development shell where `CI` is unset (empty string), it is included.

9.9.2 `node_fork`

`node_fork` is a multi-way branch: it evaluates a list of `(condition, node)` pairs and includes the *first* node whose condition is truthy. An optional `.default` provides a fallback:

```
model_type = env("MODEL")

p = pipeline {
  features = r { engineer_features(cleaned) }

  node_fork(
    model_type == "linear", {
      model = r { lm(target ~ ., data = features) }
    },
    model_type == "forest", {
      model = r { randomForest::randomForest(target
 ↪  ~ ., data = features) }
    },
    model_type == "neural", {
      model = py { train_neural_net(features) }
    },
    .default = {
      model = r { lm(target ~ ., data = features) }
    }
  )

  report = qmd { "reports/model.qmd" }
}
```

Setting `MODEL=forest` in the shell and running the pipeline will include only the random-forest node. The report node references `model` and will build against whichever implementation was selected. Adding a new model type requires one extra clause in `node_fork`, nothing else.

> **Note**
>
> Static conditionals are evaluated at pipeline *construction* time, not at build time. `env()` reads the shell environment at the moment you call `build_pipeline(p)`. If you change `MODEL` and call `build_pipeline` again in the same T session, the pipeline is reconstructed and the new value is picked up.

9.10 Custom Flakes Per Node

By default, every node in a T pipeline uses the Nix flake specified in `tproject.toml`. The packages you list there are installed in every node's environment. The *flake* determines which revision of `nixpkgs` those packages are drawn from.

The `flake` argument on `node()`, `rn()`, and `pyn()` lets individual nodes use a *different* flake entirely:

- **Pin one node to an older `nixpkgs` snapshot**: a legacy model that requires an older version of a C library.
- **Use a specialist R distribution for one node**: for example, `github:jbedo/rshells` for a node that needs R with specific compile-time options.
- **Point a node at a local flake**: for a private package not yet in `nixpkgs`.

```
p = pipeline {
  cleaned  = r { clean_missing(raw) }
  features = r { engineer_features(cleaned) }

  -- Pin a specific nixpkgs snapshot for a legacy
 ↪  Stan model
```

```
  legacy_model = rn(
    { run_stan_model(features) },
    flake = "github:NixOS/nixpkgs/nixpkgs-24.05"
  )

  -- Use a local flake for an internal package
  internal_report = rn(
    { render_internal_report(legacy_model) },
    flake = "path:../internal_flake"
  )
}
```

> **Note**
>
> The packages listed in `tproject.toml` still determine *what* is installed in every node. The `flake` argument determines *which nixpkgs revision* resolves those package names. You can combine per-node flakes with project-wide packages freely; T merges them correctly when emitting each derivation.

A practical guideline: keep the project-wide flake at a recent `nixpkgs` pin for everything except nodes with known compatibility constraints, and use per-node flakes only where necessary. Overusing per-node flakes makes the build harder to reason about.

9.11 Validation

Pipeline construction in T is cheap: values are not built until `build_pipeline` is called. But some errors (cycles, missing

dependencies, unknown runtimes) are structural and can be caught before any Nix derivation is emitted.

9.11.1 `pipeline_validate`

`pipeline_validate(p)` returns a list of error messages. It *never* throws; if the pipeline is valid, it returns an empty list. This makes it suitable for CI checks where you want to collect all errors at once:

```
errs = pipeline_validate(full_pipeline)

if length(errs) > 0 {
  for e in errs { print(e) }
  exit(1)
}
```

The validator checks:

- **Cycles**: any circular dependency in the DAG.
- **Missing dependencies**: a node references a name that is not defined.
- **Unknown runtimes**: a node declares a runtime T does not recognise.
- **Cross-runtime deserialiser gaps**: a Python node reads from an R node but no compatible deserialiser is configured.
- **File existence**: nodes that reference local files check that those files are present at validation time.

9.11.2 `pipeline_assert`

`pipeline_assert(p)` throws on the *first* error it finds. Use it as a guard at the top of a pipeline construction chain when you want to fail fast:

```
full_pipeline = chain(etl, model_pipeline)
                |> pipeline_assert()
                |> build_pipeline()
```

If `chain` produces an invalid graph, `pipeline_assert` surfaces the error immediately rather than waiting for Nix to fail deep inside a derivation build.

> **Tip**
>
> In CI, prefer `pipeline_validate` so that the log shows *all* structural errors in one run. In interactive development, prefer `pipeline_assert` so you get a fast failure at the exact line where the broken pipeline was constructed.

`pipeline_cycles(p)` is a narrower variant that returns only cycle-related errors, which is useful when you are debugging a `union` or `chain` that you suspect has introduced a circular dependency.

9.11.3 Pipeline state as DataFrames

Validation tells you what is wrong. Three companions tell you what *is*, each as a DataFrame you can filter and join like any other table:

```
pipeline_status(p)        -- name, runtime, status,
 ↪  duration, path, error
diff_summary(p)           -- per-node
 ↪  changed/unchanged between last builds
pipeline_config_to_frame(p) -- one row per node:
 ↪  serializer, provenance
```

`pipeline_status(p)` lists failed nodes first, so the interesting rows are always at the top. `diff_summary(p)` compares the two most recent builds without loading artifacts. `pipeline_config_to_frame(p)` shows the resolved configuration, including which settings came from global options and which from the node itself.

9.12 Handling Ambiguous Dependencies

T infers which nodes depend on which by scanning the lexical content of each node's command for names that match other nodes in the pipeline. This heuristic works in the vast majority of cases and requires no boilerplate. Two situations trip it up.

First, comments. T strips comments before scanning, so a node name that appears only in a comment is invisible to the analyser:

```
-- This will NOT create a dependency on "features":
model = rn(<{
  # features is the input (see the data-prep
 ↪  pipeline)
  lm(target ~ ., data = cleaned_features)
}>)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-5443f9d339f2943d/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-5443f9d339f2943d/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

Second, shell nodes that read files created by other nodes. The file name is a runtime string and the node name never appears in the command text:

```
summarise_outputs = shell {
  Rscript summarise.R > summary.txt
}
```

If `summarise.R` reads a file written by another node, T cannot infer that dependency from the shell command text alone.

For both cases, the `deps` argument force-declares dependencies that cannot be inferred:

```
model2 = rn(
  <{ lm(target ~ ., data = cleaned_features) }>,
  deps = ["features"]
)
```

```
summarise_outputs = shn(
  <{ Rscript summarise.R > summary.txt }>,
  deps = ["model_output", "validation_report"]
)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-398cc4db68da71fa/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-398cc4db68da71fa/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

Declared dependencies are merged with inferred ones. You do not need to re-declare dependencies that T already finds; `deps` only adds what would otherwise be missing.

> **Note**
>
> The `deps` argument is also the right tool when a node's command is generated dynamically (for example, via a string built at construction time) and cannot be statically scanned. Declaring deps explicitly is always safe; the only cost is a small amount of extra annotation.

9.13 Summary

The patterns in this chapter compose. A real production pipeline might look like this:

```
-- 1. Assemble the ETL and model sub-pipelines
etl   = chain(ingest_pipeline, clean_pipeline)
        |> pipeline_assert()

model = swap(baseline_pipeline, "model",
 ↪  tuned_model_node)

-- 2. For a fast CI run, skip the slow validation
 ↪  and use a cached model
ci_check = union(etl, model)
           |> mutate_node($name ==
 ↪  "deep_validation", $noop = true)
           |> downstream_of("feature_matrix")

-- 3. For the full nightly run, build everything
 ↪  including the report
nightly = union(etl, union(model, report_pipeline))

-- 4. Build whichever is appropriate for this
 ↪  invocation
target = if env("CI") != "" then ci_check else
 ↪  nightly
res    = build_pipeline(target)
log    = build_log_to_frame(res)
```

This is twelve lines of T that orchestrate conditional skipping, structural extraction, alternative implementations, and

CI/nightly branching, without a single configuration file or wrapper script.

The patterns also stack:

- Use `upstream_of` to extract a subgraph.
- Use `filter_node` or `mutate_node` to skip or swap nodes within it.
- Use `cross_pattern` to fan out over a parameter grid.
- Use `node_fork` to select an implementation based on an environment variable.
- Validate with `pipeline_validate` before any Nix work begins.

The next chapter covers debugging and error handling: what to do when a node can *fail in an expected way* and you want to propagate that failure gracefully rather than aborting the whole build. Chapter 12 then covers how to package a mature pipeline as a distributable artefact.

10 Debugging and Error Handling

In the previous chapters, you learned functional programming fundamentals and how to build reproducible pipelines with T. Now we'll add another layer of robustness: **error handling** and **debugging**.

When a pipeline runs, things inevitably go wrong. A dataset might have unexpected missing values. A computation might divide by zero. A join might drop more rows than expected. When these issues occur, the first question is always: **where did it happen, and what do we do about it?**

T answers that question at three levels:

- **Expressions**: errors are *values*, so a failing computation produces something you can inspect, branch on, and recover from instead of crashing the program.
- **Pipelines**: a failing node can be *captured* without aborting the whole build, and the build log tells you exactly what happened and where.
- **Interactive sessions**: when static inspection is not enough, you can drop into the failing node's own runtime and poke at the real data.

By the end of this chapter, you'll know how to write pipelines that fail gracefully, how to read the diagnostics T produces, and how to use the interactive debugger to get inside a failing node.

10.1 Errors Are Values

In most languages, an exception is an interruption: it unwinds the call stack, and unless someone catches it, the program dies. T takes a different view. An error in T is just another value, a first-class citizen like a number, a string, or a DataFrame, and, like any other value, it can be passed around, inspected, and transformed.

Consider the most classic failure of all:

```
err = 1 / 0
```

In a strict language this would crash. In T, the expression *evaluates* to an error value, and the program continues. You can ask the error value questions:

```
err = 1 / 0
print(is_error(err))
print(error_code(err))
print(error_msg(err))
print(error_context(err))
```

```
true
DivisionByZero
Division by zero.
{}
```

- `is_error(x)` returns `true` if `x` is an error value, `false` otherwise.
- `error_code(x)` returns the machine-readable code, here `DivisionByZero`.
- `error_msg(x)` returns the human-readable message.
- `error_context(x)` returns a dict of context attached to the error. It is empty for a bare arithmetic failure, but errors raised inside functions can carry stack information.

This is what makes T *resilient* by default: a failing expression does not stop the world, it produces a value that flows downstream, and every function downstream gets a chance to notice it and react.

10.2 Resilient Builds and Fail-Fast

That resilience extends to whole pipelines. When a node's code raises an error, T captures it as a first-class value, marks the node accordingly, and, by default, keeps building every branch of the pipeline that does not depend on the failed node.

Every node in a build ends up in one of four states:

Status	What it means
`Completed`	The node ran successfully and serialized its output artifact.
`Completed with error`	The node's code ran but raised a user-space exception. T captured the error as a value, and independent branches kept building.

Status	What it means
`Errored`	The sandbox itself failed: a syntax error, a missing package, running out of memory. This is a *hard* failure that aborts the build.
`Skipped`	The node was never run because an upstream dependency suffered a hard failure.

The important distinction is between the first two rows. A `Completed with error` node is a *soft* failure: the code ran, the error is a value, and the build carries on. An `Errored` node is a *hard* failure: Nix's build machinery itself gave up, and everything downstream is marked `Skipped`.

If you want the opposite behaviour, i.e. to stop at the first error, T has a fail-fast mode:

```
t run --failfast src/pipeline.t
```

From the REPL you can set `t_run(failfast = true, ...)` for a single run, or declare `t_make(failfast = true)` when defining a pipeline. For most data pipelines the default resilient mode is what you want: you get as many diagnostic outcomes as possible from a single pass, and you decide afterwards which failures actually matter.

10.3 Pipes: Propagate or Recover

T has two pipe operators, and the difference between them *is* the error-handling story.

The ordinary pipe `|>` is *short-circuiting*: if the value on the left is an error, the function on the right is not applied, and the error flows through untouched. The maybe-pipe `?|>` *always forwards* the value, errors included, to the function on the right, giving that function a chance to look at the error and recover.

```
doubled = 5 |> \(x: Int -> Int) x * 2
print(doubled)
e = 10 / 0
propagated = e |> \(x: Int -> Int) x + 1
print(is_error(propagated))
print(error_code(propagated))
recover = \(x: Any -> Int) if (is_error(x)) 0 else x
print(e ?|> recover)
print(5 ?|> recover)
```

```
10
true
DivisionByZero
0
5
```

Three things to notice. First, `|>` on an error passes the error straight through: `propagated` is still the original `DivisionByZero` error, which means the `x + 1` function was never applied. Second, `?|>` hands the error to `recover`, which inspects it with `is_error` and substitutes a default. Third, `?|>` is not only for errors: when the value is a normal one, the function runs exactly as it would with `|>`.

Use `|>` for the majority of your pipelines, where you expect success and want any error to keep propagating until something explicitly handles it. Use `?|>` at the specific points where you

want to *act* on an error: log it, replace it with a fallback, or transform it.

10.4 Pattern Matching

You will first meet `match` as a way to recover from errors, but it is a general tool: it branches on the *shape* of a value rather than its equality, and it works on lists, missing values, errors, and plain values alike.

A `match` expression takes a scrutinee and a set of cases. The cases are tried in order, and the first pattern that fits determines the result:

```
describe = \(x: Any -> Any) match(x) {
  [] => "empty list",
  [h, ..rest] => str_join(["head ", h, ", ",
 ↪  length(rest), " more"], ""),
  _ => "not a non-empty list"
}
print(describe([]))
print(describe([1, 2, 3]))
print(describe(42))
```

```
empty list
head 1, 2 more
not a non-empty list
```

The patterns you can write are:

Pattern	Matches
`NA`	a missing value
`[]`	the empty list
`[a, b]`	a list of exactly two elements, binding `a` and `b`
`[h, ..rest]`	a non-empty list, binding the head `h` and the remaining list `rest`
`Error { m }`	an error value, binding its message to `m`
`x`	any value, binding it to `x`
`_`	any value, without binding it

There are no literal patterns: a bare `42` is not a valid pattern, so use a variable and test it if you need to. Patterns nest, so `[a, [b, c]]` matches a list whose second element is itself a two-element list. Because cases are tried top to bottom, put specific patterns before general ones: a bare `x` or `_` at the end catches everything that did not match above, exactly as the `describe` example does.

10.4.1 Matching errors

The most common use in a data pipeline is recovery. `match` lets you destructure an error value and branch on what went wrong:

```
caught = match(1 / 0) { Error { m } =>
 ↪  str_join(["Caught: ", m], ""), _ => "no error" }
print(caught)
safe = match(1 / 0) { Error { _ } => 0, _ => 1 }
```

```
print(safe)
fallback = match(42) { Error { m } =>
 ↪  str_join(["Caught: ", m], ""), _ => 99 }
print(fallback)
```

```
Caught: Division by zero.
0
99
```

The `Error { m }` pattern binds the error's message to `m`, so you can build a diagnostic string, write it to a log, or feed it into a fallback computation. The `_` pattern catches everything else, in the last example the successful value `42`, and returns it (or a default) unchanged.

10.4.2 Recovery patterns

From there, the recovery patterns fall out naturally:

- **Default values**: `?|> \(x: Any -> Int) if (is_error(x)) 0 else x` replaces any failure with a sensible default.
- **Fallbacks**: try the expensive computation first, and if it fails, fall back to a cheaper approximation; the `match` above is exactly this shape.
- **Log and continue**: print `error_msg(x)` inside the error branch, then return a placeholder so the rest of the pipeline can run and you get a full diagnostic report from one pass.
- **Log and rethrow**: print the message, then `error(error_code(x), error_msg(x))` to keep the failure visible to whoever is upstream of you.

For a single, uniform recovery, a `?|>` with an `if (is_error(x))` is the lightest tool. When different error types need different responses, or when the recovery logic has more than one branch, `match` is clearer.

10.5 When a Pipeline Node Fails

Let's put it together. The pipeline below has three nodes: one that succeeds, one that divides by zero, and one more that succeeds.

```
p = pipeline {
  nums = [10, 0, 30]
  bad = 1 / 0
  ok = 42
}
build_pipeline(p)
print(is_error(p.bad))
print(error_code(p.bad))
print(error_msg(p.bad))
print(is_error(read_node(p.bad)))
```

```
false
DivisionByZero
Division by zero.
true
```

Because the build is resilient, `build_pipeline(p)` completes: `nums` and `ok` are built, and `bad` is marked `Completed with error`. The build summary you see on the console names the failing node and even suggests the next command to run.

The four `print` statements show how you interrogate the result. Note the subtlety: `is_error(p.bad)` is `false`, because the *node* is not an error: it is a node that *contains* an error. The error lives in the node's artifact. That is why `error_code(p.bad)` and `error_msg(p.bad)` work directly on the node (they look inside it), and why `is_error(read_node(p.bad))` is the way to check the node's *value*: `read_node` deserializes the artifact, and that value is indeed an error.

10.6 Inspecting a Failed Build

Beyond per-node checks, T gives you build-wide views of what went wrong.

`collect_exceptions(p)` gathers every captured error in the pipeline into a DataFrame with a row per failing node:

```
collect_exceptions(p)
```

```
  node  status  code            message
  ----  ------  --------------  -----------------
  bad   Error   DivisionByZero  Division by zero.
DataFrame: 1 rows x 4 cols
```

In a long pipeline with several independent failures, this is the fastest way to see the full damage report in one place.

Chapter 8 introduced `build_log(p)` and `build_log_to_frame()`, which turn the most recent build into a DataFrame with one row per node: name, status, duration, and artifact path:

```
bl = build_log(p)
build_log_to_frame(bl)
```

```
  name  status                duration  path
  ----  --------------------  --------
 ↪  ----------------------------------
  nums  Completed             0.8792
 ↪  /nix/store/ds36srq9gb2q49bhsri1z...
  bad   Completed with error  0.8896
 ↪  /nix/store/ds36srq9gb2q49bhsri1z...
  ok    Completed             0.8686
 ↪  /nix/store/ds36srq9gb2q49bhsri1z...
DataFrame: 3 rows x 4 cols
```

The `status` column is exactly the four-state vocabulary from earlier in this chapter, and the `duration` column tells you which nodes are slow even when they succeed. (One gotcha: assign the log to a variable that is not already a T builtin, since `log` is the natural logarithm, before passing it to `build_log_to_frame`.)

Finally, `inspect_node` reports the metadata T recorded for a single node:

```
inspect_node(p.bad)
```

```
dict
  name: "bad"
  runtime: "T"
  path:
 ↪  "/nix/store/lvxlc8a6bll2hlwnbxyj7vbvxyw3f3wf-pipeline_out
  serializer: "default"
  class: "Error"
```

```
  dependencies: []
  warnings: []
```

The `class` field is the node's value type as T sees it: `Int` for a healthy node, `Error` for a soft failure. Combined with the build log, this is usually all you need to triage a failed build without ever opening a debugger.

10.7 Reading Artifacts from R and Python

Everything above runs inside T. But built artifacts also live in plain files, and the `tlang` companion libraries let R and Python read them directly, without a running T session. Both companions pick the reader from each node's recorded `serializer`, so JSON, CSV, IPC, and Parquet nodes load with no extra arguments:

```
library(tlang)

model <- read_node("model")      # default
 ↪  serializer -> readRDS()
table <- read_node("features")   # ^json ->
 ↪  jsonlite, ^csv -> read.csv()
```

```
import tlang

model = tlang.read_node("model")      # default
 ↪  serializer -> pickle
table = tlang.read_node("features")   # ^json ->
 ↪  json.load, ^csv -> pandas
```

Formats with no native reader (`^pmml`, `^onnx`, `^bin`) raise and point at `return_path = TRUE` plus a custom deserializer instead of failing silently.

One node is rarely the whole story. `read_node_tree()` loads a node plus its transitive `children` (the default), `parents`, or `both`, all from the single build log selected up front, so a concurrent rebuild cannot mix two snapshots mid-read:

```
tree = tlang.read_node_tree("clean_data")
 ↪  # node + children
tree = tlang.read_node_tree("clean_data",
 ↪  include="both") # parents + children
```

A single unreadable node aborts the tree by default. Pass `on_unreadable="path"` to keep going: the bad node comes back as a small record holding the artifact `path` and the recorded `serializer`, so you can inspect or convert the file by hand. `on_unreadable="skip"` drops bad nodes instead. Both fallbacks warn naming the node, the serializer, the path, and the original error:

```
tree = tlang.read_node_tree(
    "clean_data",
    include="both",
    on_unreadable="path",
)
tree["model"]["path"]        # file to inspect by
 ↪  hand
tree["model"]["serializer"]  # format recorded in
 ↪  the build log, e.g. "pmml"
```

The same triage views from earlier in this chapter exist on the companion side. `inspect_pipeline()` lists every node with its runtime, serializer, dependencies, status, class, and path (or `unbuilt` rows from `dag.json` when nothing is built yet), while `list_logs()`, `build_log_to_frame()`, and `collect_exceptions()` mirror their T namesakes:

```
tbl <- inspect_pipeline()   # one row per node,
 ↪  newest build
errs <- collect_exceptions() # error and warning
 ↪  rows, like T
```

Use the T functions when you are already in a pipeline session; reach for the companions when the consumer is an R or Python script, a Quarto document, or a debug shell that should not depend on T at all.

10.8 Interactive Debugging

Sometimes the build log is not enough. The classic case: a Python node fails with a cryptic traceback, the real inputs are locked in the Nix store, and writing mock inputs by hand is tedious and error-prone. For that, T has an interactive debugger that drops you *inside* the failing node's runtime, with all of its upstream dependencies already materialized.

10.8.1 The `t debug` Command

From your shell, run:

```
t debug <node_name>
```

By default this looks for the pipeline in `src/pipeline.t`; if your pipeline lives elsewhere, pass the file first:

```
t debug src/my_custom_pipeline.t data_cleanup
```

T evaluates the script up to the requested node, resolves every upstream dependency, locates their latest materialized `/nix/store` paths, sets up the environment, and launches the runtime's interactive REPL. Targeting a Python node, for example, produces something like:

```
================================================
Debugging Node: Y (Runtime: Python)
================================================
Environment variables set for dependencies:
  - dataset_np =
 ↪  /nix/store/5fcfj6wfh...-pipeline_output/dataset_np

Starting interactive Python REPL...
Tip: Load upstream dependencies in Python using:
  import tlang
  dataset_np = tlang.read_node("dataset_np")
Press Ctrl+D or exit to return to T REPL.
================================================
```

10.8.2 `debug_node()` from the REPL

If you are already inside a T REPL session with a built pipeline, the same debugger is one call away:

```
debug_node(p.data_cleanup)
```

This spawns the same subshell environment. Inside it, a `tlang` companion library lets you load upstream node data live:

```
py> import tlang
py> df = tlang.read_node("raw_df")
py> df.dtypes
```

Now you can inspect the exact data that crashed your node, dry-run the offending function line by line, and confirm the fix. Press `Ctrl+D` (or `exit()`, or `q()` in R) to leave the subshell; control returns cleanly to your T session.

The subshells are customized so you always know where you are: Python starts with a `py>` prompt, R starts quietly (no welcome banner) with `r>`, and Julia shows `jl>`. Interactive debugging is supported for the three REPL-capable runtimes: Python, R, and Julia; debugging a Quarto or Bash node raises a descriptive `ValueError` instead of dropping you into a raw shell.

10.8.3 Keeping the Debug Environment Reproducible

The debugger runs inside the project environment declared in `tproject.toml`. If a runtime package needed to deserialize an upstream artifact is missing, the debugger will start without it. Use `t doctor` to spot missing packages and add them to the right section:

- `[r-dependencies].packages` for R packages

- `[py-dependencies].packages` for Python packages
- `[jl-dependencies].packages` for Julia packages

For example, a Julia node that reads a CSV ancestor needs `CSV` and `DataFrames`; one that reads Arrow or Parquet needs `Arrow` (or `Parquet2`) and `DataFrames`; a Python node reading Arrow data needs `pandas` and `pyarrow`. After editing `tproject.toml`, run `t update` and re-enter `nix develop` so the debug environment is rebuilt. Do not install these packages ad hoc with `pip`, `install.packages()`, or `Pkg.add()`: those commands are intercepted inside the debugger precisely to keep your debugging reproducible.

10.9 Polyglot Error Handling

The same rules apply to nodes written in R, Python, or Julia. When a foreign script raises a native exception, such as an R `stop()`, a Python `ValueError`, or a Julia `UndefVarError`, T captures it exactly as it captures a T error: the node is marked `Completed with error`, and `error_code`, `error_msg`, `collect_exceptions`, and the build log all work on it as before.

```
p2 = pipeline {
  data = node(
    command = to_dataframe([
      x = [1.0, 2.0, 3.0],
      y = [0L, 0L, 0L]
    ])
  )
  model = rn(
    command = <{
```

```
      data$y <- as.factor(data$y)
      glm(y ~ x, data = data, family = binomial(link
↪   = "logit"))
    }>,
    deserializer = ^ipc,
    serializer = ^pmml
  )
}
build_pipeline(p2)
```

Here the response variable has only one level, so the logistic regression fails inside the R sandbox. The build does not abort; `model` is a soft failure, and `error_msg(p2.model)` carries R's original message. And because `model` is an R node, `debug_node(p2.model)` drops you into an `r>` subshell with the exact `data` frame that caused the trouble.

The one polyglot-specific habit worth forming: keep error messages in your foreign code informative. A bare `stop()` in R or a bare `raise` in Python gives you an `Error` with little to go on; a message like `stop("no rows left after filtering")` is what `error_msg` will hand you later, and what you will thank yourself for at 2 a.m.

10.10 Common Errors

A few failures show up in nearly every pipeline. Knowing the idiomatic fix saves a lot of debugging:

- **Missing values**: `mean([1, 2, NA, 4])` raises an `NAError`. Pass `na_rm = true` when NAs are expected, or filter them deliberately.

- **Type mismatches**: `"Age: " + 25` raises a `TypeError` because T does not implicitly coerce. Convert explicitly with `to_string()` (or `str_join`).
- **Empty collections**: `mean([])` raises a `ValueError`. Check `length(data) == 0` first and return a default.
- **Division by zero**: guard the denominator, `if (count == 0) 0.0 else total / count`, or let it fail and recover downstream with `?|>` or `match`.
- **Missing columns**: `df.nonexistent` raises a `NameError`. Check `colnames(df)` before accessing, and raise a descriptive error of your own when a required column is absent.

10.11 Best Practices

- **Fail fast, fail explicitly.** Validate inputs early and raise descriptive errors (`error("ValidationError", "Empty dataset")`) at the boundary where you can say *why* it is invalid, rather than letting a cryptic failure surface ten nodes later.
- **Let soft failures stay soft.** In a long pipeline, prefer the default resilient build so one bad node does not hide three others; use `collect_exceptions` to review everything at once.
- **Keep recovery at the edges.** Use `|>` in the middle of your pipelines and concentrate `?|>`/`match` recovery where a fallback is genuinely meaningful.
- **Name your errors.** A good `error_msg` is a unit of documentation: it should tell the next reader what was expected and what they found.
- **Debug with real data.** When in doubt, `debug_node` beats a mock: the subshell gives you the exact upstream artifacts your node saw.

10.12 Summary

T treats errors as first-class values, and that single decision shapes everything in this chapter. `is_error`, `error_code`, `error_msg`, and `error_context` let you interrogate a failure; `|>` propagates errors untouched while `?|>` hands them to a recovery function; and `match` gives you declarative branching over error and success alike. At the pipeline level, resilient builds capture node failures as `Completed with error` without aborting, and `collect_exceptions`, `build_log`, and `inspect_node` turn any build, healthy or not, into data you can query. When the data is not enough, `t debug` and `debug_node` drop you into the failing node's own runtime, with its real inputs, ready to step through the code that broke.

Next, we turn to **distribution**: how to package all of this into containers so that others can run your pipelines without installing anything at all.

11 Containerisation With Docker

11.1 Introduction

Up until now, we've built reproducible environments with Nix, written testable code with FP principles, orchestrated pipelines with T, and made them robust with error handling and debugging. Now we turn to **distribution**: how do we share our work with others who may not have Nix installed?

Nix gives us fine-grained control over every package and dependency in our project, ensuring bit-for-bit reproducibility. However, when it comes to distributing a *data product*, another technology, Docker, is incredibly popular.

A common misconception is that Nix and Docker are alternatives, that you must choose one or the other. This is wrong. They are conceptually different tools that solve different problems. Nix is a **package manager and build system**: it answers "what software do I need and how do I build it reproducibly?" Docker is a **containerisation platform**: it answers "how do I package and run an application in isolation?" Understanding this distinction is key to using them together effectively.

While Nix manages dependencies for an application that runs on a host operating system, Docker takes a different approach: it

packages an application *along with* a lightweight operating system and all its dependencies into a single, portable unit called a **container**. This container can then run on any machine that has Docker installed, regardless of its underlying OS. Being familiar with both tools makes you a more versatile data scientist: you can use Nix for precise, reproducible development environments and Docker for distributing your work to colleagues, servers, or CI pipelines that may not have Nix installed.

11.1.1 Spatial vs Temporal Reproducibility

To understand when and why to use Docker, it helps to distinguish between two types of reproducibility:

- **Spatial reproducibility**: The ability to execute an analysis identically across different machines *right now*. Docker excels here: a container runs the same way on your laptop, a colleague's workstation, and a cloud server.
- **Temporal reproducibility**: The ability to execute an analysis identically *over time*. Docker is weaker here. The imperative commands in a `Dockerfile` (like `apt-get update`) are non-deterministic. Rebuilding the same file at different times can yield different images, a temporal drift that is a well-documented weakness of container-based environments (Malka, Zacchiroli, and Zimmermann 2024).

This distinction is crucial. Docker's true reproducibility promise is that a specific, pre-built image will always launch an identical container. It does *not* promise that building the same `Dockerfile` twice will yield an identical image.

As empirical research has shown, achieving deterministic builds requires systems designed for this purpose, like Nix's functional

model, where identical inputs always produce identical outputs. Studies rebuilding over 700,000 historical Nix packages from snapshots spanning several years found bit-for-bit reproducibility rates exceeding 90% (Malka, Zacchiroli, and Zimmermann 2025).

11.1.2 The Best of Both Worlds

This is why we combine Nix and Docker rather than choosing one or the other:

- **Nix** provides strong temporal reproducibility: the guarantee that an environment built today will be identical when rebuilt years later.
- **Docker** provides strong spatial reproducibility and universal distribution: the guarantee that a container runs the same everywhere Docker runs.

The pattern we'll use is simple: use Nix *inside* a Docker container, so that the container is both small and reproducible. Docker becomes a portable runtime for a Nix-managed environment, excellent for deployment to systems where Nix isn't installed. There are three ways to get Nix into the image:

1. **Install Nix inside the container** from a minimal base image. You add a few lines to the Dockerfile that install Nix, and everything after that is installed with Nix.
2. **Start from a base image that already has Nix.** You pull an existing image with Nix installed and use it as the foundation, skipping the installation step.
3. **Build the whole image with Nix.** You write the image definition in Nix itself, and Nix produces the container image for you.

This chapter covers all three, starting with the first two.

11.1.3 Interactive Development vs Distribution

This approach also clarifies *when* to use each tool:

- **Use Nix directly** for interactive development. It integrates seamlessly with your IDE and filesystem. No volume mounts, no port forwarding, no graphical application headaches.
- **Use Docker** for distributing finished data products. Package your T pipeline into an image that anyone can run with a single command.

If you've never heard of Docker before, this chapter will provide the basic knowledge required to get started. Let's start by watching this very short video[1] that introduces the core concepts.

The Rocker Project[2] provides a large collection of ready-to-use Docker images for R users.

11.2 Docker Essentials

11.2.1 Installing Docker

The first step is to install Docker. You'll find the instructions for Ubuntu here[3], for Windows here[4] (read the system requirements

[1] https://www.youtube.com/watch?v=-X3AeROGmOw
[2] https://rocker-project.org/
[3] https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository
[4] https://docs.docker.com/desktop/install/windows-install/

section as well!) and for macOS here[5] (make sure to choose the right version for the architecture of your Mac, if you have an M1 Mac use *Mac with Apple silicon*).

After installation, it might be a good idea to restart your computer, if the installation wizard does not invite you to do so. To check whether Docker was installed successfully, run the following command in a terminal:

```
docker run --rm hello-world
```

This should print the following message:

```
Hello from Docker!
This message shows that your installation appears to
be working correctly.

To generate this message, Docker took the following
steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image
 from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from
 that image which runs the
    executable that produces the output you are
    currently reading.
 4. The Docker daemon streamed that output to the
 Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an
Ubuntu container with:
```

[5] https://docs.docker.com/desktop/install/mac-install/

```
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a
free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

If you see this message, congratulations, you are ready to run Docker. If you see an error message about permissions, this means that something went wrong. If you're running Linux, make sure that your user is in the Docker group by running:

```
groups $USER
```

You should see your username and a list of groups that your user belongs to. If a group called `docker` is not listed, then you should add yourself to the group by following these steps[6].

11.2.2 The Rocker Project and Image Registries

When running a command like:

```
docker run --rm hello-world
```

what happens is that an image, in this case `hello-world`, gets pulled from a so-called *registry*. A registry is a storage and distribution system for Docker images. Think of it as a GitHub for Docker images, where you can push and pull images, much

[6] https://docs.docker.com/engine/install/linux-postinstall/

like you would with code repositories. The default public registry that Docker uses is called Docker Hub, but companies can also host their own private registries to store proprietary images.

Many open source projects build and distribute Docker images through Docker Hub, for example the Rocker Project.

The Rocker Project is instrumental for R users that want to use Docker. The project provides a large list of images that are ready to run with a single command. As an illustration, open a terminal and paste the following line:

```
docker run --rm -e PASSWORD=yourpassword -p
 ↪  8787:8787 rocker/rstudio
```

Once this stops running, go to `http://localhost:8787/` and enter `rstudio` as the username and `yourpassword` as the password. You should log in to a RStudio instance: this is the web interface of RStudio that allows you to work with R from a server. In this case, the *server* is the Docker container running the image. Yes, you've just pulled a Docker image containing Ubuntu with a fully working installation of RStudio web!

Let's open a new script and run the following lines:

```
data(mtcars)
summary(mtcars)
```

You can now stop the container (by pressing `CTRL-C` in the terminal). Let's now rerun the container… you should realise that your script is gone! This is the first lesson: whatever you do inside a container will disappear once the container is stopped. This also means that if you install the R packages that you need

while the container is running, you will need to reinstall them every time.

Thankfully, the Rocker Project provides a list of images with many packages already available. For example to run R with the `{tidyverse}` collection of packages already pre-installed, run:

```
docker run --rm -e PASSWORD=yourpassword -p
 ↪  8787:8787 rocker/tidyverse
```

11.2.3 Basic Docker Workflow

You already know about running containers using `docker run`. With the commands we ran before, your terminal will need to stay open, or else, the container will stop. Starting now, we will run Docker commands in the background using the `-d` flag (*d* as in *detach*):

```
docker run --rm -d -e PASSWORD=yourpassword -p
 ↪  8787:8787 rocker/tidyverse
```

You can run several containers in the background simultaneously. List running containers with `docker ps`:

```
docker ps
CONTAINER ID   IMAGE              COMMAND   CREATED
 ↪  STATUS         PORTS                    NAMES
c956fbeeebcb   rocker/tidyverse   "/init"   3
 ↪  minutes ago   Up 3 minutes
 ↪  0.0.0.0:8787->8787/tcp   elastic_morse
```

Stop the container using its ID:

```
docker stop c956fbeeebcb
```

Let's discuss the other flags:

- `--rm`: Removes the container once it's stopped
- `-e`: Provides environment variables to the container (e.g., `PASSWORD`)
- `-p`: Sets the port mapping (host:container)
- `--name`: Gives the container a custom name

Run a container with a name:

```
docker run -d --name my_r --rm -e
 ↪  PASSWORD=yourpassword -p 8787:8787
 ↪  rocker/tidyverse
```

You can now interact with this container using its name:

```
docker exec -ti my_r bash
```

You are now inside a terminal session, inside the running container! This can be useful for debugging purposes.

Finally, let's solve the issue of our scripts disappearing. Create a folder somewhere on your computer, then run:

```
docker run -d --name my_r --rm -e
 ↪  PASSWORD=yourpassword -p 8787:8787 \
  -v
   ↪  /path/to/your/local/folder:/home/rstudio/scripts:rw
   ↪  rocker/tidyverse
```

You should now be able to save scripts inside the `scripts/` folder from RStudio and they will appear in the folder you created on your host machine.

11.3 Making Our Own Images

To create our own images, you can start from an image provided by an open source project like Rocker, or you can start from the base Ubuntu image. Since we are using Nix to set up the reproducible development environment, we can use `ubuntu:latest`. Our development environment will always be exactly the same, thanks to Nix.

11.3.1 A Minimal Dockerfile with Nix

```
FROM ubuntu:latest

RUN apt update -y
RUN apt install curl -y

# Install Nix via Determinate Systems installer
RUN curl --proto '=https' --tlsv1.2 -sSf -L
 ↪  https://install.determinate.systems/nix | sh -s
 ↪  -- install linux \
  --extra-conf "sandbox = false" \
  --init none \
  --no-confirm

# Add Nix to the path
ENV PATH="${PATH}:/nix/var/nix/profiles/default/bin"
```

```
ENV user=root

# Configure rstats-on-nix cache
RUN mkdir -p /root/.config/nix && \
    echo "substituters = https://cache.nixos.org
     ↪ https://rstats-on-nix.cachix.org" >
     ↪ /root/.config/nix/nix.conf && \
    echo "trusted-public-keys =
     ↪ cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQF
     ↪ rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUr
     ↪ >> /root/.config/nix/nix.conf

# Copy the T project's Nix files
COPY flake.nix flake.lock tproject.toml .

# Build the environment from the flake
RUN nix build --accept-flake-config .

# Enter the flake development shell when the
 ↪ container starts
CMD ["nix", "develop", "--command", "bash"]
```

Every `Dockerfile` starts with a `FROM` statement specifying the base image. Then, every command starts with `RUN`. We install and configure Nix, copy the T project's Nix files, and build the environment from the flake. Finally, when the container starts, `nix develop` drops you into the project's development shell (the `CMD` statement). This is the flake-native command: unlike the legacy `nix-shell`, which looks for a `shell.nix` or `default.nix` file, `nix develop` uses your `flake.nix`, consistent with the `nix build --accept-flake-config .` step above. Note that it expects an interactive terminal, so run the container with `docker run -it`.

11.3.2 Splitting Into a Reusable Base Image

This image actually does two things:

- a first step which consists in setting up Nix inside Docker;
- a second step which consists in setting up our project-specific Nix development environment.

Because the first step is generic, we can split this into two stages. First, create a new Dockerfile in a separate directory, for example `nix-base/`, with a new Git repository so that you can commit and push it. Later in this chapter we will publish this image, and in a continuous integration setup you could build and publish it automatically:

```
# nix-base/Dockerfile
FROM ubuntu:latest AS nix-base

RUN apt update -y && apt install -y curl

# Install Nix via Determinate Systems installer
RUN curl --proto '=https' --tlsv1.2 -sSf -L
 ↪  https://install.determinate.systems/nix | sh -s
 ↪  -- install linux \
  --extra-conf "sandbox = false" \
  --init none \
  --no-confirm

ENV PATH="/nix/var/nix/profiles/default/bin:${PATH}"
ENV user=root

# Configure Nix binary cache
RUN mkdir -p /root/.config/nix && \
    echo "substituters = https://cache.nixos.org
     ↪  https://rstats-on-nix.cachix.org" >
     ↪  /root/.config/nix/nix.conf && \
```

```
    echo "trusted-public-keys =
     ↪  cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQF
     ↪  rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUr
     ↪  >> /root/.config/nix/nix.conf
```

Build and tag this image:

```
docker build -t nix-base:latest .
```

Now, for any project, simply reuse it:

```
FROM nix-base:latest

COPY flake.nix flake.lock tproject.toml .

RUN nix build --accept-flake-config .

CMD ["nix", "develop", "--command", "bash"]
```

The issue with this approach is that you've created a dependency between the two Dockerfiles that you need to manage. I'd only recommend the second approach if you can push the first image to a registry, either a public one or a private one from your company. Later in this chapter we will publish the first image.

In a T project, the environment is declared in `tproject.toml` rather than generated by a script. For this example:

```
[r-dependencies]
packages = ["dplyr", "ggplot2"]
```

```
[py-dependencies]
version = "python313"
packages = ["polars", "great-tables"]
```

Build and run:

```
docker build -t my-project .
docker run -it --rm --name my-project-container
 ↪  my-project
```

This drops you in an interactive Nix shell running inside Docker!

11.3.3 Making T Available in the Image

So far, our images contain Nix and the packages we asked Nix to install. If your project uses T, you also need the `t` executable inside the container. T is distributed exclusively through Nix and is never installed as a system program, so the natural approach is to build it from the T flake and put `t` on the `PATH`. Add a step to the Dockerfile:

```
RUN nix build --accept-flake-config
 ↪  github:b-rodrigues/tlang \
    && ln -s "$(readlink -f result)/bin/t"
     ↪  /usr/local/bin/t
```

`nix build` compiles (or fetches from the rstats-on-nix cache) the T package and leaves a `result` symlink pointing at the built store path. The `ln` command creates a permanent symlink from

`/usr/local/bin/t` to the `t` executable inside that store path. Because the symlink points at an absolute path in the Nix store, it survives across Docker layers, and `t` keeps working in the final image.

The `t` executable is a wrapper that already points at the R, Python, and Julia runtimes it needs. All of those runtimes live in the same Nix store, and Nix downloaded them when it built T, so the symlinked `t` runs without any extra setup.

For a fully reproducible build, pin the exact version of T that your project uses rather than the latest one on GitHub. A T project records its T version in `flake.lock`, so build that specific version:

```
RUN nix build --accept-flake-config
 ↪  github:b-rodrigues/tlang/v0.55.0 \
    && ln -s "$(readlink -f result)/bin/t"
     ↪  /usr/local/bin/t
```

Either way, `t` is available in the container without ever being installed as a system program.

11.4 Publishing Images on Docker Hub

If you want to share Docker images through Docker Hub, you first need to create a free account. A free account gives you unlimited public repositories.

List all images on your computer:

```
docker images
```

Log in to Docker Hub:

```
docker login
```

Tag the image with your username:

```
docker tag IMAGE_ID your_username/nix-base:latest
```

Push the image:

```
docker push your_username/nix-base:latest
```

This image can now be used as a stable base for other projects:

```
FROM your_username/nix-base:latest

RUN mkdir ...
```

11.4.1 Sharing Without Docker Hub

If you can't upload to Docker Hub, you can save the image to a file:

```
docker save nix-base | gzip > nix-base.tgz
```

Load it on another machine:

```
gzip -d nix-base.tgz
docker load < nix-base.tar
```

11.5 What If You Don't Use Nix?

Using Nix inside of Docker makes it very easy to set up an environment, but what if you can't use Nix for some reason? In this case, you would need to use other tools to install the right R or Python packages and it is likely going to be more difficult. The main issue you will face is missing development libraries.

For example, to install and use the R `{stringr}` package, you will need to first install `libicu-dev`:

```
FROM rocker/r-ver:4.5.1

RUN apt-get update && apt-get install -y \
    libglpk-dev \
    libxml2-dev \
    libcairo2-dev \
    libgit2-dev \
    libcurl4-openssl-dev \
    # ... many more
```

Another issue is that building the image is not a reproducible process, only running containers is. To mitigate this, use tagged images or better yet, a digest:

```
FROM
 ↪  rocker/r-ver@sha256:1dbe7a6718b7bd8630addc45a32731624fb7b
```

This will always pull exactly the same layers. However, at some point, that version of Ubuntu will be outdated. Using Nix, you can stay on `ubuntu:latest`.

11.6 Building Docker Images Directly with Nix

So far, we've used Nix *inside* Docker containers. But there's an even more powerful approach: using Nix to build Docker images directly, bypassing `Dockerfile`s entirely.

Nix provides `dockerTools`, a set of functions that can create OCI-compliant container images. Because these images are built through Nix's deterministic build system, they have stronger reproducibility guarantees than images built with `docker build`.

11.6.1 Why Skip the Dockerfile?

A `Dockerfile` is fundamentally imperative: it's a script of commands executed in order. Commands like `apt-get update` are non-deterministic by nature. Even with careful pinning, you're fighting the tool's design.

Nix's `dockerTools.buildImage` is declarative. You describe what should be in the image, and Nix figures out how to build it reproducibly. The resulting image is a pure function of its inputs.

11.6.2 A Basic Example

Here's a Nix expression that builds a Docker image containing R and some packages:

```
# docker-image.nix
let
  pkgs = import (fetchTarball

     ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/202
  ) {};

  # Define the R environment
  myR = pkgs.rWrapper.override {
    packages = with pkgs.rPackages; [
      dplyr
      ggplot2
      quarto
    ];
  };

in pkgs.dockerTools.buildImage {
  name = "my-r-env";
  tag = "latest";

  copyToRoot = pkgs.buildEnv {
    name = "image-root";
    paths = [ myR pkgs.quarto pkgs.coreutils
     ↪  pkgs.bash ];
    pathsToLink = [ "/bin" ];
  };

  config = {
    Cmd = [ "${pkgs.bash}/bin/bash" ];
```

```
    WorkingDir = "/work";
  };
}
```

Build and load it:

```
# Build the image (outputs a .tar.gz file)
nix build -f docker-image.nix

# Load into Docker
docker load < result

# Run it
docker run -it --rm my-r-env:latest
```

11.6.3 Using T with dockerTools

The previous example manually defines the R environment in Nix. If you prefer to let T define your environment (as we've done throughout this book), you can build the Docker image from your T project's flake.

First, make sure your `tproject.toml` declares the packages you need:

```
[r-dependencies]
packages = ["dplyr", "ggplot2", "quarto"]
```

Then run `t update` to generate the `flake.nix` and `flake.lock` files. Now create a `docker-image.nix` that builds the image from the project's flake:

```
# docker-image.nix
let
  pkgs = import (fetchTarball

    ↪ "https://github.com/rstats-on-nix/nixpkgs/archive/202
  ) {};

  # The T project is a Nix flake. Load it and take
   ↪ the packages from its
  # development shell as the image's build inputs.
  project  = builtins.getFlake (toString
   ↪ ./flake.nix);
  devShell = project.devShell;

in pkgs.dockerTools.buildImage {
  name = "my-t-project";
  tag = "latest";

  copyToRoot = pkgs.buildEnv {
    name = "image-root";
    paths = devShell.buildInputs ++ [ pkgs.coreutils
     ↪ pkgs.bash ];
    pathsToLink = [ "/bin" ];
  };

  config = {
    Cmd = [ "${pkgs.bash}/bin/bash" ];
    WorkingDir = "/work";
  };
}
```

This approach lets you keep your familiar T workflow for defining environments while gaining the reproducibility benefits

of `dockerTools` for image creation. Any changes to your `tproject.toml` → `flake.nix` will automatically propagate to the Docker image on the next build.

11.6.4 Benefits Over Dockerfiles

Aspect	Dockerfile	Nix dockerTools
Reproducibility	Imperative, non-deterministic	Declarative, deterministic
Layer caching	Based on command order	Based on content hashes
Image size	Often includes unnecessary packages	Minimal: only what you specify
Composition	Copy-paste between files	Reuse Nix expressions

11.6.5 Layered Images for Efficiency

For faster CI builds, you can create layered images where the base environment is cached:

```
pkgs.dockerTools.buildLayeredImage {
  name = "my-analysis";
  tag = "latest";

  contents = [ myR pkgs.quarto ];

  config = {
    Cmd = [ "${myR}/bin/R" "--vanilla" ];
```

```
  };
}
```

`buildLayeredImage` creates separate layers for each package, so unchanged dependencies are cached between builds.

11.6.6 When to Use This Approach

- **CI/CD pipelines**: When you need reproducible image builds as part of automated workflows
- **Minimal images**: When image size matters and you want only what you need
- **Complex environments**: When the environment is already defined in Nix and you want to deploy it as a container

For simpler use cases, the "Nix inside Docker" approach from earlier sections may be more accessible.

11.7 Dockerising a T Pipeline

We can package our entire T project into a single Docker image. This image can then be run by anyone with Docker installed, regardless of their host operating system or whether they have Nix.

Assume your project directory has:

```
.
  data/
    mtcars.csv
```

```
src/
    pipeline.t
    functions.R
    functions.py
tproject.toml
flake.nix
flake.lock
```

11.7.1 Step 1: The Dockerfile

```
FROM nix-base:latest
# Or: FROM your-username/nix-base:latest

WORKDIR /app

# Copy all project files
COPY . .

# Build the pipeline during image build.
# `nix develop` enters the project's devShell, which
 ↪  puts `t` on the PATH.
RUN nix develop --accept-flake-config --command t
 ↪  run src/pipeline.t

# Export the pipeline artifacts when the container
 ↪  runs
CMD ["bash", "-c", "cp -r _pipeline /output"]
```

11.7.2 Step 2: The Pipeline

The pipeline lives in `src/pipeline.t`. It loads the data in T and produces the result as a node artifact:

```
-- src/pipeline.t
p = pipeline {
  data = read_csv("data/mtcars.csv")

  mtcars_head = rn(
    command      = <{ head(data, 10) }>,
    deserializer = ^ipc,
    serializer   = ^json
  )
}

build_pipeline(p, verbose = 1)
```

When T builds the pipeline, every node's artifact is written to the `_pipeline/` directory. The `mtcars_head` node's result is stored there as JSON, ready to be exported.

11.7.3 Step 3: Build and Run

Build:

```
docker build -t my-reproducible-pipeline .
```

Run to get results:

```
mkdir -p ./output

docker run --rm --name my_pipeline_run \
  -v "$(pwd)/output":/output \
  my-reproducible-pipeline
```

Check your local `output` directory. You'll find the pipeline's artifacts, including the `mtcars_head` result, containing the exact, reproducible output of your pipeline.

You have successfully packaged a complex, polyglot pipeline into a simple, portable Docker image. This workflow combines the best of both worlds: Nix's power for creating reproducible builds and Docker's universal standard for distributing and running applications.

11.7.4 Alternative: Using dockerTools for T

You can also build your T pipeline image purely with Nix, avoiding Dockerfiles entirely. This gives you fully deterministic image builds.

Create a `docker-pipeline.nix`:

```
# docker-pipeline.nix
let
  pkgs = import (fetchTarball

    ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/2025-
  ) {};

  # The T project is a Nix flake. Load its
   ↪  development environment.
```

```
  project  = builtins.getFlake (toString
   ↪  ./flake.nix);
  devShell = project.devShell;

  # Build the pipeline as a derivation
  pipelineResult = pkgs.runCommand "pipeline-result"
   ↪  {
    buildInputs = devShell.buildInputs;
  } ''
    mkdir -p $out
    cd ${./. }
    t run src/pipeline.t
    # Copy results to output
    cp -r _pipeline $out/
  '';

in pkgs.dockerTools.buildImage {
  name = "my-t-pipeline";
  tag = "latest";

  copyToRoot = pkgs.buildEnv {
    name = "image-root";
    paths = [
      devShell.buildInputs
      pipelineResult
      pkgs.coreutils
      pkgs.bash
    ];
    pathsToLink = [ "/bin" "/" ];
  };

  config = {
    Cmd = [ "${pkgs.bash}/bin/bash" "-c" "cp -r
     ↪  /pipeline-result/* /output/" ];
```

```
    WorkingDir = "/work";
  };
}
```

Build and run:

```
nix build -f docker-pipeline.nix
docker load < result
docker run --rm -v "$(pwd)/output":/output
 ↪  my-t-pipeline:latest
```

This approach bakes the pipeline results directly into the image during the Nix build phase. The resulting image is minimal and contains only the outputs, not the full R/Python environment needed to produce them.

11.8 Summary

Docker and Nix complement each other:

- **Docker** provides a universal runtime and distribution mechanism
- **Nix** provides bit-for-bit reproducible environment management
- **Together** they enable truly reproducible, portable data products

Key Docker concepts:

- **Images** are templates; **containers** are running instances
- Containers are ephemeral; use **volumes** for persistence

- Use **registries** (Docker Hub) to share images
- The **Rocker Project** provides ready-made R images

For reproducibility:

- Use `ubuntu:latest` + Nix rather than pinning specific OS versions
- Build a reusable `nix-base` image to share across projects
- Get Nix into the image by installing it in the Dockerfile, starting from a base image that already has Nix, or building the whole image with Nix
- Build T from its flake and symlink `t` onto the `PATH`
- Package T pipelines for distribution

11.9 Further Reading

- Running Your R Script in Docker[7]
- Docker & R Reproducibility[8]
- R Docker Tutorial[9]

[7] https://www.r-bloggers.com/2019/02/running-your-r-script-in-docker/
[8] https://colinfay.me/docker-r-reproducibility/
[9] https://jsta.github.io/r-docker-tutorial/

12 A Short Intro to Packaging Your Code in R and Python

12.1 Introduction

In the previous chapter, we learned how to prove that our code works through unit testing and how to manage our collaboration with Git. We now have functions, tests, and version control. But there is one more step we can take to truly professionalise our workflow: packaging.

You might think, "I'm a data scientist, not a software engineer. Isn't this overkill?" The answer is a definitive no. Packaging your code, even for an internal analysis project, provides enormous benefits. Instead of copying and pasting your `clean_data()` function from project to project, you can simply `import mypackage` or `library(mypackage)` and use a single, trusted version. Sharing your work with a colleague becomes as simple as sending them a single command rather than emailing a zip file of scripts. Packaging also forces you into a standardised way of documenting your functions, provides a formal framework for running unit tests, and explicitly declares all of your dependencies.

This chapter will walk you through the process of creating a simple package in both R and Python. You will learn how to create, document, and test a basic R package using `{devtools}` and `{usethis}`, how to do the same for a modern Python package using `uv` and `pytest`, and how to install your own packages directly from GitHub so you can share your tools with colleagues and your future self. Finally, we will show how to integrate your packages into a T pipeline, turning them into a first-class part of your reproducible workflows. The goal is not to become an expert package developer, but to understand the structure and benefits so you can apply this powerful "packaging mindset" to all your future projects.

12.2 Part 1: Creating an R Package with `{usethis}` and `{devtools}`

The R community has developed an outstanding set of tools that make package development incredibly streamlined. The two essential packages are:

- `{devtools}`: provides core development tools like `install()`, `test()`, and `check()`.
- `{usethis}`: a workflow package that automates all the boilerplate. It creates files, sets up infrastructure, and guides you through the process.

Let's build a package called `cleanR`, which will contain a function to standardise column names. Create a folder called `cleanR` and `cd` into it.

12.2.1 Step 1: Project Setup

A T project is not only for pipelines: you can also use one purely as a reproducible development environment. Initialise a T project in this folder and declare the R packages you need for package development in `tproject.toml`:

```
[r-dependencies]
packages = ["devtools", "usethis", "roxygen2"]
```

Run `t update` to generate the Nix files, then enter the development shell with `nix develop`. Let `{usethis}` create the package structure for you. From your R console, run:

```
usethis::create_package("~/Documents/projects/cleanR")
```

This will create a new `cleanR` directory with all the necessary files and subdirectories. It will also open a new RStudio session for that project. The key components are:

- `R/`: this is where your R source code files will live.
- `DESCRIPTION`: a metadata file describing your package, its author, licence, and dependencies.
- `NAMESPACE`: a file that declares which functions your package exports for users and which functions it imports from other packages. You should never edit this file by hand. `{roxygen2}` will manage it for you.

12.2.2 Step 2: Write and Document a Function

Let's create our function. `{usethis}` helps with this too:

```
usethis::use_r("clean_names")
```

This creates a new file `R/clean_names.R` and opens it for editing. Let's add our function, including special comments for documentation. These `#'` comments are used by the `{roxygen2}` package to automatically generate the official documentation.

```
# In R/clean_names.R

#' Clean and Standardize Column Names
#'
#' This function takes a data frame and returns a
 ↪  new data frame with
#' cleaned-up column names (lowercase, with
 ↪  underscores instead of spaces
#' or periods).
#'
#' @param df A data frame.
#' @return A data frame with standardized column
 ↪  names.
#' @export
#' @examples
#' messy_df <- data.frame("First Name" = c("Ada",
 ↪  "Bob"), "Last.Name" = c("Lovelace", "Ross"))
#' clean_names(messy_df)
clean_names <- function(df) {
  old_names <- names(df)
  new_names <- tolower(old_names)
  new_names <- gsub("[ .]", "_", new_names)
  names(df) <- new_names
  return(df)
}
```

The key tags here are:

- `@param`: describes a function argument.
- `@return`: describes what the function returns.
- `@export`: this is crucial. It tells R that you want this function to be available to users when they load your package with `library(cleanR)`.
- `@examples`: provides runnable examples that will appear in the help file.

Now, run the magic command to process these comments:

```
devtools::document()
```

This updates the `NAMESPACE` file and creates the help file (`man/clean_names.Rd`). You can now see your function's help page with `?clean_names`.

12.2.3 Step 3: Add Unit Tests

A package without tests is a package waiting to break. `{usethis}` makes setting up tests trivial.

```
usethis::use_testthat() # Sets up the
 ↪  tests/testthat/ directory
usethis::use_test("clean_names") # Creates
 ↪  tests/testthat/test-clean_names.R
```

Now, edit the test file to add your expectations.

```
# In tests/testthat/test-clean_names.R
test_that("clean_names works with spaces and
 ↪  periods", {
  messy_df <- data.frame("First Name" = c("A"),
 ↪  "Last.Name" = c("B"))
  cleaned_df <- clean_names(messy_df)

  expected_names <- c("first_name", "last_name")

  expect_equal(names(cleaned_df), expected_names)
})

test_that("clean_names handles already clean names",
 ↪  {
  clean_df <- data.frame(a = 1, b = 2)
  # The function should not change anything
  expect_equal(names(clean_names(clean_df)), c("a",
   ↪  "b"))
})
```

To run all the tests for your package, use:

```
devtools::test()
```

12.2.4 Step 4: Check and Install

The final step before sharing is to run the official R CMD check, the gold standard for package quality. This command runs all tests, checks documentation, and looks for common problems.

```
devtools::check()
```

If your package passes with 0 errors, 0 warnings, and 0 notes, you are in great shape. If your package raises NOTEs or WARNINGs during the check phase, you can *most of the time* safely ignore these, especially if the package is only intended for internal usage. However, I would recommend that you still take care of the WARNINGs at the very least.

As a next step, you could edit the DESCRIPTION file. This is where you will list yourself as the package author, list the dependencies of the package and so on. I won't get into detail here, but learning how to edit the DESCRIPTION file is important for actual package development (especially listing dependencies is key).

To use your package within a project, the simplest way is to host it on GitHub or build a `.tar.gz` file and install it locally.

12.2.5 Step 5: Install from GitHub

If you can publicly host your package, hosting it on GitHub is a good way to easily share your code, and install the package in your projects without needing to publish it on CRAN.

1. Create a new, empty repository on GitHub (e.g., `cleanR`).

2. In your local project, follow the instructions GitHub provides to link your local repository and push your code. This usually involves commands like:

```
git remote add origin
 ↪  git@github.com:yourusername/cleanR.git
git branch -M main
git push -u origin main
```

3. Now, anyone (including you on a different machine) can install your package with a single command (**if you don't use Nix**):

```
# You might need to install {remotes} first
# install.packages("remotes")
remotes::install_github("yourusername/cleanR")
```

If you want to create an environment using Nix that includes this package, declare it in your `tproject.toml` as a Git reference, exactly as shown in Part 3, run `t update`, and re-enter the shell with `nix develop`.

Congratulations, you have created and shared a fully functional R package!

12.2.6 Step 5bis: Install it locally

If you can't share your package on GitHub, the alternative is to build it locally using:

```
devtools::build()
```

which will create a `.tar.gz` package. You can then install it with `devtools::install_local()` if you don't use Nix. If you do, the simplest approach is to push the package to a (possibly private) Git repository and declare it in your `tproject.toml` as in Part 3.

12.3 Part 2: Creating a Minimal Python Package with uv

There are many different ways to build packages in Python, and what I propose here is just one way to do it. While we will use Nix to manage our overall environment, we still need to define the metadata and structure for our Python package. We will use `uv`, an extremely fast and modern tool, for one specific purpose: initialising our project's configuration file. We will not use `uv` to manage a virtual environment, as Nix already handles that for us (unless you absolutely want to: however, you should then make sure that `uv` itself is being managed by Nix to ensure reproducibility).

Let's build a Python package called `pyclean`, the equivalent of our R package.

12.3.1 Step 1: Project Setup with uv

First, set up a T project that provides a Python environment. In `tproject.toml`, declare the Python packages you need and `uv` as an additional tool:

```
[py-dependencies]
version = "python313"
packages = ["pytest", "pandas"]

[additional-tools]
packages = ["uv"]
```

Run `t update` and enter the development shell with `nix develop`.

Then, create a directory for your new package and initialise it:

```
mkdir pyclean
cd pyclean
uv init --bare
```

The `--bare` flag is perfect for our Nix workflow. It creates only the essential `pyproject.toml` file without creating a virtual environment or extra directories. This leaves us with a clean slate.

Now, we must create the source and test directories manually. We'll use the standard `src` layout:

```
mkdir -p src/pyclean
mkdir tests
touch src/pyclean/__init__.py
```

Your project structure should now look like this (check it using the `tree` command):

```
pyclean/
   pyproject.toml
   src/
       pyclean/
           __init__.py
   tests/
```

12.3.2 Step 2: Write a Function and Declare Dependencies

Let's create our `clean_names` function inside a new file, `src/pyclean/formatters.py`.

```
# In src/pyclean/formatters.py
import pandas as pd

def clean_names(df: pd.DataFrame) -> pd.DataFrame:
    """Clean and standardize column names of a
    ↪ DataFrame.

    Args:
        df: The input pandas DataFrame.

    Returns:
        A pandas DataFrame with standardized column
↪   names.
    """
    new_df = df.copy()
    new_cols = {col: col.lower().replace(" ",
↪   "_").replace(".", "_") for col in
↪   new_df.columns}
    new_df = new_df.rename(columns=new_cols)
    return new_df
```

To make this function easily importable, we expose it in `src/pyclean/__init__.py`:

```
# In src/pyclean/__init__.py
from .formatters import clean_names

__all__ = ["clean_names"]
```

Next, we must declare our dependencies by manually editing `pyproject.toml`. We need `pandas` for our function and `pytest` for our tests.

```
# In pyproject.toml
[project]
name = "pyclean"
version = "0.1.0"
description = "A simple package to clean data."
dependencies = [
    "pandas>=2.0.0",
]

[project.optional-dependencies]
test = [
    "pytest",
]

[tool.pytest.ini_options]
pythonpath = [
  "src"
]
```

The `pythonpath = ["src"]` line is very important. Without it, you'd first need to install your `pyclean` library in editable mode using `pip` before running the tests. By adding this block, simply running `pytest` from the command line will work.

12.3.3 Step 3: Add Unit Tests

Create a new test file, `tests/test_formatters.py`, and add your tests.

```
# In tests/test_formatters.py
import pandas as pd
```

```
from pyclean import clean_names

def test_clean_names_happy_path():
    messy_df = pd.DataFrame({"First Name": ["Ada"],
 ↪  "Last.Name": ["Lovelace"]})
    cleaned_df = clean_names(messy_df)
    expected_cols = ["first_name", "last_name"]
    assert list(cleaned_df.columns) == expected_cols

def test_clean_names_is_idempotent():
    clean_df = pd.DataFrame({"first_name": ["a"],
 ↪  "last_name": ["b"]})
    still_clean_df = clean_names(clean_df)
    assert list(still_clean_df.columns) ==
     ↪  list(clean_df.columns)
```

Since your Nix environment provides all the tools, you can run tests directly from your terminal:

```
pytest
```

12.3.4 Step 4: Build and Install

To package your code, you need a build tool. It turns out that `uv` bundles a build tool with it, so we only need to call `uv build`:

```
# In your terminal, from the root of the 'pyclean'
 ↪  project
uv build
```

This creates a `dist/` directory containing a source distribution (`.tar.gz`) and a compiled wheel (`.whl`). The wheel is the modern standard for distribution.

Outside of a Nix shell, to use your package during development, you can install it in "editable" mode. This creates a link to your source code, so any changes you make are immediately reflected without needing to reinstall.

```
# Install the package and its test dependencies
pip install -e .[test]
```

But we are working from a Nix shell. Instead, we will edit the project's `flake.nix` to update the `PYTHONPATH` environment variable, so our package can easily be found. Open `flake.nix`, find the definition of the development shell, and add a `shellHook` attribute:

```
shellHook = ''
  export PYTHONPATH=$PWD/src:$PYTHONPATH
'';
```

The `shellHook` runs every time the shell starts. Note that `t update` regenerates `flake.nix`, so if you re-run it you will need to re-apply the hook.

With this, dropping into the shell with `nix develop`, starting the Python interpreter and then typing `import pyclean` will work without any issues. You may need to adapt the path depending on where you're developing the package.

12.3.5 Step 5: Install from GitHub

Sharing via GitHub is the most common way to distribute packages that aren't on the official Python Package Index (PyPI):

1. Create a new, empty repository on GitHub.
2. Push your local project to the remote repository.
3. Now, anyone can install your package directly from GitHub using `pip`, which is smart enough to find and process your `pyproject.toml` file: `bash pip install git+https://github.com/yourusername/pyclean.git`

For Nix environments, the cleanest approach is the one described in Part 3: declare `pyclean` as a direct Git reference in your project's Python workspace, lock it with `uv`, and run `t update`. This process is naturally more involved than simply calling `pip install`, but it has the advantage of being entirely reproducible.

12.4 Part 3: Integrating Your Packages into a T Pipeline

Throughout this book, your T pipelines have declared their runtime dependencies in `tproject.toml` (R packages under `[r-dependencies]` and Python packages under `[py-dependencies]`, Chapter 4) and you have built pipelines from those dependencies (Chapter 8). Here is the payoff of the packaging work you just did: you can now drop *your own* packages into that same mechanism and call them from any node.

12.4.1 Adding an R Package

For a package that is already on CRAN, list it in the `packages` array, exactly as in Chapter 4:

```
[r-dependencies]
packages = ["dplyr", "ggplot2"]
```

For a package you host on GitHub, such as the `cleanR` package from Part 1, declare it as a named entry with a `git` URL and a full commit `rev` (see the T project development guide, section 3.3.3):

```
[r-dependencies]
packages = ["dplyr"]
cleanR = { git =
 ↪  "https://github.com/yourusername/cleanR", rev =
 ↪  "abc123def456" }
```

The `rev` field must be the full 40-character commit hash of the version you want to use. After editing `tproject.toml`, run `t update` to regenerate `flake.nix`, then re-enter the shell with `nix develop`. The package is injected into every R node's `buildInputs`, so `library(cleanR)` works inside any `rn()` node.

12.4.2 Adding a Python Package

For a package on PyPI, the default Nix resolver is the simplest option:

```
[py-dependencies]
packages = ["pandas", "scikit-learn"]
```

If your package is only on GitHub, like `pyclean` from Part 2, and is not on PyPI, switch to the `uv` resolver (Chapter 4). This makes a `pyproject.toml` and a `uv.lock` file the source of truth for your Python dependencies:

```
[py-dependencies]
resolver = "uv"
workspace = "python"
```

Create the workspace directory and declare `pyclean` as a direct Git reference:

```
mkdir python
```

```
# python/pyproject.toml
[project]
name = "my_project_python_env"
version = "0.1.0"
requires-python = ">=3.13,<3.14"
dependencies = [
    "pandas",
    "pyclean @
     ↪  git+https://github.com/yourusername/pyclean.git",
]
```

Generate the lock file and regenerate the flake (run these from the Nix shell where `uv` is available):

```
uv lock --project python
t update
```

Commit `python/pyproject.toml`, `python/uv.lock`, and `tproject.toml` together. From then on, `import pyclean` works inside any `pyn()` node. To add or remove a dependency later, edit `dependencies` in `python/pyproject.toml`, re-run `uv lock --project python` and `t update`, and commit the updated files.

12.5 Conclusion: The Packaging Mindset in the Age of AI

You have now successfully created, tested, documented, and shared a basic package in both R and Python. While there is much more to learn about advanced package development, you have already mastered the most important part: the packaging mindset.

From now on, when you start a new analysis project, think of it as a small, internal package.

- Put your reusable logic into functions.
- Place those functions in the `R/` or `mypackage/` source directory.
- Document them.
- Write a few simple tests to prove they work.
- Manage dependencies formally in `DESCRIPTION` or `pyproject.toml`.

Adopting this structure will make your work more robust, easier to share, and fundamentally more reproducible. It is the bridge between writing one-off scripts and building reliable, professional data science tools.

This packaging mindset becomes even more powerful when you introduce a modern collaborator: the LLM. The structured, component-based nature of a package is the perfect way to interact with AI assistants.

A package provides a clear contract and a well-defined structure that LLMs thrive on. Instead of a vague prompt like, "Refactor my messy analysis script," you can now make precise, targeted requests:

- "Here is my function `clean_names`. Please write three `pytest` unit tests for it, including one for the happy path, one for an empty DataFrame, and one for names that are already clean."
- "Generate the roxygen2 documentation skeleton for this R function, including `@param`, `@return`, and `@examples` tags."
- "I need a function in my `pyclean/utils.py` module that calculates the Z-score for a pandas Series. Please generate the function and its docstring."

This synergy is a two-way street. Not only does the structure help you write better prompts, but LLMs excel at generating the very boilerplate that makes packaging robust. Tedious tasks like writing standard documentation headers, creating skeleton unit test files, or even generating a first draft of a function based on a clear description become near-instantaneous.

This elevates your role from a writer of code to an architect and a reviewer. Your job is to design the components (the functions), prompt the LLM to generate the implementation, and then, most

critically, use the testing framework you just built to rigorously verify that the AI-generated code is correct, efficient, and robust. You are the final authority, and the package structure gives you the tools to enforce quality control.

By combining the discipline of packaging with the power of LLMs, you lower the barrier to adopting best practices like comprehensive testing and documentation. This combination doesn't just make you faster; it makes you a more reliable and professional data scientist, capable of producing tools that are truly reproducible and built to last.

While a full guide to package development is beyond the scope of this course, it is the natural next step in your journey as a data scientist who produces reliable tools. When you are ready to take that step, here are the definitive resources to guide you:

- For R: the "R Packages" (2e) book by Hadley Wickham and Jennifer Bryan is the essential, comprehensive guide. It covers everything from initial setup with `{usethis}` to testing, documentation, and submission to CRAN. Read it online here.[1]
- For Python: the official Python Packaging User Guide[2] is the place to start. For a more modern and streamlined approach that handles dependency management and publishing, many developers use tools like Poetry[3] or Hatch[4].

Treating your data analysis project like a small, internal software package, complete with functions and tests, is a powerful mindset that will elevate the quality and reliability of your work.

[1] https://r-pkgs.org/
[2] https://packaging.python.org/en/latest/tutorials/packaging-projects/
[3] https://python-poetry.org/docs/
[4] https://hatch.pypa.io/latest/

13 Continuous Integration with GitHub Actions

13.1 Introduction

We are almost at the end of our journey. In the previous chapters, we built reproducible environments with Nix, organised our code into pure functions, made them robust with error handling and debugging, proved correctness with unit tests, managed our collaboration with Git, and bundled everything into shareable packages. We can now run our pipelines in a 100% reproducible way.

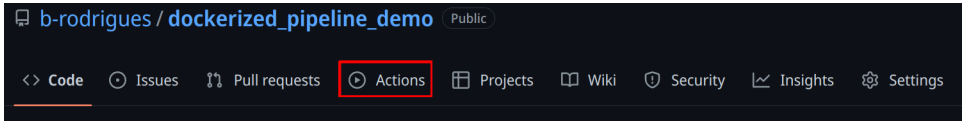
However, all of this still requires manual steps. And maybe that's not a problem; if your environment is set up and users only need to drop into a Nix shell and run the pipeline, that's already a huge improvement. But you should keep in mind that manual steps don't scale. Imagine you are part of a team that needs to quickly ship products to clients. Several people contribute to the product, and you might need to work on multiple projects in the same day. You and your teammates should be focusing on writing code, not on repetitive tasks like building images or running tests. Ideally, we would want to automate these steps. That is what we are going to learn in this chapter.

This chapter will introduce you to **Continuous Integration (CI)** with GitHub Actions. You will learn how to set up workflows that automatically run your tests when you push code, how to build Docker images and recover artifacts, and how to run your pipelines directly from GitHub's servers. Because we're using Git to trigger all the events and automate the whole pipeline, this approach is sometimes called GitOps.

You may have heard the term "CI/CD," where CD stands for Continuous Deployment or Continuous Delivery. We will focus on CI in this chapter. Continuous Deployment (automatically pushing results to a database, dashboard, or API) is highly specific to your organisation and infrastructure. What we cover here, however, gives you the foundation: once your pipeline runs reliably on CI, the "deployment" step is just one more workflow job pointing to wherever your results need to go.

13.2 Getting your repo ready for GitHub Actions

Obviously, you should use a project that is versioned on GitHub. Use the package we've developed previously. If you go on its GitHub page, you should see an "Actions" tab on top:



This will open a new view where you can select a lot of available, ready to use actions. "Actions" are premade scripts that execute some commands you might need: such as setting up R, Python, running tests, etc. Since we're using Nix, we don't really need to look for any actions to set up our environments. However, we

might want to use some pre-made actions to upload artifacts for instance.

To actually configure our repository to run actions, we need to edit a file in our project under the `.github/workflows` directory (create them if needed). In it, write a `yaml` file called `hello.yaml` and write the following in it:

```yaml
name: Hello world
on: [push]
jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello from GitHub Actions!"
      - run: echo "This command is running from an
 ↪  Ubuntu VM each time you push."
```

Let's study this workflow definition line by line:

`name: Hello world`

Simply gives a name to the workflow.

`on: [push]`

When should this workflow be triggered? Here, whenever something gets pushed.

`jobs:`

What is the actual things that should happen? This defines a list of actions.

`  say-hello:`

This defines the `say-hello` job.

```
runs-on: ubuntu-latest
```

This job should run on an Ubuntu VM. You can also run jobs on Windows or macOS VMs, but this uses more compute minutes than a Linux VM (which doesn't matter for public projects; for private projects, the amount of compute minutes is limited).

```
steps:
```

What are the different steps of the job?

```
  - run: echo "Hello from GitHub Actions!"
```

First, run the command `echo "Hello from GitHub Actions!"`. This command runs inside the VM. Then, run this next command:

```
  - run: echo "This command is running from an
  Ubuntu VM each time you push."
```

If we take a look at the commit we just pushed, on GitHub, we see this yellow dot next to the commit name. This means that an action is running. We can then take a look at the output of the job, and see that our commands, defined with the `run` statements in the workflow file, succeeded and echoed what we asked them.

13.3 Nix and GitHub Actions

To set up Nix on GitHub Actions you can use several steps (create a new file called `run-tests.yaml`):

```
- name: Install Nix
  uses: cachix/install-nix-action@v31
  with:
    nix_path:
 ↪  nixpkgs=https://github.com/rstats-on-nix/nixpkgs/archive/

- name: Setup Cachix
  uses: cachix/cachix-action@v15
  with:
    name: rstats-on-nix
```

If your repository contains a `flake.nix` and `flake.lock` (as a T project does), the same environment you've been using locally can be used on GitHub Actions just as easily. The committed flake pins the exact T version and every dependency, so there is nothing to generate: `nix develop` reads it directly:

```
- name: Run in the project environment
  run: nix develop --command t --version
```

You can then use the shell to run whatever you need. For example, if you're developing a package, you could run unit tests on each push:

```
- name: devtools::test() via nix develop
  run: nix develop --command Rscript -e
 ↪  "devtools::test(stop_on_failure = TRUE)"
```

`stop_on_failure = TRUE` is needed to make the step fail if there's an error, otherwise, the step would run successfully, even with failing tests.

Of course, if you're developing a Python package, use `nix develop --command pytest` instead to run the tests.

I highly recommend you run tests when pull requests get opened:

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

This will ensure that if someone contributes to your project, you know immediately if what they did breaks tests or not. If it does, ask them to fix the code until tests pass.

13.4 Running a T Pipeline on GitHub Actions

Now that we know how to set up Nix on GitHub Actions, running a T pipeline is straightforward. Recall from Chapter 4 that T is distributed exclusively via Nix: you never install it in the traditional sense. A T project pins the exact version of T it uses in its committed `flake.nix` and `flake.lock` files, and `nix develop` drops you into a shell where that pinned `t` executable is available alongside all of your R, Python, and Julia runtimes.

That means "installing T" on CI is really just "installing Nix." Once Nix is present, the project's own flake takes care of the rest. Here is a complete workflow that runs our pipeline on every push and pull request to `main`:

```
name: T Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Nix
        uses: cachix/install-nix-action@v31
        with:
          extra_nix_config: |
            experimental-features = nix-command
 ↪  flakes
            substituters = https://cache.nixos.org
 ↪  https://rstats-on-nix.cachix.org
            trusted-public-keys =
 ↪  cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcD
 ↪  rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUrhi1Jz

      - name: Setup Cachix
        uses: cachix/cachix-action@v15
        with:
          name: rstats-on-nix

      - name: Run pipeline
        run: nix develop --command t run --failfast
 ↪  src/pipeline.t
```

The `extra_nix_config` block enables Nix flakes and points Nix at the `rstats-on-nix` binary cache, so R packages are downloaded pre-built rather than compiled from source. The `Setup Cachix` step registers that cache so the runner can pull from it.

The important line is the last one:

```
run: nix develop --command t run --failfast
src/pipeline.t
```

`nix develop` enters the project's environment using the committed `flake.lock`, which is where the T version is pinned. We then run `t run` on `src/pipeline.t`, exactly as we would locally. The `--failfast` flag makes the step, and therefore the whole workflow, fail as soon as any node errors, which is what you want on CI: a red build should mean something is actually broken.

A couple of notes:

- If you have changed `tproject.toml` (added a package, bumped a version), run `t update` first to regenerate `flake.nix` and `flake.lock`, then commit those files, the same one-time sync you do locally.
- Chapter 8 showed the `pipeline_to_ga()` function, which generates a workflow like this one automatically, including artifact caching. Writing it by hand, as here, just makes each moving part explicit.

13.4.1 Caching T pipeline outputs between runs

Nix already caches builds in its store, but a fresh CI runner starts with an empty store, so every node is rebuilt from scratch on the

first run. T has native functions for shipping a pipeline's cached artifacts between runs, so a second run can skip the work a first run already did.

`export_artifacts(p, archive_path)` bundles the cached Nix artifacts of every node in `p` into a single portable archive file. All nodes must already exist in the local store, so you call it right after a successful build:

```
build_pipeline(p)
export_artifacts(p, "artifacts.tar")
```

On the next run, `import_artifacts()` restores those artifacts into the local Nix store before you build, so unchanged nodes are pulled from the archive instead of rebuilt:

```
import_artifacts("artifacts.tar")
build_pipeline(p)
```

The two-argument form, `import_artifacts(p, "artifacts.tar")`, additionally verifies that the imported store paths match the pipeline's expected signatures, catching a stale or mismatched archive.

To see what an archive contains without touching the local store, use `inspect_artifacts()`. It loads the archive into a temporary store and returns a data frame with one row per node:

```
inspect_artifacts("artifacts.tar")
-- columns: node, store_path, hash, size_bytes,
 ↪  references
```

On GitHub Actions the pattern is to export the archive after a successful build and upload it as a workflow artifact, then download and import it at the start of the next run. Combined with the `rstats-on-nix` binary cache above, this keeps even a cold runner from recompiling anything that has not changed.

13.5 Running a dockerized workflow

This next example can be found in this repository[1]. This example doesn't use Nix or T, but the point here is to show how a Docker container can be executed on GitHub Actions, and artifacts can be recovered. The process is always the same, regardless of what is inside the Docker image. If you want to follow along, fork this repository.

This is what our workflow file looks like:

```
name: Reproducible pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:

  build:

    runs-on: ubuntu-latest
```

[1] https://github.com/b-rodrigues/dockerized_pipeline_demo

```
    steps:
    - uses: actions/checkout@v5
    - name: Build the Docker image
      run: docker build -t my-image-name .
    - name: Docker Run Action
      run: docker run --rm --name
↪   my_pipeline_container -v
↪   /github/workspace/fig/:/home/graphs/:rw
↪   my-image-name
    - uses: actions/upload-artifact@v4
      with:
        name: my-figures
        path: /github/workspace/fig/
```

For now, let's focus on the `run` statements, because these should be familiar:

```
run: docker build -t my-image-name .
```

and:

```
run: docker run --rm --name my_pipeline_container -v
/github/workspace/fig/:/home/graphs/:rw
my-image-name
```

The only new thing here, is that the path has been changed to `/github/workspace/`. This is the home directory of your repository, so to speak. Now there's the `uses` keyword that's new:

```
uses: actions/checkout@v5
```

This action checks out your repository inside the VM, so the files in the repo are available inside the VM. Then, there's this action here:

```
- uses: actions/upload-artifact@v4
  with:
    name: my-figures
    path: /github/workspace/fig/
```

This action takes what's inside `/github/workspace/fig/` (which will be the output of our pipeline) and makes the contents available as so-called "artifacts". Artifacts are the outputs of your workflow. In our case, as stated, the output of the pipeline. So let's run this by pushing a change, and let's take a look at these artifacts!

After the action is done running, you will be able to download a zip file containing the plots. It is thus possible to rerun our workflow in the cloud. This has the advantage that we can now focus on simply changing the code, and not have to bother with boring manual steps. For example, let's change this target in the `_targets.R` file:

```
tar_target(
  commune_data,
  clean_unemp(
    unemp_data,
    place_name_of_interest = c(
      "Luxembourg", "Dippach",
      "Wiltz", "Esch/Alzette",
      "Mersch", "Dudelange"),
    col_of_interest = active_population)
)
```

I've added "Dudelange" to the list of communes to plot. Pushing this change to GitHub triggers the action we've defined before. The plots (artifacts) get refreshed, and we can download

them. Take a look and see that Dudelange was added in the `communes.png` plot!

It is also possible to "deploy" the plots directly to another branch, and do much, much more. I just wanted to give you a little taste of GitHub Actions (and more generally GitOps). The possibilities are virtually limitless, and I still can't get over the fact that GitHub Actions is free for public repositories.

13.6 Building a Docker image and pushing it to a registry

It is also possible to build a Docker image and have it made available on an image registry. You can see how this works on this repository[2]. These images can then be used as a base for other reproducible pipelines, as in this repository[3]. Why do this? Well because of "separation of concerns". You could have a repository which builds an image containing your development environment: this could be an image with a specific version of R and R packages built with Nix. And then have as many repositories as projects that run pipelines using that development environment image as a basis. Simply add the project-specific packages that you need for each project.

[2]https://github.com/b-rodrigues/ga_demo
[3]https://github.com/b-rodrigues/ga_demo_rap/tree/main

13.7 A Complete T Pipeline Workflow on GitHub Actions

The previous section showed the essential workflow in a few lines. Here is a complete, annotated version that also inspects the built pipeline and reads a node's result. Because Nix handles all dependencies reproducibly, you don't need Docker as an intermediary. The tlang[4] repository contains several complete examples; here we will walk through the key steps.

The workflow triggers on pushes and pull requests to `main`:

```
on:
  pull_request:
    branches: [main, master]
  push:
    branches: [main, master]
```

After checking out the repository and installing Nix (with Cachix for faster builds), there is no environment-generation step. A T project commits its `flake.nix` and `flake.lock`, which pin the exact T version and every dependency, and the pipeline itself is the committed `src/pipeline.t` file. `nix develop` reads the flake directly, so nothing needs to be generated.

The core step builds the pipeline exactly as you would locally:

```
- name: Build pipeline
  run: nix develop --command t run src/pipeline.t
```

[4] https://github.com/b-rodrigues/tlang

You can validate the pipeline's structure with `t check`, which inspects the pipeline without running any Nix builds:

```
- name: Check pipeline structure
  run: nix develop --command t check src/pipeline.t
```

Finally, read a node's result to confirm the pipeline produced what you expect:

```
- name: Show result
  run: nix develop --command t run --expr
  ↪  'read_node("confusion_matrix")'
```

13.7.1 Caching Pipeline Outputs Between Runs

While Nix caches derivations, CI runners are ephemeral: each run starts fresh. To avoid rebuilding the entire pipeline every time, T provides `export_artifacts()` and `import_artifacts()` to persist outputs between runs.

Before building, check if cached outputs exist and import them:

```
- name: Import cached outputs if available
  run: |
    if [ -f
  ↪  "../outputs/my_pipeline/pipeline_outputs.tar" ];
  ↪  then
      nix develop --command t import_artifacts
  ↪  src/pipeline.t
  ↪  ../outputs/my_pipeline/pipeline_outputs.tar
    else
```

```
      echo "No cached outputs found, will build from
↪   scratch"
    fi
```

After building, export the outputs so they can be reused:

```
- name: Export outputs to avoid rebuild
  run: |
    mkdir -p ../outputs/my_pipeline
    nix develop --command t export_artifacts
↪   src/pipeline.t
↪   ../outputs/my_pipeline/pipeline_outputs.tar
```

Finally, commit the cached outputs back to the repository:

```
- name: Push cached outputs
  run: |
    cd ..
    git config --global user.name "GitHub Actions"
    git config --global user.email
↪   "actions@github.com"
    git pull --rebase --autostash origin main
    git add outputs/my_pipeline/pipeline_outputs.tar
    if git diff --cached --quiet; then
      echo "No changes to commit."
    else
      git commit -m "Update cached pipeline outputs"
      git push origin main
    fi
```

This pattern ensures that only changed nodes are rebuilt on subsequent runs.

13.7.2 The Easy Way: `pipeline_to_ga()`

If the above seems like a lot of boilerplate, T provides a helper function that generates a complete GitHub Actions workflow for you:

```
pipeline_to_ga(p, file =
 ↪  ".github/workflows/pipeline.yml")
```

This creates a `.github/workflows/pipeline.yml` file that handles everything: installing Nix, setting up Cachix, building the pipeline, and importing and exporting artifacts. It stores the cached Nix store in a dedicated `t-runs` branch, keeping your main branch clean while persisting the cached outputs between runs.

For most projects, running `pipeline_to_ga()` once and committing the generated workflow file is all you need to get your pipeline running on CI.

13.8 GitHub Actions without Nix

If you're not using Nix, you'll have to set up GitHub Actions manually. Suppose you have a package project and want to run unit tests on each push. See for example the `{myPackage}` package, in particular this file[5]. This action runs on each push and pull request on Windows, Ubuntu and macOS:

[5] https://github.com/b-rodrigues/myPackage/blob/main/.github/workflows/rcmdcheck.yaml

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  rcmdcheck:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest,
        macos-latest]
```

Several steps are executed, all using pre-defined actions from the `r-lib` project:

```
    steps:
    - uses: actions/checkout@v4
    - uses: r-lib/actions/setup-r@v2
    - uses: r-lib/actions/setup-r-dependencies@v2
      with:
        extra-packages: any::rcmdcheck
        needs: check
    - uses: r-lib/actions/check-r-package@v2
```

An action such as `r-lib/actions/setup-r@v2` will install R on any of the supported operating systems without requiring any configuration from you. If you didn't use such an action, you would need to define three separate actions: one that would be executed on Windows, on Ubuntu and on macOS. Each of these operating-specific actions would install R in their operating-specific way.

Check out the workflow results to see how the package could be improved here[6].

Here again, using Nix simplifies this process immensely. Look at this workflow file from T's repository here[7]. Setting up the environment is much easier, as is running the actual test suite.

13.9 Advanced patterns

Now that you understand the basics, let's look at some more advanced patterns that will make your CI workflows more efficient and informative.

13.9.1 Caching with Cachix

Building Nix environments from scratch on every CI run can be slow. Cachix solves this by providing a binary cache for your Nix derivations. Once you build something, subsequent runs can download the pre-built binaries instead of rebuilding from source.

To use Cachix, you first need to create a free account at cachix.org[8] and create a cache. Then, generate an auth token and add it as a secret in your GitHub repository settings (under Settings → Secrets and variables → Actions). Call it something like `CACHIX_AUTH`.

Here is a workflow that builds your development environment and pushes the results to your Cachix cache:

[6] https://github.com/b-rodrigues/myPackage/actions/runs/12348361696
[7] https://github.com/b-rodrigues/tlang/blob/main/.github/workflows/unit-tests.yaml
[8] https://cachix.org

```yaml
name: Update Cachix cache

on:
  push:
    branches: [main]

jobs:
  build-and-cache:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Nix
        uses:
 ↪  DeterminateSystems/nix-installer-action@main

      - uses: cachix/cachix-action@v15
        with:
          name: your-cache-name
          authToken: '${{ secrets.CACHIX_AUTH }}'

       - name: Build and push to cache
         run: |
           nix build .#devShell
           nix-store -qR --include-outputs $(nix
 ↪  eval --raw .#devShell) | cachix push
 ↪  your-cache-name
```

The key line here is the `nix-store` command at the end. It queries all the dependencies of your build and pushes them to Cachix. The next time you or anyone else runs this workflow, the `cachix/cachix-action` will automatically pull from your cache, dramatically speeding up the build.

If you want to build on both Linux and macOS (since Nix binaries are platform-specific), you can use a matrix:

```
jobs:
  build-and-cache:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest]
```

13.9.2 Storing outputs in orphan branches

When running a pipeline on CI, you often want to keep the outputs (plots, data, reports) without committing them to your main branch. A clean solution is to store them in an orphan branch. An orphan branch has no commit history and is completely separate from your main code.

Here is the pattern:

```
- name: Check if outputs branch exists
  id: branch-exists
  run: git ls-remote --exit-code --heads origin
  ↪  pipeline-outputs
  continue-on-error: true

- name: Create orphan branch if needed
  if: steps.branch-exists.outcome != 'success'
  run: |
    git checkout --orphan pipeline-outputs
    git rm -rf .
    echo "Pipeline outputs" > README.md
```

```
    git add README.md
    git commit -m "Initial commit"
    git push origin pipeline-outputs
    git checkout -

- name: Push outputs to branch
  run: |
    git config --local user.name "GitHub Actions"
    git config --local user.email
 ↪  "actions@github.com"
    git fetch origin pipeline-outputs
    git worktree add ./outputs pipeline-outputs
    cp -r _outputs/* ./outputs/
    cd outputs
    git add .
    git commit -m "Update outputs" || echo "No
 ↪  changes"
    git push origin pipeline-outputs
```

This pattern first checks if the branch exists using `git ls-remote`. If not, it creates an orphan branch. Then it uses `git worktree` to work with both branches simultaneously, copies the outputs, and pushes them. T uses this pattern to store pipeline outputs between runs.

13.9.3 Creating workflow summaries

GitHub Actions has a built-in feature for creating rich summaries that appear directly on the workflow run page. You write Markdown to a special file path stored in the `GITHUB_STEP_SUMMARY` environment variable.

```
- name: Create summary
  run: |
    echo "## Pipeline Results  " >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "" >> $GITHUB_STEP_SUMMARY
    echo "| Metric | Value |" >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "|--------|-------|" >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "| Tests passed | 42 |" >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "| Coverage | 87% |" >>
 ↪  $GITHUB_STEP_SUMMARY
```

You can also generate the summary dynamically from your R or Python code:

```
- name: Generate summary from R
  run: |
    nix develop --command Rscript -e '
      results <- readRDS(\"results.rds\")
      cat(\"## Analysis Complete\n\n\", file =
 ↪  Sys.getenv(\"GITHUB_STEP_SUMMARY\"), append =
 ↪  TRUE)
      cat(paste(\"Processed\", nrow(results),
 ↪  \"observations\n\"), file =
 ↪  Sys.getenv(\"GITHUB_STEP_SUMMARY\"), append =
 ↪  TRUE)
    '"
```

This is particularly useful for:

- Showing test results at a glance

- Displaying key metrics from your analysis
- Providing download links to artifacts
- Reporting any warnings or issues

The summary appears right on the Actions tab, making it easy for collaborators to see what happened without digging through logs.

13.10 Conclusion

This chapter introduced Continuous Integration with GitHub Actions, the final piece of our reproducible workflow.

Key takeaways:

- **Automation removes manual steps**: Every push triggers tests, builds, and deployments without human intervention
- **Nix simplifies CI setup**: The same `flake.nix` you use locally works on GitHub Actions, eliminating "works on my machine" problems
- **Cachix speeds up builds**: By caching Nix derivations, subsequent runs avoid rebuilding unchanged dependencies
- **`pipeline_to_ga()` handles the boilerplate**: One function call generates a complete workflow for running T pipelines on CI
- **Artifact caching persists outputs**: Using `export_artifacts()` and an orphan branch, pipeline outputs survive between ephemeral CI runs

With continuous integration in place, your reproducible analytical pipeline is truly automated. Push your code, and GitHub takes care of the rest: running tests, building your environment,

executing your pipeline, and storing the results. This frees you to focus on what matters: the analysis itself.

14 T for Data Manipulation

So far, when a pipeline node needed to wrangle data, you have reached for R and `{dplyr}`. That is a perfectly good habit: `{dplyr}` is mature, and Chapter 5 showed you exactly how to call it from a node. But it is not the only option.

Remember the promise we made in Chapter 5: T ships with its own data manipulation verbs, inspired by `{dplyr}` and `{tidyr}`. This chapter keeps it. Those verbs live in T's `colcraft` package. They take a DataFrame as their first argument and return a new DataFrame, which makes them composable with the pipe operator you already know.

The point is not to replace R. It is to give you a reason to stay in T for the quick stuff: a filter here, a column there, a summary you need before you hand the data to a model. When the work gets heavy, you still have R and Python. But for the small transformations that glue a pipeline together, T is often enough, and staying in one language means one fewer serializer in the loop.

Every example in this chapter is written in T and marked so that it does not execute during the build. You can paste them into a REPL or a node and run them for real.

14.1 The core verbs

The `colcraft` verbs follow the shape you already know from `{dplyr}`: a verb takes a DataFrame, does one thing, and hands back a DataFrame. You chain them with the pipe.

14.1.1 select(): choose columns

`select()` keeps only the columns you name, using the `$column` syntax:

```
df |> select($name, $age)
df |> select($name)
```

If you name a column that does not exist, you get a `KeyError` that names the missing column, so a typo fails fast rather than silently dropping data.

14.1.2 filter(): choose rows

`filter()` keeps the rows where a predicate is true. You write the predicate against the `$column` syntax, and T turns it into a row-wise function for you:

```
df |> filter($age > 30)
df |> filter($dept == "eng")
```

To combine conditions, use the logical operators `&&` (and) and `||` (or):

```
df |> filter($dept == "eng" && $age > 25)
df |> filter($dept == "sales" || $score > 90)
```

> **No silent magic.** The bitwise operators `&` and `|` are for scalar, element-wise work. If you accidentally use them to combine filter conditions, T raises a `TypeError` that points you at `&&` and `||`.

14.1.3 mutate(): add or transform columns

`mutate()` adds a new column or replaces an existing one. The new column is named with `$` and defined by an expression over the existing columns:

```
df |> mutate($age_plus_10 = $age + 10)
df |> mutate($bonus = $salary * 0.1)
```

14.1.4 arrange(): sort rows

`arrange()` sorts by one or more columns, in ascending order by default or `"desc"` for descending:

```
df |> arrange($age)
df |> arrange($score, "desc")
```

14.1.5 group_by() and summarize()

`group_by()` marks the columns that define groups. It does not change the data; it tells the next verb how to split the work. `summarize()` then collapses each group into a single row:

```
df |> group_by($dept)
   |> summarize($count = nrow($dept), $avg_score =
 ↪  mean($score))
```

The result has one row per group, with the grouping column and the aggregates you asked for.

14.1.6 Composing verbs

The power is in the chaining. A filter, a select, a sort, and a count become a single readable expression:

```
df |> filter($age > 25)
   |> select($name, $score)
   |> arrange($score, "desc")
   |> nrow
```

Read it top to bottom and it describes the analysis in plain language. That is the same readability you get from `{dplyr}`, with no runtime switch.

14.2 Joins, duplicates, and conditional values

The core verbs work on a single table. Real analyses usually involve several, so `colcraft` brings the join family, duplicate handling, and conditional values.

14.2.1 Joining tables

The join verbs take the other table first and the key columns second; if you name no key, T joins on the columns the two tables share:

```
employees |> inner_join(projects, $emp_id)
employees |> left_join(projects, $emp_id)
employees |> anti_join(projects, $emp_id)
employees |> full_join(projects, $emp_id)
employees |> right_join(projects, $emp_id)
employees |> cross_join(projects)
```

`inner_join` keeps rows whose key appears in both tables; `left_join` keeps every row of the left table and fills in what it can from the right; `anti_join` keeps the left rows with *no* match, the workhorse for "who has no record yet?"; `full_join` keeps everything from both sides. `right_join` mirrors `left_join`: it keeps every row of the *right* table. `cross_join` takes no key and returns every pairing of left and right rows, the cartesian product — overlapping right column names gain a `_y` suffix.

14.2.2 Duplicates and counts

`distinct()` drops duplicate rows, optionally restricted to the columns you name as the uniqueness key. `count()` collapses a table into one row per group with a count column:

```
df |> distinct()
df |> distinct($name)
df |> count($species)
```

```
df |> count($species, $year, name = "freq")
n_distinct(df["species"])
n_distinct(df["species"], na_rm = true)
```

`n_distinct()` is the vector version of the same question: how many different values, with `na_rm = true` to leave missing values out of the count.

14.2.3 Conditional values

`ifelse()` is the vectorised ternary: a condition, two outcomes, and an optional value for rows where the condition itself is missing. `case_when()` generalises it to many branches, written as `condition ~ value` formulas with a `.default`:

```
df |> mutate($status = ifelse($score >= 50, "pass",
 ↪  "fail"))
df |> mutate($grade = case_when(
    $score >= 90 ~ "A",
    $score >= 80 ~ "B",
    .default = "C"))
```

`case_when` reuses the `~` from model formulas: one syntax for "this predicts that" and "this yields that".

14.2.4 Operators in data work

A few operators from the Chapter 4 reference earn their place in data work.

`in` tests membership against a list, which makes it the natural filter for "any of these":

```
df |> filter($dept in ["eng", "sales"])
```

Inside `filter`, the predicate is evaluated row by row, so each `$column` is a scalar and the plain operators (`&&`, `||`, `==`, `in`) are the right tools. The broadcasting operators come in when you hold a whole Vector or List at once, for example a column pulled out with a lens (Chapter 15):

```
scores = [72, 85, 91]
scores .+ 10          -- [82, 95, 101]
1 .== [1, 2]          -- [true, false]
```

And `%name%` is the escape hatch for custom infix operators: packages define them, and you consume them wherever a named call would be clunkier: the chrono package ships `%within%` for testing whether a date falls inside an interval.

14.3 Reshaping and missing values

Real data is rarely tidy. `colcraft` includes the reshaping verbs you know from `{tidyr}`, plus a small set of tools for missing values.

14.3.1 pivot_longer() and pivot_wider()

`pivot_longer()` turns wide data (several columns holding measurements) into long data (a name column and a value column). `pivot_wider()` does the reverse:

```
-- wide to long
df |> pivot_longer($age, $score, names_to =
 ↪  "measure", values_to = "val")

-- long to wide
df |> select($name, $dept, $salary)
   |> pivot_wider(names_from = $dept, values_from =
 ↪  $salary)
```

14.3.2 separate() and unite()

`separate()` splits one column into several on a delimiter; `unite()` joins several columns back into one:

```
df |> separate($date, into = ["year", "month",
 ↪  "day"], sep = "-")
df |> unite("full_date", $year, $month, $day, sep =
 ↪  "-")
```

14.3.3 nest() and unnest()

`nest()` packs selected columns into a single column of nested DataFrames, grouping by the columns you leave out. `unnest()` expands them back out. This is how you build, and later dissolve, the hierarchical structures that lenses (covered at the end of this chapter) are designed to reach:

```
nested = df |> group_by($cyl) |> nest()
flat = nested |> unnest($data)
```

14.3.4 Missing values

Three verbs handle the gaps. `drop_na()` removes rows with missing values in the named columns, `replace_na()` fills them with a constant, and `fill()` propagates the last non-missing value forward:

```
df |> drop_na($score)
df |> replace_na([score: 0])
df |> fill($category, .direction = "down")
df |> mutate($score = coalesce($score,
 ↪  $backup_score))
```

`coalesce()` picks the first non-missing value position by position across equal-length vectors, so fallback chains read left to right.

14.4 Working with strings

Text columns are everywhere, and T names its string helpers with a `str_` prefix so they are easy to spot:

```
str_nchar("hello")              -- 5
str_substring("hello", 1, 4)    -- "ell"
str_replace("banana", "a", "o") -- "bonono"
str_trim("  hello  ")           -- "hello"
str_join(["a", "b", "c"], "-")  -- "a-b-c"
str_split("a,b,c", ",")         -- ["a", "b", "c"]
str_squish("  too   much   space  ") -- "too much
 ↪  space"
```

A few helpers intentionally keep their plain names, because they double as column-selection helpers in `colcraft`:

```
contains("petal_width", "width")
starts_with("petal_width", "petal")

-- the same names select columns in a DataFrame
df |> select(starts_with("petal"))
```

That overlap is deliberate: `contains`, `starts_with`, and `ends_with` work as string predicates and as selection helpers, so the data-manipulation API stays consistent.

14.5 Dates and times

The `chrono` package brings date and time handling to T, modelled on R's `{lubridate}`. You can parse strings into dates with shorthand helpers:

```
d = ymd("2024-01-15")
dt = ymd_hms("2024-01-15 09:30:00")
```

Extract components, and do calendar-aware arithmetic with periods:

```
year(d)
month(d, label = true)   -- "Jan"

-- adding a month to Jan 31 lands on the last day of
 ↪ February
ymd("2024-01-31") + months(3)   -- 2024-04-30
```

The arithmetic is calendar-aware, not just "add a number of days", which is exactly what you want when a rule is "the same day next month". A typical workflow parses a date column, derives a month, and summarises by month:

```
df |> mutate(date = ymd($order_date),
             month = month($date, label = true))
   |> filter(year($date) == 2024)
   |> group_by($month)
   |> summarize($total = sum($sales))
```

14.6 Factors and categorical data

A factor is categorical data with an explicit list of levels. The level order matters: `arrange()` sorts a factor by its levels, not alphabetically. That makes factors the right tool for ordered categories such as shirt sizes, survey responses, or reporting buckets.

```
sizes = to_factor(["medium", "small", "large"],
                  levels = ["small", "medium",
 ↪  "large"])
levels(sizes)   -- ["small", "medium", "large"]
```

If you omit `levels`, T derives them from the data and sorts them alphabetically. Use `ordered()` when the ordering is meaningful and should be preserved.

The `fct_*` helpers (a nod to R's `{forcats}`) let you reorder, relabel, and collapse levels after creation:

```
df |> mutate($segment = fct_infreq($segment))
df |> mutate($segment = fct_relevel($segment,
 ↪  "enterprise"))
df |> mutate($segment = fct_lump_n($segment, n = 3))
```

14.7 Arrays

For heavy numerical work, T has first-class NDArrays. Unlike lists, an NDArray has a fixed shape and optimised storage, and it supports element-wise arithmetic with broadcasting:

```
v = ndarray([1, 2, 3, 4, 5])
m = ndarray([[1, 2, 3], [4, 5, 6]])

shape(m)      -- [2, 3]
m + 10        -- broadcasts the scalar across the
 ↪  array
```

A small set of linear-algebra functions is available for matrix work:

```
a = ndarray([[1, 2], [3, 4]])
matmul(a, a)
inv(a)
diag(a)
```

Keep one constraint in mind: NDArrays cannot hold `NA` values, so handle missing data before you convert.

14.8 Formulas and models

T borrows R's formula syntax for specifying models. The `~` operator builds a formula you can pass to a modelling function:

```
model = lm(data = df, formula = salary ~ age)
model.r_squared
model.coefficients.age
```

Multiple predictors and interactions follow the familiar rules:

```
model = lm(data = df, formula = y ~ x1 + x2 + x3)
model = lm(data = df, formula = y ~ x1 * x2)
```

`lm()` returns a dictionary with the coefficients, standard errors, R^2, and a tidy DataFrame of the results. It is not a replacement for a full statistical toolkit, but it covers the quick "does this variable matter" check without leaving T.

14.9 Quick stats in T

Around `lm()` sits a small stats package for the fast checks you do while exploring: the "does this variable matter, and what is its shape?" questions.

Correlation, covariance, and the five-number summary are the usual first moves:

```
cor(df["mpg"], df["wt"])
cov(df["mpg"], df["wt"])
fivenum(df["mpg"])
```

`cut()` discretises a numeric vector into intervals, which is how you turn a continuous variable into a categorical one for a table or a plot:

```
cut(df["mpg"], 4)
cut(df["mpg"], [10, 20, 30])
```

Cumulative functions and ranking are available as window functions inside `mutate`, so they respect the current grouping:

```
df |> mutate($cum = cumsum($amount))
df |> mutate($cum_avg = cummean($amount))
df |> group_by($dept) |> mutate($rank =
 ↪  dense_rank($score))
```

When you do fit a model, `conf_int()` gives you confidence intervals for its coefficients:

```
model = lm(data = df, formula = salary ~ age)
conf_int(model)
conf_int(model, 0.99)
```

And when you have two models and want to know whether the bigger one actually fits better, `anova()` compares them with an F-test:

```
m1 = lm(data = df, formula = y ~ x1)
m2 = lm(data = df, formula = y ~ x1 + x2)
anova(m1, m2)
```

None of this replaces a statistics course, but it keeps the exploratory loop (look, check, reshape, look again) inside one language.

14.10 Lenses: reaching the nested

The verbs above are built for flat DataFrames. Once your data becomes nested, after a `nest()` or from a hierarchical API, the standard verbs turn into a pyramid of nested lambdas. Lenses solve that.

A lens names a *path* into a structure. You build it once with `compose()`, then use it to read or transform the value at that path in a single step:

```
-- path: clients -> projects -> milestones ->
 ↪  pct_int
pct_l = compose(col_lens("projects"),
                col_lens("milestones"),
                col_lens("pct_int"))

-- apply a 10-point uplift to every milestone,
 ↪  everywhere
updated = clients |> over(pct_l, \(x) min(x .+ 10,
 ↪  100))
```

The three core operations are `get()` (read), `set()` (replace), and `over()` (transform). `col_lens()` targets a column or key, `row_lens()` and `idx_lens()` target a row or index, and `filter_lens()` is a traversal that focuses on every element matching a condition:

```
-- add 10 to every even score
even_l = filter_lens(\(x) x % 2 == 0)
scores |> over(even_l, \(x) x + 10)
```

One detail matters when you transform a whole column: `over()` hands your function a Vector, so use the broadcasting operators (`.+`, `.*`) for element-wise math rather than the scalar ones.

For flat DataFrames, stick with the ordinary verbs: they are simpler and more idiomatic. Lenses earn their keep in nested and polymorphic data, where the alternative is a tower of `map` calls.

14.11 Where to go from here

You now have a full data-manipulation toolkit that lives in T itself: the `colcraft` verbs for selecting, filtering, mutating, and summarising; joins, duplicates, and conditional values; reshaping and missing-value tools; string, date, and factor helpers; arrays for numerical work; formulas and a quick stats package for models; and lenses for the nested structures the flat verbs cannot reach.

None of this is a reason to abandon R. It is a reason to keep the small, glue-y transformations in the language that already orchestrates your pipeline, and reach for R or Python when the analysis genuinely calls for it. The next chapter goes deeper on T itself: the REPL, metaprogramming, serializers, plotting, and packaging.

15 Beyond the Basics: T in Depth

The earlier chapters took you from a blank Nix installation to a fully automated, reproducible pipeline running on GitHub Actions. Along the way you learned the core loop: declare nodes, build, inspect, test, and ship. But T is a larger language than that loop requires, and this chapter is where we go deeper.

The material here now focuses on more of T's features. We cover ten topics, in the order you are likely to need them. Each section is self-contained, but it builds on the foundations from the earlier chapters, and we point back to them rather than repeating what they already teach.

15.1 The Interactive T: Magic Commands

Chapter 5 introduced the T REPL as a place to explore data and prototype node logic before committing it to a pipeline. The REPL has a small set of conveniences that make interactive work faster: the magic commands.

Magic commands are REPL-only shortcuts prefixed with `%`. They give you quick access to common operations without parentheses

or string quotes. They are not part of the language proper (they only work at the REPL prompt) so they will never appear in a `pipeline.t` file.

15.1.1 Navigating your environment

The first group of commands helps you get your bearings. `%pwd` prints the current working directory, and `%cd` changes it (it expands `~` correctly, and reports an error if the target directory does not exist). `%ls` lists the contents of the current directory, and `%env` prints every environment variable, one per line.

```
%pwd
/home/user/projects
%cd data
%ls
raw.csv  clean.csv
```

These mirror the shell commands you already know, so the REPL feels less like a foreign object and more like a workspace you can move around in.

15.1.2 Inspecting what you have

When you have been working in the REPL for a while, it is easy to lose track of what is in scope. `%objects` (aliased `%who`) lists every user-defined variable, its type, and a compact summary:

```
x = 42
df = read_csv("data.csv")
%objects
--  name  type       summary
--  x     Int        42
--  df    DataFrame  150 rows x 5 cols
```

`%history` shows the entries from the REPL command history, stored in `~/.t_history`, which is handy for re-running or copying something you did a few minutes ago. `%reset` removes all user-defined variables and returns you to a clean base environment.

15.1.3 Timing and capturing work

`%time` evaluates any T expression and prints both its result and how long it took. It is the quickest way to get a feel for whether a piece of logic is fast enough to keep:

```
%time df |> filter($x > 10) |> nrow()
42
Execution time: 0.1540 seconds
```

`%save` writes a transcript of the session, i.e. every command and its result to a file, so you can turn an exploratory session into a starting point for a script. In compact mode it records a one-line summary of each result; in verbose mode it records the full pretty-printed output:

```
%save my_session.t
%save verbose my_session.t
```

A common workflow is to run `%reset` to clear the transcript, then do a clean run and `%save` it, giving you a tidy record of exactly what you tried. Typing `%magic` lists every magic command with a description, and if you mistype one, the REPL offers a fuzzy match: `%objcts` suggests `Did you mean: %objects?`.

15.2 Functions in Depth

You have already written functions in T: Chapter 6's functional core gave you pure functions and composition, which is enough to build a pipeline. But T's functions go further: they are first-class values you can build, transform, and pass around. This section covers the parts you will reach for as your pipelines grow.

15.2.1 Lambda syntax

T defines functions with a lambda. The preferred style is the R-like `\(...)` form, though a `function` keyword is also accepted:

```
square = \(x) x * x
square = function(x) x * x    -- equivalent
```

Multi-argument functions are just as natural:

```
add = \(a, b) a + b
add(3, 7)  -- 10
```

15.2.2 Closures

A function captures its enclosing environment, which lets you build parameterised families of functions:

```
make_adder = \(n) \(x) x + n
add5 = make_adder(5)
add5(10)  -- 15
```

15.2.3 Higher-order functions

Because functions are values, you can pass them to other functions. The standard library is full of them:

```
numbers = [1, 2, 3, 4, 5]
map(numbers, \(x) x * x)      -- [1, 4, 9, 16, 25]
filter(numbers, \(x) x > 3)  -- [4, 5]
```

15.2.4 Auto-quotation (`$param`)

A particularly useful feature when wrapping data verbs: prefix a parameter with `$` and the caller can pass a bare name, a column name say, which is captured as a **Symbol** rather than evaluated. This is the same mechanism the data verbs themselves use (see the next section):

```
my_select = \(df, $col) select(df, col)
my_select(df, salary)    -- 'salary' is captured as
 ↪  a symbol
```

15.3 The Type System

Chapter 4 introduced annotations in passing: read them as a promise about what a function accepts and returns. Here is what is actually going on.

15.3.1 Lambda signatures

A lambda is either untyped or typed. The typed form annotates every parameter and the return type. The return type is written *inside* the parameter parentheses, which keeps the whole signature on one line:

```
add     = \(x, y) x + y                          --
 ↪ untyped
add_int = \(x: Int, y: Int -> Int) (x + y)       --
 ↪ typed
```

Generic lambdas declare their type variables explicitly with `<...>`:

```
id   = \<T>(x: T -> T) x
pair = \<A, B>(x: A, y: B -> Tuple[A, B]) [x, y]
```

The annotation grammar covers the base types (`Int`, `Float`, `Bool`, `String`, `NA`) and the composite forms `List[T]`, `Dict[K, V]`, `Tuple[T1, T2, ...]`, and `DataFrame[schema]`, where the schema names the columns the DataFrame is promised to carry.

15.3.2 Two modes: repl and strict

The same code can be checked differently depending on how you run it:

- `t repl` runs in **repl mode**, which is permissive: untyped top-level functions are fine, which is what you want while exploring.
- `t run <file.t>` runs in **strict mode** by default. For every top-level function assignment, strict mode requires all parameter types to be annotated, a return type to be present, and any type variables to be declared. Miss one and the script fails before it runs:

```
add = \(x, y) x + y
```

```
Error(TypeError): [add.t:L1:C7] Strict mode:
 ↪  top-level function 'add'
must annotate all parameter types.
```

`--mode repl|strict` overrides the default in either direction, so a script can opt into leniency and a REPL session into rigor.

15.3.3 How far it goes today

Be clear-eyed about what strict mode is: a **signature validation layer**, not yet a full static typechecker. It checks that top-level functions carry complete, well-formed signatures; it does not (yet) infer types across expressions, check function applications end to end, or enforce `DataFrame[schema]` column by column. That is planned, and the practical advice follows from it: put

explicit signatures on your top-level and exported functions now, so the code is ready when the checker grows into the promises you wrote.

15.3.4 Semantic type names

Annotations accept several spellings for the same type, so signatures can read naturally: `int`/`integer`, `string`/`text`, `bool`/`boolean`/`logical`, `float`/`double`/`number`/`numeric`, and `to_dataframe`/`table`. The name `any` (synonyms `value`, `all`, `mixed`) is a wildcard meaning "no constraint."

The most concrete payoff today is schema checking. `t check --schema` propagates column names and types across nodes and flags a reference that cannot be resolved, in milliseconds, before the pipeline ever runs. Chapter 5 covers the full tiered `t check` story; this is the tier that turns a `DataFrame` schema annotation into an actual guarantee.

15.4 Metaprogramming: Quotation and Quasiquotation

Chapter 6 introduced T's functional core: pure functions and function composition. Most of the time you can stay in that world and never think about how expressions are evaluated. But sometimes you need to treat code itself as data: to build expressions programmatically, to forward a column name that is only known at runtime, or to write a function that behaves like the data verbs such as `mutate` and `filter`. That is what metaprogramming is for.

T's metaprogramming is modelled on Lisp and on R's `rlang`, and it rests on a small set of ideas:

- **Quotation** captures an expression without evaluating it.
- A **quosure** is a quoted expression paired with the environment in which it was written.
- **Unquoting** injects an already-evaluated value back into a quoted expression.
- **Splicing** expands a collection into the arguments of a call.

15.4.1 Capturing code: `to_expr` and `quo`

The two ways to capture code differ in whether they remember the surrounding environment. `to_expr()` captures a bare expression; `quo()` captures a quosure: the expression plus the environment at the call site.

```
x = 10
q = quo(1 + x)   -- captures x = 10
x = 99
eval(q)          -- 11, not 100: it runs in the
 ↪  captured environment
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b3a7186f4c7eec9d/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b3a7186f4c7eec9d/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

```
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
99
11
```

That distinction is the whole point of a quosure. When `eval()` runs a quosure, it evaluates it in the environment the quosure captured, not in whatever environment happens to be current. For a bare expression from `to_expr()`, `eval()` uses the current environment.

There are plural forms too: `to_exprs(...)` and `quos(...)` capture several expressions at once, returning a list of bare expressions or quosures respectively.

15.4.2 Unquoting: `!!` and `!!!`

Quasiquotation is how you fill in the blanks of a captured expression. The `!!` operator evaluates its operand and injects the result into the surrounding quoted expression. If the operand is

a quosure, only the expression part is injected; the environment is stripped.

```
x = 10
e = to_expr(1 + !!x)
print(e)          -- to_expr(1 + 10)
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-aa6caea85282f257/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-aa6caea85282f257/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
```

```
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-aa6caea85282f257/ch
 ↪  Division by zero.")
```

The `!!!` operator goes further: it evaluates its operand and *splices* the elements into the surrounding call. The operand must be a list, vector, or dict.

```
vals = [1, 2, 3]
e = to_expr(sum(!!!vals))
print(e)          -- to_expr(sum(1, 2, 3))
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-d9bdc359bf30905b/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-d9bdc359bf30905b/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
```

```
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-d9bdc359bf30905b/
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-d9bdc359bf30905b/
 ↪  Division by zero.")
```

If you splice a named list, the names become argument names:

```
my_args = [x: 10, y: 20]
e = to_expr(f(!!!my_args, z: 30))
print(e)          -- to_expr(f(x = 10, y = 20, z =
 ↪  30))
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-a79751cc0cca08a5/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-a79751cc0cca08a5/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

```
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-a79751cc0cca08a5/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-a79751cc0cca08a5/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-a79751cc0cca08a5/ch
 ↪  Division by zero.")
```

15.4.3 Dynamic names and symbols

A frequent need is to use a name that is only known at runtime, a column name stored in a string, say. `to_symbol()` turns a string into a symbol that `!!` can inject, and the `!!name := value` form lets a computed name become an argument or column name.

```
col = "age"
e = to_expr(mutate(df, !!col := 42))
print(e)          -- to_expr(mutate(df, age = 42))
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-ce2d0a1d7a16f3ef/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-ce2d0a1d7a16f3ef/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-ce2d0a1d7a16f3ef/
 ↪  Division by zero.")
```

```
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-ce2d0a1d7a16f3ef/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-ce2d0a1d7a16f3ef/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-ce2d0a1d7a16f3ef/ch
 ↪  Division by zero.")
```

`get()` is the companion for the reverse direction: it retrieves a variable's value from a string or symbol name, which is how you look up a column dynamically.

15.4.4 Non-standard evaluation

The most common reason to reach for all of this is to write a function that accepts an unevaluated column name, the way the data verbs do. T gives you three tools for it, in order of increasing power:

1. **Auto-quoted parameters.** Prefix a parameter with `$` and the caller can pass a bare column name, which you forward into an NSE-aware verb with `!!`.

```
my_mean = \(df, $col) {
  summarize(df, result = mean(!!col))
}
```

2. **`enquo(param)`** captures the caller's full expression for a named parameter, as a quosure. Use it when you need more than a column name.

3. **`enquos(...)`** captures all variadic expressions as a list of quosures, which you can splice into a verb.

```
my_summarize = \(df: DataFrame, ... -> DataFrame) {
  cols = enquos(...)
  eval(to_expr(df |> summarize(!!!cols)))
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6d0c4c66a8b7ee9c/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6d0c4c66a8b7ee9c/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
```

```
0
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6d0c4c66a8b7ee9c/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6d0c4c66a8b7ee9c/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6d0c4c66a8b7ee9c/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-6d0c4c66a8b7ee9c/ch
 ↪  Division by zero.")
```

The default advice is simple: use `quo` over `to_expr` when in doubt, so your code remembers its environment; use the `$param` form for the common "accept a column" case; and reach for `enquo`/`enquos` only when you genuinely need the caller's expression.

One detail is worth knowing because it prevents a whole class of confusing bugs. Inside a data verb, expressions are resolved against a **data mask**: T looks up `$column` in the mask first, so a global variable or function with the same name as a column does not interfere. That is why `df |> mutate(new = $score * 2)` works even when a function called `score` exists in scope.

15.5 Data Access and Lenses

T provides a unified way to retrieve data from variables, collections, dicts, and pipelines through the `get()` primitive, and a

Lens library for addressing specific parts of complex structures. Lenses matter because they are first-class, serializable values: you can pass them between pipeline nodes and apply them without rebuilding.

15.5.1 The `get()` built-in

`get()` is the primary entry point for retrieval. It adapts to what you give it:

```
salary = 50000
get("salary")              -- variable lookup,
 ↪  R-style      50000

lst = [10, 20, 30]
get(lst, 1)                -- 0-based collection
 ↪  indexing   20

d = [a: 1, b: 2]
get(d, "a")                -- dict key lookup
 ↪  1
get(d, "missing", 99)      -- default when key is
 ↪  absent    99
```

Inside an NSE data verb (`mutate`, `filter`, `summarize`, …), `get()` checks the **data mask** first: a name that is a column in the current row or DataFrame is resolved there, and only if not found does it fall back to the global environment. That is what makes `get("a")` inside a `mutate` refer to the column `a`, not a global variable.

15.5.2 Lenses

Where `get()` takes an ad-hoc index, a **lens** is a structured, reusable "address" you can compose and pass around. The built-in lens creators are:

Creator	Target	Description
`col_lens(name)`	`DataFrame`, `Dict`, `List`	Targets a column or key by name.
`idx_lens(i)`	`List`, `Vector`	Targets an element by index.
`row_lens(i)`	`DataFrame`	Targets a specific row as a dictionary.
`node_lens(name)`	`Pipeline`	Targets a specific node within a pipeline.
`env_var_lens(n, v)`	`Pipeline`	Targets an environment variable `v` from node `n`.

You focus a lens with `get()` and compose lenses with `compose()` to reach deep into nested structures:

```
l = col_lens("mpg")
get(mtcars, l)            -- the 'mpg' column as a
 ↪  Vector

data = [users: [Alice: [id: 1], Bob: [id: 2]]]
l_deep = compose(col_lens("users"),
 ↪  col_lens("Alice"), col_lens("id"))
get(data, l_deep)         -- 1
```

To *update* data through a lens, use `over()`:

```
data_updated = over(data, l_deep, \(x) 99)
get(data_updated, l_deep) -- 99
```

Pipeline lenses are serializable: you can pass them between nodes and apply them with `mutate_node()` or `set_pipeline_global_options()`, which is how you make cross-node data operations part of the reproducible build.

15.6 Introspection: `explain()` and `intent`

A language that can describe its own values is easier to debug, easier to test, and, increasingly, easier to work on alongside an agent. T has both halves of that story: `explain()`, which turns any value into a structured description, and `intent`, a block that records *why* a piece of code exists.

15.6.1 `explain()`: a value's self-description

`explain(x)` works on any value and returns a `Dict` describing it. The first field is always `kind`, and the rest depend on what you handed it:

```
explain(42)
-- { `kind`: "value", `type`: "Int", `value`: 42 }

explain([1, 2, 3])
-- { `kind`: "value", `type`: "List", `length`: 3,
 ↪  `na_count`: 0, `examples`: [1, 2, 3] }
```

For a DataFrame, the description is a compact summary (row and column counts, the storage backend, the schema, and per-column missingness), and you can drill into individual fields with dot access:

```
df = to_dataframe([x: [1, 2, 3], y: ["a", "b",
 ↪  "c"]])
explain(df).nrow
-- 3
explain(df).schema
-- [{`name`: "x", `type`: "Int"}, {`name`: "y",
 ↪  `type`: "String"}]
explain(df).na_stats
-- { `x`: 0, `y`: 0 }
```

Errors are values too, and `explain` sees through them, surfacing the code, the message, and where it happened:

```
explain(1 / 0)
-- { `kind`: "value", `type`: "Error", `error_code`:
 ↪  "DivisionByZero",
--   `error_message`: "Division by zero.", ... }
```

The same function reaches the rest of the language. `explain(y ~ x)` reports a formula's response and predictors; `explain(add_int)` reports a function's argument names and types; `explain(p)` reports a pipeline's node count and a per-node summary, while `explain(p.b)` inspects a single node: its name, runtime, dependencies, and configuration. And `explain_json(x)` returns the same description as a single string, which is handy when you want to log a value's shape or hand it to another tool.

The point is that introspection is *first-class*: the same call that describes a scalar also describes a DataFrame, an error, a formula, and a whole pipeline. You do not need a separate debugging API per type.

15.6.2 `intent`: code says what, intent says why

T has a small, dedicated syntax for recording *why* a piece of code exists: a block of key-value pairs that travels with the code:

```
intent {
  description: "Analyze customer churn"
  goal: "Identify segments at risk of leaving"
}
```

An intent block is inert at runtime (it does not change what the code does), but you can read it back programmatically:

```
i = intent { description: "Analyze customer churn",
 ↪  goal: "Find at-risk segments" }
intent_get(i, "description")
-- "Analyze customer churn"
intent_fields(i)
-- { `description`: "Analyze customer churn",
 ↪  `goal`: "Find at-risk segments" }
```

The payoff is provenance. When a pipeline is a collaboration between a human and an agent, or between you today and you in six months, the *why* is the first thing to be lost. `intent` keeps it in the source, versioned in git and queryable by the tools that read it. It is the line-level companion to the `AGENTS.md`

convention from Chapter 4: that file states the project's intent, and an `intent` block states the intent of the code right next to it.

15.7 Escaping to the Shell: `?<{ }>`

Chapter 8 introduced `shn()`, the node constructor that runs a shell command inside a pipeline's sandbox. That is the right tool when the shell work is a first-class step in your pipeline. But there is a lighter way to reach the shell from T itself: the shell escape. The `?<{ }>` syntax runs an arbitrary shell command directly from within T, at the REPL, in a script, or in a node's T code, and is the quickest way to touch the filesystem, drive `git`, or call a CLI tool without standing up a whole shell node.

15.7.1 Running a command

As a standalone statement, the command is executed and its output is printed straight to `stdout`. The statement's value is `NA`:

```
?<{ls -la}>
?<{git status}>
```

15.7.2 Capturing the output

When the escape is part of an expression, most often an assignment, T captures the command's `stdout` and hands it back as a `String`, ready to be split, parsed, or passed on:

```
files = ?<{ls}>
current_user = ?<{whoami}>
```

15.7.3 Changing directory

`cd` is special-cased. Rather than changing the directory of a throwaway sub-shell, it changes the working directory of the T interpreter itself, so subsequent commands, and `?<{pwd}>`, see the new location. `~` is expanded correctly:

```
?<{cd /tmp}>
?<{pwd}>        -- /tmp
?<{cd ~}>
```

15.7.4 Multiline commands

A shell escape can span several lines. The indentation common to all non-empty lines is stripped, so you can write a readable block of shell without backslash continuations:

```
?<{
  git add .
  git commit -m "update"
  git push
}>
```

15.7.5 Errors

If a command exits with a non-zero status, the escape yields a `ShellError` carrying the command's `stderr`, so a failing command is easy to diagnose. You can test for it with `is_error()`:

```
result = ?<{ls /nonexistent}>
is_error(result)   -- true
```

15.8 Serializers in Depth

Chapter 5 showed you that pipeline nodes pass data to one another through **serializers**, and that you can name a built-in one with the `^` prefix. This section goes deeper: what the built-ins are, how to choose between them, and how to write your own.

15.8.1 The built-in serializers

Every node's output is written to disk in some format, and every node reads its inputs in the corresponding format. T ships a set of built-in serializers, identified by `^` symbols:

Identifier	Format	Best for
`^ipc`	Apache Arrow IPC	Fast live hand-off between nodes
`^parquet`	Apache Parquet	Durable, compressed storage
`^csv`	CSV	Simple tabular interchange
`^json`	JSON	Config, lists, dicts
`^pmml`	PMML	Predictive models
`^onnx`	ONNX	ML model interchange

Identifier	Format	Best for
`^text`	Plain text	Logs, shell output
`^bin`	Binary	Opaque blobs (default for `fetchurl`)

Most of these are symmetric across the T, R, Python, and Julia runtimes, which is what makes polyglot pipelines possible: a DataFrame written by an R node can be read by a Python node because both sides understand the same format.

The choice that comes up most often is between `^ipc` and `^parquet`. Both are columnar, type-preserving Arrow formats that work in every runtime. The difference is **live hand-off versus durable artifact**. `^ipc` writes the in-memory Arrow layout straight to disk: the fastest possible round trip, but uncompressed. `^parquet` is a compressed, storage-optimized layout: smaller files (often several times smaller for numeric data), column pruning on read, and first-class support in Spark, DuckDB, and pandas. The rule of thumb is to pass data *between nodes while a pipeline runs* with `^ipc`, and to *persist, ship, or store* the final result with `^parquet`. You can do both in one pipeline.

15.8.2 Symbols, variables, and the string trap

There is a subtle but important distinction in how you name a serializer. Built-in serializers are **symbols** with the `^` prefix. A custom serializer you have defined is a **variable**, and you pass its name with no `^`.

```
node(command = read_csv("large.csv"), serializer =
 ↪  ^ipc) -- built-in
import "src/my_ser.t" [my_ser]
node(command = ..., serializer = my_ser)
 ↪  -- custom
```

```
T execution failed:
Error running t (error code 1): DataFrame(4 rows x 8
 ↪  cols: [name, runtime, serializer, deserializer,
 ↪  noop, deps, depth, command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b56b7aa75700d603/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b56b7aa75700d603/ch
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b56b7aa75700d603/ch
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b56b7aa75700d603/ch
 ↪  Division by zero.")
```

```
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b56b7aa75700d603/
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-b56b7aa75700d603/
 ↪  Division by zero.")
```

And here is the trap: in a node constructor (`node`, `rn`, `pyn`, `jln`, `shn`, `qn`), a **string literal is not allowed**. Writing `serializer = "ipc"` raises a `TypeError`. You must use a `^` symbol for built-ins or a variable for custom serializers. (The `mutate_node()` and `set_pipeline_global_options()` functions are more permissive and accept both strings and symbols.)

If you do not specify a serializer at all, T uses the `default`, which is each runtime's native binary format: `saveRDS` for R, `pickle` for Python, and so on. Shell nodes default to `^text`.

15.8.3 Custom serializers

A serializer is a first-class value: a record with a `format`, a `writer`, and a `reader`.

```
type serializer = {
  format: string,
  writer: function(path: string, value: any) ->
 ↪  result[NA, string],
  reader: function(path: string) -> result[any,
 ↪  string]
}
```

To define one, you write a record that matches that shape. The `format` field should be a `^` symbol so it stays consistent with T's symbol-based serialization.

```
my_log_serializer = {
  format: ^log,
  writer: \(path, val) {
    -- write val to path in your log format
    Ok(NA)
  },
  reader: \(path) {
    -- read the log back from path
    Ok("log content")
  }
}

node(command = ..., serializer = my_log_serializer)
```

For a serializer to work across *other* runtimes, you can add optional `r_writer`, `r_reader`, `py_writer`, and `py_reader` fields. These are code snippets, plain strings or foreign code blocks for readability, that T injects into the generated build script for that runtime.

```
my_custom_ser = [
  format: ^custom,
  writer: \(path, val) { Ok(NA) },
  reader: \(path) { Ok(42) },
  r_writer: <{ function(obj, path) {
↪   writeCustom(obj, path) } },
  r_reader: <{ function(path) { readCustom(path) }
↪   },
  py_writer: <{ def write_custom(obj, path): ... },
```

```
  py_reader: <{ def read_custom(path): ... }
]
```

T performs **static coherence checks** when you build a pipeline that uses a custom serializer: if a node is declared for the R runtime but the serializer has no `r_writer` or `r_reader`, you get an error at build time rather than a mystery failure at run time. And when T detects that your serializer references R or Python functions, it can add the corresponding `[r-dependencies]` or `[py-dependencies]` to the generated build automatically (controlled by `TLANG_AUTO_ADD_PIPELINE_DEPS`).

15.9 Plotting in Pipelines

Chapter 8 showed you how to structure a pipeline as data. This section covers what happens when a node's output is not a DataFrame but a plot.

15.9.1 Returning a plot from a node

Any node can return a plot object; a `Plot` from CairoMakie, a ggplot2 object, a matplotlib figure, and so on. When it does, T serializes the plot and, crucially, records **visualization metadata** alongside the artifact. That metadata is what lets downstream tools know the output is something to be *shown*, not just *read*.

Let's study this pipeline:

```
p = pipeline {
  iris_data = rn(
    command = <{
      # Load iris dataset
      iris <- datasets::iris

      # Return the data
      iris
    }>,
    serializer = ^csv
  )
  iris_ggplot = rn(
    command = <{
      # Load ggplot2
      library(ggplot2)

      ggplot(iris_data, aes(x = Petal.Length, y =
 ↪  Sepal.Length)) +
        geom_point() +
        facet_wrap(~ Species) +
        labs(title = "Sepal.Length ~ Petal.Length |
 ↪  Species")

    }>,
    deserializer = ^csv
  )
}
```

```
DataFrame(4 rows x 8 cols: [name, runtime,
 ↪  serializer, deserializer, noop, deps, depth,
 ↪  command_type])
["cleaned", "features"]
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-9cebdaac5b6719f6/cl
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
```

```
Error(TypeError:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-9cebdaac5b6719f6/
 ↪  Function `pipeline_nodes` expects a Pipeline,
 ↪  but got Error.")
true
DivisionByZero
Division by zero.
{}
10
true
DivisionByZero
0
5
empty list
head 1, 2 more
not a non-empty list
Caught: Division by zero.
0
99
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-9cebdaac5b6719f6/
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-9cebdaac5b6719f6/
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-9cebdaac5b6719f6/
 ↪  Division by zero.")
Error(DivisionByZero:
 ↪  "[/tmp/nix-shell-128126-363217024/tlang-9cebdaac5b6719f6/
 ↪  Division by zero.")
```

15.9.2 Reading a plot back

Once built, it is possible to *read* the plot using `read_node()`, which on a plotting node returns not the raw artifact but a small metadata record describing it: the path to the serialized plot, its type, and the runtime that produced it.

```
read_node(p.iris_ggplot)
```

returns:

```
ggplot
   backend: "R"
   title: "Sepal.Length ~ Petal.Length | Species"
   mapping
       x: "Petal.Length"
       y: "Sepal.Length"
   labels
       title: "Sepal.Length ~ Petal.Length | Species"
   layers
        geom_point: "Point"
```

and to actually see the plot in your browser, use `show_plot()`:

```
T> show_plot(p.iris_plot)
```

This should open the plot in your browser, if not, the path is emitted and you can simply open the generated PNG file using whichever tool you prefer:

```
_pipeline/show_plot_render_20260821_132239_6808.png
```

This plot is generated in the actual R environment defined in your pipeline, but should something look off, try `debug_node(p.iris_plot)` and build it from the actual R session!

15.9.3 Plots in literate documents

The most common place this matters is a Quarto document. T's Quarto integration has a dual behaviour that is worth understanding. A `{t}` chunk that produces a plot records the visualization metadata, and a companion `{r}` or `{python}` chunk can then call `show_plot()` to render it. This lets you compute the plot in the runtime that is best suited to it and display it in the document without copying data between runtimes.

The mechanism depends on a few underlying libraries: CairoMakie for T plots, `kaleido` for exporting ggplot2 objects to images, and `cloudpickle` for serializing Python plot objects that the standard `pickle` module cannot handle. If a plot does not render the way you expect in a document, the first thing to check is whether the runtime that produced it has the right export library available in its Nix environment.

15.10 Nix Build Options and Orchestration

Chapter 2 introduced Nix as the foundation that makes T reproducible. Chapter 5 showed you `build_pipeline()` and `t run` at

the command line. This section covers the knobs in between: the `nix_options` dictionary that controls how a pipeline is actually built, and the ways to orchestrate that build.

15.10.1 The `nix_options` dictionary

The build functions, `populate_pipeline()`, `build_pipeline()`, `pipeline_run()`, and the `t_make()` helper, all accept a `nix_options` argument. It is a dictionary of build-time settings that override the defaults. The most useful keys are:

- `max_jobs` and `max_cores`: how much of the machine a build may use.
- `dry_run`: evaluate the build plan without running any node.
- `force`: rebuild nodes even if their outputs already exist.
- `targets`: build only a subset of the pipeline's nodes.
- `cache`: which Nix cache to use for the build.
- `builders`: which machines to build on (local or remote).
- `keep_env` and `sandbox`: how much of the host environment a node sees.
- `env_vars`: per-node environment variables, set when the node is defined.

Validation is strict: pass a key that is not a valid option, or a value of the wrong type, and you get a `TypeError` or `ValueError` before the build starts.

```
build_pipeline(p, nix_options = { max_jobs = 4,
 ↪  dry_run = true })
```

15.10.2 Dry runs and plans

Setting `dry_run = true` is one of the most useful habits to form. Instead of executing the pipeline, the build returns a **plan** as a DataFrame: every node that would run, in dependency order, with its inputs and outputs. You can inspect it, filter it, or assert on it in a test. It is the cheapest way to check that your pipeline is wired the way you think it is.

15.10.3 Where environment comes from

It is worth separating two related ideas. `keep_env` controls whether a node sees the host's full environment or a clean one; this is a build-time isolation choice. `env_vars`, by contrast, is how you declare specific variables a node needs, and it is set when the node is defined, not at build time. The first is about isolation; the second is about configuration.

15.10.4 Building on other machines

The `builders` option is what turns a local build into a distributed one. You can point a build at a remote Nix builder, a faster machine or a machine with the right architecture, and Nix will ship the build steps there. This is particularly useful when your pipeline has heavy nodes that would take minutes locally but seconds on a larger box. The `pipeline_to_drv(p)` function returns the underlying Nix derivation, which is the escape hatch for when you need to hand the build to a tool that understands derivations directly.

The `cache` option is the companion to `builders`. Where `builders` decides *where* a build runs, `cache` decides *where*

pre-built artifacts come from. Like `builders`, it is a key in the `nix_options` dictionary. Passing a Cachix cache name registers it as an extra Nix binary substituter:

```
build_pipeline(p, nix_options = { cache = "my-team"
 ↪  })
```

Nix then pulls any matching store paths from `my-team.cachix.org` before falling back to building them, and can push freshly built artifacts back to the cache. This is how a team shares build results across machines and CI runners without each one recompiling the same R and Python packages from source.

15.11 Building T Packages

Chapter 12 covered packaging T pipelines for distribution as R and Python packages. But T can also be packaged *in T*. This section is about that: turning a collection of T functions into a reusable package that others can import.

15.11.1 Initialising a package

`t init --package` scaffolds a T package. The result is a directory with a `DESCRIPTION.toml` that declares the package's metadata, its dependencies, and its build inputs, and a `flake.nix` that makes the package itself a Nix input.

```
[package]
name = "mylib"
version = "0.1.0"

[dependencies]
otherlib = { git = "https://github.com/me/otherlib",
 ↪  tag = "0.3.0" }

[additional-tools]
# extra tools the package needs at build time
```

Dependencies are declared by git URL and tag, which keeps them reproducible in the same way the rest of T is.

15.11.2 Public and private API

By default every top-level function in a package is part of its public API. Mark a function `@private` to exclude it from the documented surface. This matters because the public API is what `t doctor` checks for and what the generated documentation advertises.

15.11.3 Testing a package

`t test` runs the package's tests, and it honours the same flags as the pipeline tests from Chapter 7: you can select a subset, run in verbose mode, and so on. A test marked `noop` is skipped, useful for a test that depends on an optional dependency that may not be present.

15.11.4 T-Doc documentation

T packages use a lightweight documentation format, T-Doc. A function is documented with a `--#` block that supports directives:

```
--# @param x The input value.
--# @return A transformed value.
--# @example
--#   myfunc(1)
--# @seealso myotherfunc
--# @family transforms
myfunc = \(x) { x * 2 }
```

`t doc --parse --generate` parses these blocks and generates the package's documentation. `t doctor` is the linter: it checks that public functions are documented, that `@param` names match the signature, and that cross-references resolve.

15.11.5 Publishing

`t publish` publishes the package by tagging the git repository. There is no central T package registry; distribution is by git, which is deliberate: it means a package version is pinned to an exact commit, and consumers depend on it the same way they depend on any other Nix input.

15.11.6 Importing

Consumers import a T package the same way they import any T module, by name or by path, with optional selective or aliased imports:

```
import "mylib"
import "mylib" [myfunc]
import "mylib" [myfunc as transform]
```

15.12 Property-Based Testing with Propcraft

Chapter 7 introduced unit testing with Testcraft: you write concrete examples and assert on their results. That is the right tool for most functions, but it has a blind spot. A function can pass every example you thought of and still be wrong for the input you did not think of. Property-based testing is how you close that gap, and in T it is provided by Propcraft.

15.12.1 The idea

Instead of testing specific inputs, you describe a **property** that should hold for *all* inputs in some domain, and Propcraft generates hundreds of random inputs to try to break it. If it finds one that fails, it **shrinks** the failing input down to a minimal, easy-to-read counterexample.

The entry point is `prop_for_all`, which takes a generator for each argument and a property function:

```
prop_for_all(
  prop_gen_int_range(0, 100),
  prop_gen_int_range(0, 100),
  \(a, b) { add(a, b) == add(b, a) }
)
```

15.12.2 Generators

A generator describes a domain of values. Propcraft ships generators for the common types:

- `prop_gen_int`, `prop_gen_int_range(lo, hi)`
- `prop_gen_float_range(lo, hi)`
- `prop_gen_bool`
- `prop_gen_string_from(alphabet)`
- `prop_gen_factor(levels)`
- `prop_gen_date_range(lo, hi)`
- `prop_gen_df(...)` and `prop_gen_df_from(template, na_prob = 0.1)`

The DataFrame generators are the ones you will use most. `prop_gen_df` builds a random DataFrame with the column types you specify, and `na_prob` controls how many missing values to sprinkle in, which is exactly the kind of input that exposes bugs in real pipelines.

You can also build generators out of other generators with combinators: `prop_map_gen` transforms a generator's output, `prop_such_that` filters it to values satisfying a predicate, and `prop_resize` scales the size of a collection-valued generator.

15.12.3 The property contract

There is one rule that catches people out: **a property that returns `NA` fails**. A property must return a boolean, and `NA` is not a `true`. This is deliberate: a property that cannot decide is not a valid property. The `Expect` type is understood by Propcraft, so you can write properties in terms of the same assertions you use elsewhere. A property that raises an `Error` also fails.

15.12.4 A concrete example

Here is the kind of bug property-based testing finds. Suppose a function is supposed to drop rows with missing values in a given column:

```
prop_for_all(
  prop_gen_df(na_prob = 0.3),
  \(df) { nrow(drop_na(df, $x)) <= nrow(df) }
)
```

The property, that the result has no more rows than the input, is obviously true, yet it fails. Propcraft shrinks the failing input to a tiny DataFrame and shows you the exact rows that broke it, instead of leaving you to guess.

15.12.5 Seeding and sample size

Generated inputs are random, so tests can be flaky unless you control the randomness. `set_seed(n)` and `with_seed(n, ...)` make a property test deterministic for a given seed. The `n` argument of `prop_for_all` sets how many cases to try; it defaults to 100.

15.12.6 Running with `t test`

Propcraft properties are just tests, so `t test` runs them alongside your Testcraft unit tests. A failing property shows the shrunk counterexample and the seed that reproduces it, so you can pin the seed and turn the counterexample into a permanent regression test.

One caveat: property-based testing is a **package-hardening** tool. It shines when you are writing a library that others will call with arbitrary inputs. For a one-off analysis pipeline, the concrete examples from Chapter 7 are usually enough, and the cost of generating and shrinking random DataFrames is not worth it.

15.13 Where to go from here

You now have the full toolkit: the REPL conveniences, the metaprogramming primitives, the shell escape, the serializer and plotting machinery, the build orchestration knobs, the package toolchain, and property-based testing. None of these are required for a simple pipeline, but together they are what let a T project grow from a script into a library, a service, or a platform without changing its foundations. The next chapter pulls the threads together.

16 Conclusion: Adopting Reproducibility in the Real World

16.1 Introduction

If you have made it this far, you now have a powerful toolkit at your disposal. You understand how Nix provides truly reproducible environments, how functional programming leads to testable and predictable code, how packages bundle your work for sharing, and how CI/CD automates the boring parts. But knowing these tools is only half the battle. The harder question is: how do you actually start using them?

If you work alone, the answer is simple: just start. Pick one project and commit to doing it the "right" way. You will make mistakes, you will get frustrated, and you will learn. But if you work in a team, the challenge is different. You cannot simply mandate that everyone learns Nix by next Tuesday. Change management is a skill in itself, and this final chapter offers some practical strategies for introducing reproducible practices to yourself, your team, and your organisation.

The philosophy here is simple: show, don't tell. Nobody was ever convinced to adopt a new tool by a slideshow. They are

convinced when they see it solve a real problem.

16.2 Start with yourself

Before you try to convince anyone else, you need to become proficient yourself. Pick a small, low-stakes project and use it as your learning ground. A perfect candidate is a personal side project or an internal analysis that nobody else depends on.

Your goal in this phase is to build muscle memory. You want to reach the point where writing a `pipeline.t` feels as natural as opening RStudio. You want to be able to troubleshoot common Nix errors without panic. You want to have a working CI pipeline that you understand inside and out.

This investment pays dividends later. When a colleague asks "Why is this so complicated?", you will have a ready answer because you asked yourself the same question two months ago.

16.3 The two-minute demo

Once you are comfortable, look for opportunities to demonstrate value. The best opportunities are problems that everyone recognises but nobody has solved.

"It works on my machine" is the classic. The next time a colleague struggles to run your code, don't just send them a fixed script. Send them the T project and a one-liner: `nix develop --command t run`. When the whole pipeline runs on the first try, you will have their attention.

Another opportunity is the dreaded "update everything and pray" scenario. When a project breaks after a package update, you can show how Nix lets you pin exact versions. "This analysis will run the same way in five years" is a powerful statement.

The key is to solve real problems, not hypothetical ones. Do not lecture your colleagues about reproducibility in the abstract. Show them how it saved you two hours of debugging on Tuesday.

16.4 Make it easy for others

If you want your team to adopt these practices, you need to lower the barrier to entry as much as possible. This means:

1. Write excellent documentation. Not for yourself, but for the colleague who has never heard of Nix. Assume they will spend five minutes reading before giving up.

2. Provide templates. Create a repository with a working T project: a `pipeline.t`, a `tproject.toml`, a CI workflow, and a simple example analysis. When someone starts a new project, they can clone this template and be productive immediately.

3. Automate the setup. If your organisation allows it, provide a script that installs Nix with a single command. The fewer steps, the fewer places where someone can get stuck.

4. Be available. When someone tries your approach and hits a wall, they need help within minutes, not days. If you are not responsive, they will abandon the effort and go back to their old ways.

16.5 Pick your battles

You cannot change everything at once. Be strategic about where you focus your energy.

Some projects are not worth the effort to convert. If an analysis was run once three years ago and will never be touched again, leave it alone. Focus on new projects and on projects that are actively maintained.

Some colleagues will never be convinced. That is fine. You do not need everyone on board. You need a few early adopters who will become advocates themselves. Focus on the curious ones, the ones who ask "how did you do that?" when they see your workflow.

Some organisations have constraints you cannot change. Maybe you cannot install Nix on the production server. Maybe you are stuck with Windows machines. Work within these constraints rather than fighting them. Docker can bridge many gaps. WSL2 makes Nix possible on Windows. Find the path of least resistance.

16.6 The gradual adoption path

Here is a realistic roadmap for introducing reproducibility to a team:

Month 1-2: Personal mastery. Use Nix and the tools from this book on your own projects. Get comfortable. Build your own templates and workflows.

Month 3-4: Show and tell. Start sharing your work. When you give a code review, mention that the project runs in a Nix

shell. When someone asks how to add a package, show them that it is a line in `tproject.toml` followed by `t update`. Let people discover the value organically.

Month 5-6: Pilot project. Propose using the full stack (Nix, tests, CI) for one team project. Pick something visible but not critical. Document everything. Make it a success.

Month 7-12: Expand. Use the pilot project as a template for new work. Gradually, "the new way" becomes "the way we do things here." Update onboarding documentation to include Nix setup.

This is slow. It takes patience. But lasting change always does.

16.7 Common objections and how to address them

"This is too complicated."

It is more complicated than doing nothing. But it is simpler than debugging environment issues at 3am before a deadline. Complexity is a trade-off. You are trading upfront learning for long-term reliability. Acknowledge the learning curve, but emphasise that you are there to help.

"We don't have time for this."

You do not have time not to do this. Every hour spent on environment issues is an hour not spent on analysis. Every "it works on my machine" is a risk. Frame reproducibility as a time investment, not a time cost.

"My code works fine without it."

It works today. Will it work in six months? Will it work when you hand it to a colleague? Will it work when the package maintainer pushes a breaking change? Reproducibility is insurance. You hope you never need it, but you are grateful when you do.

"I'll learn it later."

Later never comes. Start with one small project. Commit to using one new tool. You can learn incrementally. You do not need to master everything before you begin.

16.8 LLMs as adoption accelerators

Throughout this book, we have seen how LLMs can assist with writing tests, generating documentation, and even drafting functions. But there is a bigger point to make here: T was designed with LLMs in mind, and that design choice changes how you adopt everything else in this book.

The old objection to tools like Nix was that the learning curve was too steep. You had to understand a new language, a new paradigm, and a lot of unfamiliar syntax. T softens that objection in two ways. First, its syntax is simple enough that a human can read and write it comfortably, without a manual. Second, and more importantly, it was designed from the start to be written by LLMs. The nodes, the dependencies, the data flow: all of it maps cleanly onto the kind of structured, declarative description that language models are good at producing.

So who writes your pipeline? In practice, both. You can open a `pipeline.t` file and understand it. You can also hand a problem to an LLM and get a working pipeline back. The same file can be edited by a human on Monday and regenerated by a model on Tuesday, and both versions stay legible to both of you. That is a

rare property, and it is what makes T a good fit for AI-assisted work.

But the language is only half the story. What actually determines whether an LLM contributes meaningfully is the context you give it, and the harness that surrounds it. Every T project ships with an `AGENTS.md` file and a `T-LANGUAGE-REFERENCE.md`. The `AGENTS.md` is where you tell the model how your project works: where the data lives, what the conventions are, what "done" looks like. The language reference gives it the precise syntax it needs. Together they turn a general-purpose model into something that knows your project. `t init` generates both for you, which is why a brand-new project is already LLM-ready.

This is where the real investment is, and it is not in prompting. It is in thinking. Take the time to understand your project well enough to explain it. A model given a rich, accurate description of your data and your goals will produce something you can build on. A model given a vague one-liner will produce something plausible but wrong. The quality of the context is the quality of the result.

And there is a lot you can do, as a human, to make the model's life easier. You do not have to ask the LLM to do everything. If downloading and cleaning the raw data into a tidy, ready-to-use format is something you can do once by hand, do it, and store the result in the project. Now the model starts from clean input instead of spending its effort, and your review time, writing a script to scrape and wrangle data you could have prepared yourself. The same goes for pinning a tricky dependency, sketching the first version of the pipeline's nodes, or writing the hard function yourself. You are not offloading your thinking to the model; you are clearing the path so that when it does think, it thinks about the right things.

The human's job, then, shifts from writing every line to reviewing and directing. The bottleneck is no longer "I need to learn everything"; it is "I need to understand enough to verify the output." That is a much smaller ask, and it is why the combination works. T provides the structure and the reproducibility. The LLM provides the speed and the accessibility. And the context you invest provides the accuracy. Together, they make best practices achievable for teams that would never have adopted them otherwise.

16.9 A note on literate programming

This book has focused on the plumbing of reproducibility: environments, packages, pipelines, and automation. But there is another dimension we have not covered in depth: literate programming.

Literate programming is the practice of combining code, prose, and outputs into a single document. Tools like Quarto, R Markdown, and Jupyter notebooks (Kluyver et al. 2016) make this possible. The idea is that the analysis and its documentation are the same thing. You cannot have one without the other.

This book itself was written with Quarto. Every code example runs. Every output is generated fresh on each build. This is literate programming in action.

If you are interested in this approach, there are excellent resources available. The Quarto documentation at quarto.org[1] is a great starting point. For R Markdown, "R Markdown: The Definitive Guide" by Xie, Allaire, and Grolemund is comprehensive. The principles you have learned in this book apply directly: pin your

[1] https://quarto.org

environment with Nix, run your document rendering in CI, and your reports become just as reproducible as your analyses.

16.10 Final thoughts

Reproducibility is not a destination. It is a practice. You will never have a perfectly reproducible workflow because requirements change, tools evolve, and you learn new things. The goal is not perfection but continuous improvement.

Start where you are. Use what you have. Do what you can.

If this book has given you one new tool or one new idea, it has done its job. If it has changed how you think about your work, even better. And if you go on to share these practices with your colleagues and build a culture of reproducibility in your team, then you have not only learned but taught. That is the highest compliment.

Good luck. Stay curious. Build things that last.

References

Baker, Monya. 2016. "1,500 Scientists Lift the Lid on Reproducibility." *Nature* 533 (7604): 452–54. https://doi.org/10.1038/533452a.

Brodeur, Abel, D. Mikola, Nicholas Cook, et al. 2026. "Reproducibility and Robustness of Economics and Political Science Research." *Nature* 652: 151–56. https://doi.org/10.1038/s41586-026-10251-x.

Chen, Alyssia, Carol Wong, Bonita Sharif, and Anthony Peruma. 2025. "Exploring Code Comprehension in Scientific Programming: Preliminary Insights from Research Scientists." https://arxiv.org/abs/2501.10037.

Dolstra, Eelco. 2006. "The Purely Functional Software Deployment Model." PhD thesis, Utrecht University. https://edolstra.github.io/pubs/phd-thesis.pdf.

Dolstra, Eelco, Merijn De Jonge, and Eelco Visser. 2004. "Nix: A Safe and Policy-Free System for Software Deployment." In *18th Large Installation System Administration Conference*, 79–92.

Kluyver, Thomas, Benjamin Ragan-Kelley, Fernando Pérez, Brian E. Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, et al. 2016. "Jupyter Notebooks – a Publishing Format for Reproducible Computational Workflows." In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, edited by Fernando Loizides and Birgit Schmidt, 87–90. IOS Press. https://doi.org/10.3233/978-1-61499-649-

1-87.

Köster, Johannes, and Sven Rahmann. 2012. "Snakemake – a Scalable Bioinformatics Workflow Engine." *Bioinformatics* 28 (19): 2520–22. https://doi.org/10.1093/bioinformatics/bts480.

Landau, William Michael. 2021. "The Targets r Package: A Dynamic Make-Like Function-Oriented Pipeline Toolkit for Reproducibility and High-Performance Computing." *Journal of Open Source Software* 6 (57): 2959.

Lessig, Lawrence. 2006. *Code: Version 2.0*. New York: Basic Books.

Malka, Julien, Stefano Zacchiroli, and Théo Zimmermann. 2024. "Reproducibility of Build Environments Through Space and Time." In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 97–101. ICSE-NIER'24. ACM. https://doi.org/10.1145/3639476.3639767.

———. 2025. "Does Functional Package Management Enable Reproducible Builds at Scale? Yes." In *22nd International Conference on Mining Software Repositories*. Ottawa, Canada. https://hal.science/hal-04913007.

Norman, Donald A. 2013. *The Design of Everyday Things*. Revised and Expanded. New York: Basic Books.

North, Douglass C. 1990. *Institutions, Institutional Change and Economic Performance*. Cambridge: Cambridge University Press.

Open Science Collaboration. 2015. "Estimating the Reproducibility of Psychological Science." *Science* 349 (6251): aac4716. https://doi.org/10.1126/science.aac4716.

Peng, Roger D. 2011. "Reproducible Research in Computational Science." *Science* 334 (6060): 1226–27.

Pineau, Joelle, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché-Buc,

Emily Fox, and Hugo Larochelle. 2021. "Improving Reproducibility in Machine Learning Research (a Report from the NeurIPS 2019 Reproducibility Program)." *Journal of Machine Learning Research* 22 (164): 1–20. https://jmlr.org/papers/v22/20-303.html.

Stodden, Victoria, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. 2016. "Enhancing Reproducibility for Computational Methods." *Science* 354 (6317): 1240–41. https://doi.org/10.1126/science.aah6168.

Thaler, Richard H., and Cass R. Sunstein. 2008. *Nudge: Improving Decisions about Health, Wealth, and Happiness*. New Haven: Yale University Press.